LATTICE BASED CRYPTOGRAPHY

Notes For Year 2 Undergraduate

Author
Molly Shi
University College London
June 2026

Contents

1 Factor Groups & Rings **4**

1.1 Factor Groups . . . 4
1.1.1 Normal Groups . . . 4
1.1.2 Factor Groups . . . 6
1.2 Factor Rings . . . 11
1.2.1 Rings & Subrings . . . 11
1.2.2 Ideals . . . 12
1.2.3 Factor Rings . . . 14
1.2.4 Prime Ideals and Maximal Ideals . . . 20
1.3 Extra . . . 24
1.3.1 Convolution Polynomial Rings . . . 24

2 Lattices **26**

2.1 Motivation . . . 26
2.2 Basics of Lattices . . . 27
2.2.1 Definition of Lattices . . . 27
2.2.2 Propositions of Lattices . . . 29
2.2.3 An Alternative Characterisation of Lattices . . . 36
2.2.4 A Digression on Linear Independence . . . 39
2.2.5 Finding a Basis from a Generating Set . . . 40
2.3 Operations on Lattices . . . 42
2.3.1 Sums & Intersections . . . 42
2.3.2 The Dual of a Lattice . . . 43
2.3.3 Duality of Sums & Intersections . . . 48

3 Latticed-Based Cryptography **50**

3.1 Short Vectors in Lattices . . . 50
3.1.1 Fundamental Lattice Problems . . . 50
3.1.2 Hermite's Theorem & Minkowski's Theorem . . . 52
3.1.3 The Gaussian Heuristic . . . 55
3.2 Babai's Algorithm . . . 58
3.3 Cryptosystems based on Hard Lattice Problems . . . 62
3.4 The GGH Public Key Cryptosystem . . . 64
3.5 NTRU Public Key Cryptosystem . . . 70
3.5.1 Convolution Polynomial Rings . . . 70
3.5.2 NTRUEncrypt . . . 72
3.5.3 Mathematical Problems for NTRUEncrypt . . . 77
3.5.4 NTRUEncrypt as a Lattice-Based Cryptosystem . . . 77
3.6 Lattice Reduction Algorithms . . . 79
3.6.1 Gaussian Lattice Reduction in Dimension 2 . . . 79
3.6.2 The LLL Lattice Reduction Algorithm . . . 81
3.7 Application of LLL to Cryptanalysis . . . 90
3.7.1 Application to GGH Cryptosystem . . . 90
3.7.2 Application to NTRUEncrypt . . . 91

References **94**

A Python Code **95**

A.1 GGH Cryptosystem: Worked Example 95

A.2 GGH Cryptosystem: Extension 98

A.3 NTRU Cryptosystem: Worked Example 111

A.4 Gaussian Lattice Reduction Algorithm 123

A.5 LLL Lattice Reduction Algorithm 124

1 Factor Groups & Rings

1.1 Factor Groups

1.1.1 Normal Groups

Motivation: If H is a subgroup of a group G, it is not always true that $aH = Ha$ for all $a \in G$. But there are certain situations where this does hold and are important in the theory of groups.

Definition 1.1. A subgroup H of a group G is called a *normal* subgroup of G if $aH = Ha$ for $\forall a \in G$. Denoted by $H \triangleleft G$.

Remark 1.2.

1. $aH = \{ah : h \in H\}$ is the *left* coset of subgroup H in group G. We used it in the proof of Lagrange's Theorem in *MATH0006 Algebra 2*

   $Ha = \{ha : h \in H\}$ is the *right* coset of subgroup H in group G.

2. H is normal in $G \nRightarrow ah = ha, \forall a \in G, h \in H$ (It implies the commutativity of subgroup H)

   Instead, the definition means that for any $a \in G$ and $h \in H$, $\exists h', h'' \in H$ such that $ah = h'a$ and $ha = ah''$

Theorem 1.3. A subgroup H is normal in group $G \iff xHx^{-1} \subseteq H$ for all $x \in G$

Proof. ($\implies$) Assume H is normal in G. Then for any $x \in G$ and $h \in H$ there is an $h' \in H$ such that $xh = h'x$, by Definition 1.1.

Since x^{-1} exists in group G, multiplying both sides by x^{-1} on the right gives:

$$xhx^{-1} = h' \in H$$

Since $xHx^{-1} = \{xhx^{-1} : h \in H\}$, it follows that:

$$xHx^{-1} \subseteq H, \forall x \in G$$

($\impliedby$) Assume that $xHx^{-1} \subseteq H$ for all $x \in G$. We can prove $aH = Ha$ for any $a \in G$ from two directions: $aH \subseteq Ha$ and $Ha \subseteq aH$.

For any $a \in G$, a^{-1} exists in group G.

- *Part 1: Prove $aH \subseteq Ha$*

  Let $x = a$. Our assumption gives:

$$aHa^{-1} \subseteq H$$

This implies that for any element in the set on the left, it must land in H. That is:

$$\forall h \in H,\ \exists h' \in H \quad \text{such that} \quad aha^{-1} = h'$$

Multiplying both sides by a on the right yields:

$$ah = h'a$$

Since $h' \in H$, the element $h'a$ belongs to the right coset Ha. Because this holds for all $h \in H$, we conclude:

$$aH \subseteq Ha$$

- *Part 2: Prove $Ha \subseteq aH$*
  Letting $x = a^{-1}$, our assumption gives:

$$a^{-1}H(a^{-1})^{-1} \subseteq H$$

  Since $(a^{-1})^{-1} = a$ and by similar steps in Part 1 above, this yields:

$$Ha \subseteq aH$$

In conclusion, $aH \subseteq Ha$ and $Ha \subseteq aH$, then $aH = Ha$ for all $a \in G$, meaning H is normal in G. □

Example 1.4 ([1, pp.179])**.**

1. Every subgroup H of an *Abelian* group G is normal.
   In this case, $ah = ha, \forall a \in G, h \in H$.
2. $A_n \triangleleft S_n$, where the alternating group A_n is the group of even permutations on $\{1, \ldots, n\}$.
3. $SL(2, \mathbb{R}) \triangleleft GL(2, \mathbb{R})$, where $SL(2, \mathbb{R})$ is the group of 2×2 real matrices with determinant 1 and $GL(2, \mathbb{R})$ is the group of 2×2 real matrices with nonzero determinant.

1. It is obviously true from the definition of Abelian groups.
2. *Proof.* (*Prove by Definition 1.1*)
   For any $\sigma \in S_n$ and $\tau \in A_n$, we need to find $\tau' \in A_n$ such that $\sigma\tau = \tau'\sigma$. Take $\tau' = \sigma\tau\sigma^{-1}$.
   Since $sign(\tau) = 1$ and $sign(\sigma) = sign(\sigma^{-1}) = \pm 1$, then:

$$sign(\tau') = sign(\sigma\tau\sigma^{-1}) = sign(\sigma)sign(\tau)sign(\sigma^{-1}) = 1 \cdot (\pm 1)^2 = 1$$

   Thus:

$$\tau' \in A_n$$

Then:

$$\sigma\tau = (\sigma\tau\sigma^{-1})\sigma = \tau'\sigma$$

Since this holds for all $\sigma \in S_n$ and $\tau \in A_n$, we have $\sigma A_n = A_n\sigma$ for all $\sigma \in S_n$, so $A_n \triangleleft S_n$. □

3. *Proof.* (*Prove by Theorem 1.3*)

Let $X \in GL(2,\mathbb{R})$ and $H \in SL(2,\mathbb{R})$. Then:

$$\det(XHX^{-1}) = \det(X)\det(H)\det(X^{-1}) = \det(X)\cdot 1\cdot \det(X)^{-1} = 1$$

So $XHX^{-1} \in SL(2,\mathbb{R}) \Rightarrow XSL(2,\mathbb{R})X^{-1} \subseteq SL(2,\mathbb{R})$ for all $X \in GL(2,\mathbb{R})$.

By Theorem 1.3, $SL(2,\mathbb{R}) \triangleleft GL(2,\mathbb{R})$. □

1.1.2 Factor Groups

Motivation: Why normal subgroups are important? When subgroup H is normal in group G, then the set of left(or right) cosets of H in G is itself a group.

Definition 1.5. Let G be a group and let $H \triangleleft G$. The set

$$G/H = \{aH \mid a \in G\}$$

is called the *factor group* (or *quotient group*) of G by H, under the operation

$$(aH)(bH) = abH$$

Remark 1.6. To complete the definition, we need to show the operation $(aH)(bH) = abH$ is well-defined and closed.
We also need to show G/H satisfies the group axioms. (i.e. associativity, identity and inverse of each element)

Theorem 1.7 (Hölder,1889)**.** Let G be a group and let $H \triangleleft G$. Then G/H is a group under the operation $(aH)(bH) = abH$.

Before proving this theorem, we may first prove 2 claims:

Claim 1. *If H is a subgroup of G, $aH = a'H$ and $bH = b'H$ for some $a, a', b, b' \in G$, then $a' = ah_1$ and $b' = bh_2$ for some $h_1, h_2 \in H$.*

Proof. Suppose $aH = a'H$ and $bH = b'H$ for some $a, a', b, b' \in G$.
Then $a' = a'e \in a'H = aH$, where e is the identity of H.
By definition of $aH = \{ah : h \in H\}$, we have $a' = ah_1$ for some $h_1 \in H$.
By the same argument applied to $bH = b'H$, we have $b' = bh_2$ for some $h_2 \in H$. □

Claim 2. *H is a group, $h \in H \Rightarrow hH = H$*

Proof. First, suppose $x \in hH$, then $x = hy$ for some $y \in H$.
Since $h, y \in H$ and the operation of H is closed, then $x \in H$
Thus, $hH \subseteq H$
Next, suppose $x \in H$, then choose $y := h^{-1}x$.
Since $h \in H$ and H is a subgroup, $h^{-1} \in H$.
Since $x \in H$ and H is closed, $h^{-1}x = y \in H$.
This gives: $x = h(h^{-1}x) = hy$, with $y \in H \Rightarrow x \in hH$
Thus, $H \subseteq hH$
Therefore, $hH = H$. □

Proof. **Step 1: Well-definedness.**
Suppose $aH = a'H$ and $bH = b'H$ for some $a, a', b, b' \in G$. We must show $abH = a'b'H$.
Then $a' = ah_1$ and $b' = bh_2$ for some $h_1, h_2 \in H$.(Claim 1 above)
Then:

$$a'b' = ah_1bh_2$$

Since $H \triangleleft G$, by Definition 1.1, there exists $h_1' \in H$ such that $h_1b = bh_1'$. Thus:

$$a'b' = abh_1'h_2$$

Since $h_1'h_2 \in H$ (as H is closed), we have $a'b' = ab \cdot h$ where $h := h_1'h_2 \in H$.
Therefore $a'b'H = (abh)H = ab(hH) = abH$, as $hH = H$ (Claim 2 above).
Thus the operation $(aH)(bH) = abH$ is well-defined.

Step 2: Closure.
Since $a, b \in G \Rightarrow ab \in G$ by closure of G, we have $abH \in G/H$ for any $aH, bH \in G/H$.

Step 3: Group axioms.

- *Associativity:* For any $aH, bH, cH \in G/H$, we want to show $((aH)(bH))(cH) = abcH = (aH)((bH)(cH))$.

  Left side:

$$\begin{aligned}((aH)(bH))(cH) &= (abH)(cH) \\ &= \{xy : x \in abH, y \in cH\} \\ &= \{abh_1 \cdot ch_2 : h_1, h_2 \in H\}\end{aligned}$$

  Since $H \triangleleft G$, by Definition 1.1, $\exists h_1' \in H$ such that $h_1c = ch_1'$. Thus:

$$abh_1ch_2 = abch_1'h_2$$

  Since H is closed, $h_1'h_2 \in H$. Therefore:

$$((aH)(bH))(cH) = \{abch : h \in H\} = abcH$$

  Right side:

$$(aH)((bH)(cH)) = (aH)(bcH)$$

$$= \{xy : x \in aH, y \in bcH\}$$
$$= \{ah_1 \cdot bch_2 : h_1, h_2 \in H\}$$

Since $H \triangleleft G$, by Definition 1.1, for $bc \in G, \exists h_1' \in H$ such that $h_1 bc = h_1(bc) = (bc)h_1' = bch_1'$.

Thus:

$$ah_1 bch_2 = abch_1' h_2$$

Since H is closed, $h_1' h_2 \in H$. Therefore:

$$(aH)((bH)(cH)) = \{abch : h \in H\} = abcH$$

Hence $((aH)(bH))(cH) = abcH = (aH)((bH)(cH))$.

- *Identity:* We claim that $H = eH$ is the identity of G/H, where e is the identity of G.

  For any $aH \in G/H$:

  $$(aH)(eH) = aeH = aH$$
  $$(eH)(aH) = eaH = aH$$

  so eH is the identity element of G/H.

- *Inverse:* For any $aH \in G/H$, since $a^{-1} \in G$ we have $a^{-1}H \in G/H$. Then:

  $$(aH)(a^{-1}H) = aa^{-1}H = eH = H$$
  $$(a^{-1}H)(aH) = a^{-1}aH = eH = H$$

  so $(aH)^{-1} = a^{-1}H$.

Hence G/H is a group.

□

Example 1.8. Let $4\mathbb{Z} = \{0, \pm 4, \pm 8, \pm 12, \ldots\}$
The factor group $\mathbb{Z}/4\mathbb{Z}$ consists of the left cosets of $4\mathbb{Z}$ in $\mathbb{Z}$:

$$0 + 4\mathbb{Z} = 4\mathbb{Z} = \{0, \pm 4, \pm 8, \ldots\},$$

$$1 + 4\mathbb{Z} = \{\ldots, -7, -3, 1, 5, 9, \ldots\},$$

$$2 + 4\mathbb{Z} = \{\ldots, -6, -2, 2, 6, 10, \ldots\},$$

$$3 + 4\mathbb{Z} = \{\ldots, -5, -1, 3, 7, 11, \ldots\}.$$

Every integer k may be written as

$$k = 4q + r, 0 \leq r < 4,$$

so

$$k + 4\mathbb{Z} = r + 4\mathbb{Z}.$$

Hence,

$$\mathbb{Z}/4\mathbb{Z} = \{0 + 4\mathbb{Z}, 1 + 4\mathbb{Z}, 2 + 4\mathbb{Z}, 3 + 4\mathbb{Z}\}.$$

Addition is defined by

$$(a + 4\mathbb{Z}) + (b + 4\mathbb{Z}) = (a + b) + 4\mathbb{Z}.$$

The operation table is

$+$	$0+4\mathbb{Z}$	$1+4\mathbb{Z}$	$2+4\mathbb{Z}$	$3+4\mathbb{Z}$
$0+4\mathbb{Z}$	$0+4\mathbb{Z}$	$1+4\mathbb{Z}$	$2+4\mathbb{Z}$	$3+4\mathbb{Z}$
$1+4\mathbb{Z}$	$1+4\mathbb{Z}$	$2+4\mathbb{Z}$	$3+4\mathbb{Z}$	$0+4\mathbb{Z}$
$2+4\mathbb{Z}$	$2+4\mathbb{Z}$	$3+4\mathbb{Z}$	$0+4\mathbb{Z}$	$1+4\mathbb{Z}$
$3+4\mathbb{Z}$	$3+4\mathbb{Z}$	$0+4\mathbb{Z}$	$1+4\mathbb{Z}$	$2+4\mathbb{Z}$

Therefore,

$$\mathbb{Z}/4\mathbb{Z} \cong \mathbb{Z}_4.$$

Remark 1.9. In general case, for any $n > 0$, $\mathbb{Z}/n\mathbb{Z} \cong \mathbb{Z}_n$.

Example 1.10. Let $G = \mathbb{Z}_{18}$, and let $H = \{0, 6, 12\}$.
Since G is abelian, every subgroup is normal, so $H \triangleleft G$.
The distinct cosets of H are

$$0 + H, \quad 1 + H, \quad 2 + H, \quad 3 + H, \quad 4 + H, \quad 5 + H.$$

Indeed,

$$6 + H = H,$$

so there are only six distinct cosets. Hence,

$$G/H = \{0 + H, 1 + H, 2 + H, 3 + H, 4 + H, 5 + H\}.$$

The group operation is

$$(a + H) + (b + H) = (a + b) + H.$$

For example,

$$(5 + H) + (4 + H) = 9 + H.$$

Since $9 = 3 + 6$ and $6 \in H$, we have

$$9 + H = 3 + 6 + H = 3 + H.$$

Therefore,

$$(5+H)+(4+H)=3+H.$$

Notice that

$$3+H=\{3,9,15\}.$$

Thus the coset $3+H$ contains three elements.
However,

$$(3+H)+(3+H)=6+H=0+H.$$

Hence the order of the element $3+H$ in the quotient group is

$$|3+H|=2.$$

This illustrates that *the number of elements in a coset* is different from *the order of that coset as an element of the factor group.*

Remark 1.11. When we pass from a group G to the factor group G/H, every element of the normal subgroup H becomes the identity element of G/H.
Consequently, two elements $a, b \in G$ represent the same element of G/H precisely when

$$aH=bH,$$

or equivalently,

$$a^{-1}b \in H.$$

Thus, in the quotient group, any difference between two elements that lies in H is ignored.
For example, in $\mathbb{Z}/4\mathbb{Z}$, every multiple of 4 becomes 0, so

$$5+4\mathbb{Z}=1+4\mathbb{Z}, \qquad 6+4\mathbb{Z}=2+4\mathbb{Z}.$$

Similarly, in Example 8,

$$9+H=3+H, \qquad 15+H=3+H,$$

because

$$9-3=6\in H, \qquad 15-3=12\in H.$$

Algebraists often describe the process of creating G/H as *killing* the subgroup H.

1.2 Factor Rings

1.2.1 Rings & Subrings

Recall: Definition of Ring

Definition 1.12. A non-empty set R with two closed binary operations $+$ and $\cdot$ is a ring if

1. $(R, +)$ is an abelian group.
2. multiplication $\cdot$ is associative.
3. there exists $1 \in R$ such that $1 \cdot a = a \cdot 1 = a \quad$ for all $a \in R$.
4. the distributivity laws hold:

$$(a + b) \cdot c = a \cdot c + b \cdot c \quad \text{and} \quad a \cdot (b + c) = a \cdot b + a \cdot c$$

for all $a, b, c \in R$.

Remark 1.13.

1. A ring is *commutative* if the *multiplication* is commutative.
2. For nonzero a in a commutative ring R, it is a *unit* if $a^{-1} \in R$.

Example 1.14.

1. $\mathbb{Z}$ under ordinary addition and multiplication is a commutative ring.
2. $n\mathbb{Z}$ is *not* a ring, since it has no multiplicative identity.
3. $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ under addition and multiplication *modulo* n is a commutative ring.
4. $\mathbb{Z}[x]$ is the set of all polynomials in the variable x with integer coefficient. It is a commutative ring under ordinary addition and multiplication, with identity $f(x) = 1$.
5. Similar to example above, $\mathbb{R}[x]$ is the set of all polynomials in the variable x with real coefficient. It is a commutative ring under ordinary addition and multiplication, with identity $f(x) = 1$.
6. $M_2(\mathbb{Z})$ is the set of 2×2 matrices with integer entries.It is a *non-commutative* ring under matrix addition and multiplication, with identity I_2.

Definition 1.15. A subset S of a ring R is a *subring* of R if S is itself a ring with the operations of R. (*It contains multiplicative identity.*)

Theorem 1.16. A non-empty subset S of a ring R is a subring if and only if S is closed under subtraction and multiplication, that is $a - b, ab \in S, \forall a, b \in S$.

Example 1.17.

1. R are subrings of any ring R.
2. $n\mathbb{Z}$ (for $n \in \mathbb{Z}, |n| \neq 1$) is not subring of $\mathbb{Z}$, since it doesn't have any multiplicative identity.
3. $\{0, 2, 4\}$ is a subring of $\mathbb{Z}_6$.

   One thing worth to notice here it that 1 is the identity of $\mathbb{Z}_6$, but 4 is the identity of $\{0, 2, 4\}$:

   - $0 \cdot 4 = 0 \pmod 6$
   - $2 \cdot 4 = 8 = 2 \pmod 6$
   - $4 \cdot 4 = 16 = 4 \pmod 6$
4. *Gaussian integers* $\mathbb{Z}[i] = \{a + bi : a, b \in \mathbb{Z}\}$ is a subring of $\mathbb{C}$.

1.2.2 Ideals

Motivation: Normal subgroups permit us to construct factor groups, then how can we construct factor rings?

Definition 1.18. A non-empty subset A of a ring R is an ideal if

1. $a - b \in A$ whenever $a, b \in A$
2. ra and ar are in A whenever $a \in A$ and $r \in R$.

Remark 1.19.

1. A subring A is an ideal of R if $rA = \{ra : a \in A\} \subseteq A$ and $Ar = \{ar : a \in A\} \subseteq A$ for all $r \in R$. (i.e. A "*absorbs*" elements from R)
2. An ideal A of R is a *proper* ideal of R if A is a proper subset of R (i.e. $A \subset R$ and $A \neq R$)
3. For an ideal A of ring R, although there exists the multiplicative identity $1 \in R$, it does not necessarily exist in A. *If* $1 \in A$, *then* $A = R$. (See Claim 3 later).

Proposition 1.20. Let R be a commutative ring and let $a \in R$. Then the set $\langle a \rangle = \{ra : r \in R\}$ is an ideal of R, called the *principal ideal generated by* a.

Proof. (i) Since $a = 1 \cdot a$ and $1 \in R$, then $a \in \langle a \rangle$,so it is not empty.

Next, $x \in \langle a \rangle \Rightarrow x = ra$ for some $r \in R$. As $a, r \in R$, $x \in R \Rightarrow \langle a \rangle \subseteq R$.

(ii) Let $x, y \in \langle a \rangle$, then $\exists r_1, r_2 \in R$ such that $x = r_1 a, y = r_2 a$.

Thus, $x - y = r_1 a - r_2 a = (r_1 - r_2)a$, where $r_1 - r_2 \in R$.

Hence, $x - y \in \langle a \rangle$.

(iii) Let $x \in \langle a \rangle, y \in R$, then $\exists r \in R$ such that $x = ra$.

Thus, $xy = yx = y(ra) = (yr)a$. Since $yr \in R$, $xy = yx \in \langle a \rangle$.

□

Proposition 1.21.

1. Let R be a commutative ring and let $a_1, a_2, \ldots a_n \in R$. Then $I = \langle a_1, a_2, \ldots, a_n \rangle = \{r_1 a_1 + r_2 a_2 + \cdots + r_n a_n : r_1, r_2, \ldots, r_n \in R\}$ is an ideal of R, called the *ideal generated by* $a_1, a_2, \ldots, a_n$.
2. Let J be any ideal of R that contains $a_1, a_2, \ldots, a_n$, then $I \subseteq J$. (i.e. I is the smallest ideal of R that contains $a_1, \ldots, a_n$)

Proof. 1. (i) Since $1 \in R$, we have $a_1 = 1 \cdot a_1 + 0 \cdot a_2 + \cdots + 0 \cdot a_n \in I$,so I is not empty.

Next, let $x \in I$, so $x = r_1 a_1 + \cdots + r_n a_n$ for some $r_1, \ldots, r_n \in R$.

Since $r_i, a_i \in R, \forall 1 \leq i \leq n$, then $r_i a_i \in R$. Since R is closed under addition, $x \in R$.

Hence $I \subseteq R$.

(ii) Let $x, y \in I$, then $x = r_1 a_1 + \cdots + r_n a_n$ and $y = s_1 a_1 + \cdots + s_n a_n$ for some $r_1, \ldots, r_n, s_1, \ldots, s_n \in R$.

Thus, $x - y = (r_1 - s_1)a_1 + \cdots + (r_n - s_n)a_n$, where each $r_i - s_i \in R$. Hence $x - y \in I$.

(iii) Let $x \in I, y \in R$, then $x = r_1 a_1 + \cdots + r_n a_n$ for some $r_1, \ldots, r_n \in R$.

Thus, $xy = yx = y(r_1 a_1 + \cdots + r_n a_n) = (yr_1)a_1 + \cdots + (yr_n)a_n$.

Since each $yr_i \in R$, $xy = yx \in I$.

By (i), (ii), (iii), I is an ideal of R.

2. Let J be any ideal of R containing $a_1, \ldots, a_n$ and $x \in I$, then $x = r_1 a_1 + \cdots + r_n a_n$ for some $r_1, \ldots, r_n \in R$.

Since J is an ideal and $a_i \in J$ for each i, the absorption property gives $r_i a_i \in J$ for each $i = 1, \ldots, n$.

Since J is an ideal, it is in particular an additive subgroup of R, hence closed under addition.

Then, $x = r_1 a_1 + r_2 a_2 + \cdots + r_n a_n \in J$, so $I \subseteq J$.

□

Remark 1.22. In Proposition 1.20, the *commutativity* of R is necessary. Otherwise, $xy = (ra)y \neq (yr)a$ or $(ry)a$, so (iii) may not hold in this case. Similar statement holds for Proposition 1.21.

Example 1.23.

1. - $\{0\}r = r\{0\} = \{0\}$ and $rR = Rr = R$ for any $r \in R$
   - $\{0\}$ and R are subrings of R by Example 1.17

   Hence, $\{0\}$ and R are ideals of R for any ring R. Also, $\{0\}$ is called the *trivial* ideal of R.

2. Since $\mathbb{Z}$ is a commutative ring, $n \in \mathbb{Z}$, we can find:

$$\langle n \rangle = \{kn : k \in \mathbb{Z} = \{0, \pm n, \pm 2n, \dots\} = n\mathbb{Z}$$

   By Proposition 1.20, $n\mathbb{Z}$ is an ideal of $\mathbb{Z}$.

3. Since $\mathbb{R}[x]$ is a commutative ring and let A denote its subset of all polynomials with constant term 0, then

$$\begin{aligned} A &= \{0 + r_1x + r_2x^2 + r_3x^3 + \cdots \mid r_1, r_2, r_3 \cdots \in \mathbb{R}\} \\ &= \{(r_1 + r_2x + r_3x^2 + \dots)x \mid r_1, r_2, r_3 \cdots \in \mathbb{R}\} \\ &= \{rx \mid r \in \mathbb{R}[[x]]\} \\ &= \langle x \rangle \subseteq \mathbb{R}[x] \end{aligned}$$

   By Proposition 1.20, A is an ideal of $\mathbb{R}[x]$.

4. Since $\mathbb{Z}[x]$ is a commutative ring and let I denote its subset of all polynomials with even constant terms, then

$$\begin{aligned} I &= \{2k_0 + k_1x + k_2x^2 + \cdots \mid k_i \in \mathbb{Z}, \forall i = 0, 1, 2 \dots\} \\ &= \{(k_1 + k_2x + \dots) \cdot x + k_0 \cdot 2 \mid k_i \in \mathbb{Z}, \forall i = 0, 1, 2 \dots\} \\ &= \{\alpha \cdot x + \beta \cdot 2 \mid \alpha, \beta \in \mathbb{Z}[x]\} \\ &= \langle x, 2 \rangle \subseteq \mathbb{Z}[x] \end{aligned}$$

   By Proposition 1.21, I is an ideal of $\mathbb{Z}[x]$.

1.2.3 Factor Rings

Motivation: Let R be a ring and let A be an ideal of R. Since R is an abelian group under addition, A is a normal subgroup of $(R, +)$ (Recall: Example 1.4 (1)). So we may form the factor group

$$R/A = \{r + A : r \in R\},$$

whose elements are the cosets of A in R.

The natural question is whether this factor group can also be given a multiplication so that it becomes a ring.

Since each coset $r + A$ represents all elements of the form $r + a$ with $a \in A$, we would like multiplication to depend only on the cosets and not on the particular representatives

chosen. The most natural definition is

$$(r + A)(s + A) = rs + A.$$

However, this definition is only meaningful if it is *well-defined* (i.e. replacing r or s by different representatives of the same coset must produce the same result). This is precisely where the ideal property is needed: if A is merely a subring, the product may depend on the chosen representatives, whereas if A is an ideal, multiplication is well-defined. This leads to the notion of a *factor ring*.

Definition 1.24. Let R be a ring and let A be an ideal of R. The *factor ring* (or *quotient ring*) of R by A is the set

$$R/A = \{r + A : r \in R\},$$

equipped with the operations

$$(r + A) + (s + A) = (r + s) + A$$

and

$$(r + A)(s + A) = rs + A.$$

Theorem 1.25 (Existence of Factor Rings)**.** Let R be a ring and let A be a subring of R. Then the set of cosets

$$R/A = \{r + A : r \in R\}$$

forms a ring under the operations

$$(s + A) + (t + A) = (s + t) + A \quad \text{and} \quad (s + A)(t + A) = st + A$$

if and only if A is an ideal of R.

Proof. Since $(R, +)$ is an abelian group and A is a normal subgroup of $(R, +)$, the set of cosets R/A is already an abelian group under addition $(s+A)+(t+A) = s+t+A, \forall s, t \in R$ by Theorem 1.7.

Moreover, once we know multiplication is a well-defined binary operation, existence of identity, the associativity and distributivity of multiplication over addition are trivial:

- *Existence of identity:*

  Since R is a ring, then there exists $1 \in R$ such that $1 \cdot r = r \cdot 1 = r, \forall r \in R$, so $1 + A \in R/A$.

  Since $(1 + A)(r + A) = 1 \cdot r + A = r + A$ and $(r + A)(1 + A) = r \cdot 1 + A = r + A$ for all $r + A \in R/A$, so $1 + A$ is the multiplicative identity of R/A.

- *Associativity:*

$$((r+A)(s+A))(t+A) = (rs)t + A = r(st) + A = (r+A)((s+A)(t+A))$$

  where associativity follows from associativity in R.

- *Distributivity:*

$$\begin{aligned}(r+A)((s+A)+(t+A)) &= (r+A)((s+t)+A)\\ &= r(s+t)+A\\ &= (rs+rt)+A\\ &= (rs+A)+(rt+A)\\ &= (r+A)(s+A)+(r+A)(t+A)\end{aligned}$$

  where distributivity follows from distributivity in R. The right distributive law is analogous.

Thus it remains to show that multiplication is well-defined if and only if A is an ideal of R.

($\Longleftarrow$) Suppose A is an ideal of R. Let

$$s + A = s' + A \quad \text{and} \quad t + A = t' + A.$$

Then $s' = s + a$ and $t' = t + b$ for some $a, b \in A$, by Claim 1 (applied to additive group).

Hence, $s't' = (s+a)(t+b) = st + at + sb + ab$ by Distributivity Law of ring R.

Since A is an ideal:

- $at \in A$, since $a \in A$ and $t \in R$ (right absorption)
- $sb \in A$, since $b \in A$ and $s \in R$ (left absorption)
- $ab \in A$, since $a, b \in A$ and A is a subring (closure)

Therefore $at + sb + ab \in A$, and so:

$$s't' + A = st + (at + sb + ab) + A = st + A$$

since $at + sb + ab + A = A$ by Claim 2.

Thus multiplication is well-defined.

($\Longrightarrow$) (Contrapositive) Suppose A is a subring of R but not an ideal. Then there exist $a \in A$ and $r \in R$ such that $ar \notin A$ or $ra \notin A$. Without loss of generality, assume $ar \notin A$.

Since $a \in A$ and A is a subgroup of $(R, +)$, by Claim 2 (applied to the additive group), $a + A = A = 0 + A$. That is, a and 0 represent the same coset.

Now compute the product of the coset $a+A = 0+A$ with $r+A$ using each representative:

$$(a+A)(r+A) = ar + A$$

$$(0+A)(r+A) = 0r + A = 0 + A = A.$$

Since $ar \notin A$, we have $ar + A \neq A$, so:

$$(a + A)(r + A) \neq (0 + A)(r + A)$$

even though $a + A = 0 + A$.

Hence the product depends on the choice of representative, so multiplication is not well-defined.

Therefore R/A is a ring under the given operations if and only if A is an ideal of R. □

Remark 1.26. When R/A is a ring:

- The additive identity is $0 + A = A$
- The additive inverse of $r + A$ is $(-r) + A$

Example 1.27. Since $4\mathbb{Z}$ is an ideal of $\mathbb{Z}$, then $\mathbb{Z}/4\mathbb{Z} = \{r + 4\mathbb{Z} : r \in \mathbb{Z}\} = \{0 + 4\mathbb{Z}, 1 + 4\mathbb{Z}, 2 + 4\mathbb{Z}, 3 + 4\mathbb{Z}\}$ is a factor ring.
This ring follows the addition and multiplication modulo 4, as below:

$$(2 + 4\mathbb{Z}) + (3 + 4\mathbb{Z}) = (2 + 3) + 4\mathbb{Z} = 5 + 4\mathbb{Z} = 1 + 4\mathbb{Z}$$

$$(2 + 4\mathbb{Z})(3 + 4\mathbb{Z}) = (2 \cdot 3) + 4\mathbb{Z} = 6 + 4\mathbb{Z} = 2 + 4\mathbb{Z}.$$

Example 1.28. Since $\mathbb{Z}[i]$ is a commutative ring and $\langle 2 - i \rangle$ is the principal ideal generated by $2 - i$, $\mathbb{Z}[i]/\langle 2 - i \rangle = \{a + bi + \langle 2 - i \rangle \mid a, b \in \mathbb{Z}\}$ is a factor ring.
Now, we need to find out *what distinct cosets look like*:
Since $2 - i + \langle 2 - i \rangle = 0 + \langle 2 - i \rangle$, this means that when we are dealing with *coset representatives*, we can treat $0 = 2 - i \Rightarrow i = 2$. Thus,

$$a + bi + \langle 2 - i \rangle = a + b \cdot 2 + \langle 2 - i \rangle = c + \langle 2 - i \rangle,$$

where we take $c = a + 2b \in \mathbb{Z}$.
For example,

$$3 + 4i + \langle 2 - i \rangle = 3 + 4 \cdot 2 + \langle 2 - i \rangle = 11 + \langle 2 - i \rangle.$$

Next, how can we *further reduce the set of distinct coset representatives*?
Observe that $i = 2$, then $i^2 = 2^2 \Rightarrow -1 = 4 \Rightarrow 5 = 1 + 4 = 1 + (-1) = 0$, so 5 may be the additive identity in $\mathbb{Z}[i]/\langle 2 - i \rangle$.
Let $c(a + 2b) = r + 5k$ for some $k, r \in \mathbb{Z}, 0 \leq r \leq 4$, then:

$$a + bi + \langle 2 - i \rangle = c + \langle 2 - i \rangle = r + 5k + \langle 2 - i \rangle = r + \langle 2 - i \rangle.$$

Continue with same example above:

$$3 + 4i + \langle 2 - i \rangle = 11 + \langle 2 - i \rangle = 1 + 5 \cdot 2 + \langle 2 - i \rangle = 1 + \langle 2 - i \rangle.$$

Hence, every element of $\mathbb{Z}[i]/\langle 2-i\rangle$ is equal to one of the following cosets:

$$0+\langle 2-i\rangle, 1+\langle 2-i\rangle, 2+\langle 2-i\rangle, 3+\langle 2-i\rangle, 4+\langle 2-i\rangle$$

In this case, we need to prove *these five cosets are distinct*: It is sufficient to show that $1+\langle 2-i\rangle$ has additive order 5, because an element of order 5 generates 5 distinct multiples $0, 1, 2, 3, 4$ in the group, and these multiples are exactly our five cosets.
Since $5(1+\langle 2-i\rangle) = 5+\langle 2-i\rangle = 0+\langle 2-i\rangle$, $1+\langle 2-i\rangle$ has order 1 or 5.
Suppose $1+\langle 2-i\rangle$ has order 1, then $1+\langle 2-i\rangle = 0+\langle 2-i\rangle \Rightarrow 1 \in \langle 2-i\rangle$
$\Rightarrow 1 = (2-i)(a+bi)$ for some $a, b \in \mathbb{Z}$.
Taking modulus on both sides, $N(1) = 1 = N(2-i)N(a+bi) = 5(a^2+b^2)$, which forces $a^2+b^2 = \frac{1}{5}$ and it is impossible as $a, b \in \mathbb{Z}$, leading to a contradiction.
Therefore, $1+\langle 2-i\rangle$ has order 5, so the five cosets are distinct, and $\mathbb{Z}[i]/\langle 2-i\rangle = \{0+\langle 2-i\rangle, 1+\langle 2-i\rangle, 2+\langle 2-i\rangle, 3+\langle 2-i\rangle, 4+\langle 2-i\rangle\}$, which is essentially the same as $\mathbb{Z}_5$.
We can prove this by defining:

$$\phi : \mathbb{Z}[i]/\langle 2-i\rangle \to \mathbb{Z}_5, \qquad r+\langle 2-i\rangle \mapsto r. \quad (0 \le r \le 4)$$

This map is a well-defined bijection, and computation will show that ϕ respects addition and product.
Hence ϕ is a ring isomorphism, then $\mathbb{Z}[i]/\langle 2-i\rangle \cong \mathbb{Z}_5$.

Remark 1.29. Alternatively, this isomorphism can be obtained directly from the First Isomorphism Theorem for Rings, applied to a homomorphism defined on $\mathbb{Z}[i]$ itself (rather than on the quotient).
Define

$$\phi : \mathbb{Z}[i] \to \mathbb{Z}_5, \qquad \phi(a+bi) = a+2b \pmod 5.$$

ϕ **is a ring homomorphism:** Clearly $\phi((a+bi)+(c+di)) = \phi(a+bi)+\phi(c+di)$.
For products, since $2^2 \equiv -1 \pmod 5$ (i.e. 2 plays the role of i modulo 5),

$$(a+2b)(c+2d) = ac+2(ad+bc)+4bd \equiv (ac-bd)+2(ad+bc) \pmod 5,$$

which is exactly $\phi((a+bi)(c+di)) = \phi((ac-bd)+(ad+bc)i)$. So $\phi((a+bi)(c+di)) = \phi(a+bi)\phi(c+di)$.
ϕ **is surjective:** for any $r \in \mathbb{Z}_5$, $\phi(r+0i) = r$.
Computing $\ker\phi$**:** We claim $\ker\phi = \langle 2-i\rangle$.
($\supseteq$) $\phi(2-i) = 2+2(-1) = 0$, so $2-i \in \ker\phi$; since $\ker\phi$ is an ideal, $\langle 2-i\rangle \subseteq \ker\phi$.
($\subseteq$) Suppose $a+2b \equiv 0 \pmod 5$, say $a+2b = 5k$. Using $5 = (2-i)(2+i)$ (since $N(2-i) = 4+1 = 5$),

$$a+bi = (a+2b)-b(2-i) = 5k-b(2-i) = (2-i)(k(2+i)-b) \in \langle 2-i\rangle.$$

So $\ker\phi \subseteq \langle 2-i\rangle$.

Hence $\ker \phi = \langle 2-i \rangle$. By *The First Isomorphism Theorem for Rings*, since $\phi : \mathbb{Z}[i] \to \mathbb{Z}_5$ is a surjective ring homomorphism,

$$\mathbb{Z}[i]/\langle 2-i \rangle \cong \mathbb{Z}_5.$$

Example 1.30. Since $\mathbb{R}[x]$ is a commutative ring and $\langle x^2+1 \rangle = \{f(x)(x^2+1) \mid f(x) \in \mathbb{R}[x]\}$ is a principal ideal, then $\mathbb{R}[x]/\langle x^2+1 \rangle = \{g(x) + \langle x^2+1 \rangle \mid g(x) \in \mathbb{R}[x]\}$ is a factor ring.
If $g(x) \in \mathbb{R}[x]$, then $g(x) = p(x)(x^2+1) + r(x)$, where $p(x), r(x) \in \mathbb{R}[x]$ and $\deg(r(x)) < \deg(x^2+1) = 2$.
Thus, $\deg(r(x)) \leq 1 \Rightarrow$ let $r(x) = ax + b$ for some $a, b \in \mathbb{R}$, then:

$$g(x) + \langle x^2+1 \rangle = r(x) + p(x)(x^2+1) + \langle x^2+1 \rangle = ax + b + \langle x^2+1 \rangle,$$

since $\langle x^2+1 \rangle$ absorbs $p(x)(x^2+1)$ as it is an ideal.
Hence, $\mathbb{R}[x]/\langle x^2+1 \rangle = \{ax + b + \langle x^2+1 \rangle \mid a, b \in \mathbb{R}\}$.
Let's consider *the additive identity* in this factor ring: Since $x^2 + 1 + \langle x^2+1 \rangle = 0 + \langle x^2+1 \rangle$, the coset $x + \langle x^2+1 \rangle$ satisfies $(x + \langle x^2+1 \rangle)^2 = -1 + \langle x^2+1 \rangle$, so we can treat $x^2 = -1$ here.
Thus, if we multiply 2 element $ax + b + \langle x^2+1 \rangle$ and $cx + d + \langle x^2+1 \rangle$, we get:

$$\begin{aligned}(ax + b + \langle x^2+1 \rangle)(cx + d + \langle x^2+1 \rangle) &= (ax+b)(cx+d) + \langle x^2+1 \rangle \\ &= acx^2 + (ad+bc)x + bd + \langle x^2+1 \rangle \\ &= (ad+bc)x + (-ac+bd) + \langle x^2+1 \rangle\end{aligned}$$

Thus, the result is still in the form of $\alpha x + \beta + \langle x^2+1 \rangle$.
Moreover, since we take $x^2 = -1$ in this case, we may see that x corresponds to i. Indeed, define

$$\phi : \mathbb{R}[x]/\langle x^2+1 \rangle \to \mathbb{C}, \qquad ax + b + \langle x^2+1 \rangle \mapsto ai + b.$$

This map is a well-defined bijection, and the multiplication computation above shows ϕ respects products, since $(ad+bc)i + (bd-ac) = (ai+b)(ci+d)$. Also, a similar check shows ϕ respects addition.
Hence ϕ is a ring isomorphism, so $\mathbb{R}[x]/\langle x^2+1 \rangle$ is algebraically the same ring as $\mathbb{C}$.

GENERAL CASE: What if we take any finite terms $f(x) \in \mathbb{R}[x]$ with $\deg(f(x)) \geq 1$ instead of x^2+1, the factor ring $\mathbb{R}[x]/\langle f(x) \rangle$ will become now?
By similar steps for *division of polynomials*, we may have

$$\mathbb{R}[x]/\langle f(x) \rangle = \{r(x) + \langle f(x) \rangle \mid \deg(r(x)) < \deg(f(x)), r(x) \in \mathbb{R}[x]\},$$

which is the set of remainders of all polynomials in $\mathbb{R}[x]$ divided by $f(x)$.
As in the division algorithm, existence and uniqueness of $r(x)$ for each $g(x)$ guarantees that these coset representatives are *well-defined and pairwise distinct.*

Remark 1.31. Alternatively, apply the First Isomorphism Theorem to the *evaluation homomorphism* at i:

$$\phi : \mathbb{R}[x] \to \mathbb{C}, \qquad \phi(g(x)) = g(i).$$

ϕ is a ring homomorphism: evaluation homomorphisms always satisfy $\phi(g+h) = \phi(g)+\phi(h)$ and $\phi(gh) = \phi(g)\phi(h)$, since these identities hold pointwise for polynomial addition and multiplication.
ϕ is surjective: for any $a + bi \in \mathbb{C}$, take $g(x) = bx + a$; then $\phi(g) = bi + a$.
Computing $\ker\phi$**:** We claim $\ker\phi = \langle x^2 + 1\rangle$.
($\supseteq$) $\phi(x^2+1) = i^2 + 1 = 0$, so $x^2 + 1 \in \ker\phi$; since $\ker\phi$ is an ideal, $\langle x^2+1\rangle \subseteq \ker\phi$.
($\subseteq$) Suppose $g(i) = 0$. By the division algorithm (as established above), $g(x) = q(x)(x^2+1) + r(x)$ with $r(x) = ax + b$, $a, b \in \mathbb{R}$. Evaluating at i:

$$0 = g(i) = q(i)(i^2 + 1) + ai + b = ai + b,$$

since $i^2 + 1 = 0$. As a, b are real, $ai + b = 0$ forces $a = b = 0$, so $r(x) = 0$ and $g(x) = q(x)(x^2 + 1) \in \langle x^2 + 1\rangle$.
Hence $\ker\phi = \langle x^2 + 1\rangle$. By *The First Isomorphism Theorem for Rings*, since $\phi : \mathbb{R}[x] \to \mathbb{C}$ is a surjective ring homomorphism,

$$\mathbb{R}[x]/\langle x^2 + 1\rangle \cong \mathbb{C}.$$

1.2.4 Prime Ideals and Maximal Ideals

Definition 1.32.

1. A *prime ideal* A of of a commutative ring R is a *proper* ideal of R such that $a, b \in R$ and $ab \in R$ imply $a \in R$ or $b \in R$.
2. A *maximal ideal* A of a commutative ring R is a *proper* ideal of R such that, whenever B is an ideal of R and $A \subseteq B \subseteq R$, then $B = A$ or $B = R$

Theorem 1.33. Let R be a commutative ring and A be an ideal of R. Then R/A is an *integral domain* if and only if A is a *prime*.

Before proving this theorem, we firstly need to recall some definitions:

Definition 1.34.

1. A *zero-divisor* is a nonzero element a of a commutative ring R such that there is a nonzero element $b \in R$ with $ab = 0$.
2. An *integral domain* is a commutative ring and no zero-divisors.

Example 1.35. The ring of Gaussian integers $\mathbb{Z}[i] = \{a + bi | a, b \in \mathbb{Z}\}$ is an integral domain.

Proof. ($\Longrightarrow$) Suppose that R/A is an integral domain and $ab \in A$. Then $(a + A)(b + A) = ab + A = A$, the zero element of the ring R/A.

So, either $a + A = A$ or $b + A = A \Rightarrow$ either $a \in A$ or $b \in A$.

Hence, A is prime.

($\Longleftarrow$) Suppose A is a prime ideal, we need to prove R/A is a commutative ring and has no zero-divisors.

Firstly, R/A is a ring, since A is an ideal of R, by Theorem 1.25.

- *Commutative:* For $a, b \in R$ and $a + A, b + A \in R/A$,

  $(a + A)(b + A) = ab + A = ba + A = (b + A)(a + A)$ followed from the commutativity of R.

- *No zero-divisors:* Assume $(a + A)(b + A) = 0 + A = A$, then $ab + A = A \Rightarrow ab \in A$.

  Thus, $a \in A$ or $b \in A$ as A is prime, so one of $a + A$ or $b + A$ is the zero coset in R/A.

  Hence R/A has no zero-divisors.

Therefore, R/A is an integral domain. □

Theorem 1.36. Let R be a commutative ring and A be an ideal of R. Then R/A is an *field* if and only if A is a *maximal*.

Before proving this theorem, we may recall a definition of *field* and prove 2 claims:

Definition 1.37.

1. A field is a commutative ring in which every nonzero element is a unit.
2. A finite integral domain is a field.

Claim 3. *If A is an ideal of a ring R and 1 belongs to A, prove that $A = R$.*

Proof.

- $A \subseteq R$ is trivial
- Suppose $r \in R$, then $r \cdot 1 \in A$ since $1 \in A$ (left absorption).

  So $r \in A$, and then $R \subseteq A$

Thus, $A = R$ □

Claim 4. *Let R be a commutative ring and A be a proper ideal of R. Then for $b \in R$ but $b \notin A$, $B = \{br + a : r \in R, a \in A\}$ is an ideal of R that properly contains A*

Proof. First,we need to prove B is an ideal of R:

- *Non-empty Subset*:

  Since $0 \in A, 0 \in R$, $0 = b \cdot 0 + 0 \in B$, so B is not empty.

  As $br + a \in R$ as $b, r, a \in R$, $B \subseteq R$.

- *Closed under Subtraction*:

  Suppose $x, y \in B$,and $x = br_1 + a_1, y = br_2 + a_2$, where $r_1, r_2 \in R, a_1, a_2 \in A$.

  Then $x - y = (br_1 + a_1) - (br_2 + a_2) = b(r_1 - r_2) + (a_1 - a_2)$.

  Since R is a ring and A is a subgroup of R under addition, then $r_1 - r_2 \in R$ and $a_1 - a_2 \in A$.

  Thus, $x - y \in B$.

- *Closed under Multiplication by arbitrary $r \in R$*:

  Suppose $x = br + a \in B$ with $r \in R, a \in A$ and $\tilde{r} \in R$.

  Then $\tilde{r}x = x\tilde{r} = (br + a)\tilde{r} = br\tilde{r} + a\tilde{r}$.

  Since $r\tilde{r} \in R$ (Closure) and $a\tilde{r} \in A$ (A is an ideal), $\tilde{r}x, x\tilde{r} \in B$.

Now we know that B is an ideal of R, we need to prove it *properly* contains A:

- For $a \in A$, let $r = 0 \in R$, then $a = b \cdot 0 + a \in B \Rightarrow A \subseteq B$.
- Since $b = b \cdot 1 + 0$ and $1 \in R, 0 \in A$, then $b \in B$ and $b \notin A \Rightarrow A \neq B$

Therefore, $A \subset B$. □

Proof. ($\Longrightarrow$) Suppose R/A is a field and B is an ideal of R that *properly* contains A.

Firstly, $A \subset B \subseteq R \Rightarrow A \subset R$, so A is a proper ideal of R.

Next, as $A \neq B$, there exists $b \in B$ but $b \notin A$.

Then $b + A$ is a nonzero element of R/A, and therefore by definition of field, there exists $c + A \in R/A$ such that

$$(b + A)(c + A) = 1 + A,$$

where $1 + A$ is the multiplicative identity of R/A.

Since $b \in B$,$c \in R$ and B is an ideal, we have $bc \in B$.

Since $bc + A = 1 + A$, we have $1 - bc \in A \subset B$.

Since $bc \in B$ and $1 - bc \in B$, we have $1 = (1 - bc) + bc \in B$ by closure of B under addition.

By Claim 3 proved above, $B = R$, so A is maximal.

($\Longleftarrow$) Suppose A is maximal and $b + A$ is non-zero element of R/A with $b \in R$.

Then $b \notin A$. Otherwise, if $b \in A$, $b + A = 0 + A$ is the zero element in R/A.

Since every nonzero element of R/A is represented by some $b \notin A$, it is sufficient to show that every $b + A$ with $b \notin A$ has a multiplicative inverse.

Let $B = \{br + a : r \in R, a \in A\}$, by Claim 4, B is an ideal of R and $A \subset B$.

Since A is maximal, then $B = R$ by Definition 1.32.

Thus, $1 \in B \Rightarrow 1 = bc + a'$ for some $c \in R, a' \in A$

$\Rightarrow 1 + A = bc + a' + A = bc + A = (b + A)(c + A)$ by Claim 2.

Hence, $c + A \in R/A$ is the multiplicative inverse of $b + A$.

Therefore, by Definition 1.37, R/A is a field. □

Example 1.38.

1. For $n > 1, n \in \mathbb{Z}$, the ideal $n\mathbb{Z}$ is a prime ideal $\iff$ n is prime. ($\{0\}$ is also a

prime ideal)

2. For $\mathbb{Z}_n$, $\langle d \rangle$ is a maximal ideal of $\mathbb{Z}_n \iff d$ is a prime divisor of n.

3. The ideal $\langle x^2+1 \rangle$ is not prime in $\mathbb{Z}_2[x]$, which is the ring of all polynomials with coefficients in $\mathbb{Z}_2 = \{0, 1\}$.

   Since $x^2+1 = x^2+2x+1 = (x+1)^2$ in $\mathbb{Z}_2[x]$, we have $(x+1)(x+1) \in \langle x^2+1 \rangle$, but $x+1 \notin \langle x^2+1 \rangle$ (as $\langle x^2+1 \rangle$ contains only 0 and polynomials of degree ≥ 2). This violates the defining property of a prime ideal, so $\langle x^2+1 \rangle$ is not prime.

Example 1.39. Continue with Example 1.30, the ideal $\langle x^2+1 \rangle$ is maximal in $\mathbb{R}[x]$.

Proof. Let A be an ideal of $\mathbb{R}[x]$ that properly contains $\langle x^2+1 \rangle$ (i.e. $\langle x^2+1 \rangle \subset A \subseteq \mathbb{R}[x]$).
Then we can prove $\langle x^2+1 \rangle$ is a maximal ideal by proving $A = \mathbb{R}[x]$, by Definition 1.32.
Firstly, since $\langle x^2+1 \rangle \subset A$, there exists $f(x) \in A$, but $f(x) \notin \langle x^2+1 \rangle$.
Then by division of polynomials, $f(x) = q(x)(x^2+1) + r(x)$, where $\deg(r(x)) \leq 1$ and $r(x) \neq 0$.
Let $r(x) = ax + b$, with $a, b \in \mathbb{R}$, a and b are not both zero, then:

$$ax + b = r(x) = f(x) - q(x)(x^2+1) \in A,$$

since $f(x) \in A, q(x)(x^2+1) \in \langle x^2+1 \rangle \subset A$.
Thus, since $ax-b \in \mathbb{R}[x], ax+b \in A$ and A is an ideal, $a^2x^2-b^2 = (ax-b)(ax+b) \in A$.
Since $a^2(x^2+1) \in \langle x^2+1 \rangle \subset A$, $a^2+b^2 = a^2(x^2+1) - (a^2x^2-b^2) \in A$.
Let $c := a^2+b^2 \in \mathbb{R}$; since a, b are not both zero, $c \neq 0$. So the non-zero constant polynomial $h(x) = c \in A \subseteq \mathbb{R}[x] \Rightarrow \exists h^{-1}(x) = \frac{1}{c} \in \mathbb{R}[x] \Rightarrow 1 = \frac{1}{c} \cdot c \in A$.
Hence, by Claim 3 above, $A = \mathbb{R}[x]$. □

Moreover, by Theorem 1.36, $\mathbb{R}[x]/\langle x^2+1 \rangle$ is a field.

Example 1.40. The ideal $\langle x \rangle$ of $\mathbb{Z}[x]$ is a prime ideal, but not a maximal ideal.

Proof. First, we can consider elements in $\langle x \rangle = \{f(x) \cdot x \mid f(x) \in \mathbb{Z}[x]\}$:
Suppose $g(x) \in \langle x \rangle$, then $g(x) = f(x) \cdot x$ for some $f(x) \in \mathbb{Z}[x]$.
Thus, $g(0) = f(0) \cdot 0 = 0 \Rightarrow g(x) \in A$ if we define $A := \{g(x) \in \mathbb{Z}[x] \mid g(0) = 0\}$.
Hence, $\langle x \rangle \subseteq A$. Conversely, if we let $h(x) = a_0 + a_1x + a_2x^2 + \cdots \in A$:

$$h(0) = a_0 = 0 \Rightarrow h(x) = a_1x + a_2x^2 + \cdots = x(a_1 + a_2x + \dots) \Rightarrow h(x) \in \langle x \rangle$$

Therefore, $A \subseteq \langle x \rangle \Rightarrow \langle x \rangle = A = \{g(x) \in \mathbb{Z}[x] \mid g(0) = 0\}$.

Next, we need to prove $\langle x \rangle$ is a prime ideal:
Let $g(x), h(x) \in \mathbb{Z}[x]$ and $g(x)h(x) \in \langle x \rangle$, then $g(0)h(0) = 0$.

Since $\mathbb{Z}$ is an integral domain, this forces $g(0) = 0$ or $h(0) = 0$, so $g(x) \in \langle x \rangle$ or $h(x) \in \langle x \rangle$.
Hence $\langle x \rangle$ is a prime ideal.

Finally, recall (4) in Example 1.23: $\langle x, 2 \rangle = \{xf(x) + 2g(x) \mid f(x), g(x) \in \mathbb{Z}[x]\}$ is an ideal of $\mathbb{Z}[x]$.
We first check $\langle x, 2 \rangle$ is proper: every element $xf(x) + 2g(x) \in \langle x, 2 \rangle$ has constant term $2g(0)$, which is even, so $1 \notin \langle x, 2 \rangle$, giving $\langle x, 2 \rangle \neq \mathbb{Z}[x]$.
Since $x \in \langle x, 2 \rangle$, we have $\langle x \rangle \subseteq \langle x, 2 \rangle$.
Moreover, 2 has constant term $2 \neq 0$, so by the statement $\langle x \rangle = A$ above, $2 \notin \langle x \rangle$, giving $\langle x \rangle \subset \langle x, 2 \rangle \subset \mathbb{Z}[x]$.
Hence $\langle x \rangle$ is not a maximal ideal. □

1.3 Extra

1.3.1 Convolution Polynomial Rings

Example 1.41. Since we know that $\mathbb{Z}[x]$ is a commutative ring and $\langle x^N - 1 \rangle = \{g(x)(x^N - 1) \mid g(x) \in \mathbb{Z}[x]\}$ is a principal ideal generated by $x^N - 1$, then

$$\mathbb{Z}[x]/\langle x^N - 1 \rangle = \{f(x) + \langle x^N - 1 \rangle \mid f(x) \in \mathbb{Z}[x]\}$$

is a factor ring.
Let's first consider "0" in this ring: since $x^N - 1 + \langle x^N - 1 \rangle = 0 + \langle x^N - 1 \rangle$, we may treat x^N and 1 as representing the same element of the quotient, i.e. $x^N \equiv 1 \pmod{\langle x^N - 1 \rangle}$.
More generally, for $n \geq 0$, write $n = qN + r$ with $q, r \in \mathbb{Z}$, $0 \leq r \leq N - 1$. Then in $\mathbb{Z}[x]$:

$$x^n = x^{qN+r} = (x^N)^q x^r,$$

and since $x^N - 1 \in \langle x^N - 1 \rangle$, so is $(x^N)^q x^r - x^r = ((x^N)^q - 1)x^r$ (as $(x^N)^q - 1$ is divisible by $x^N - 1$, using the factorisation $y^q - 1 = (y - 1)(y^{q-1} + \cdots + 1)$ with $y = x^N$). Hence:

$$x^n + \langle x^N - 1 \rangle = x^r + \langle x^N - 1 \rangle, \qquad \text{i.e.} \qquad x^n \equiv x^r \pmod{\langle x^N - 1 \rangle}.$$

Existence of a low-degree representative.
Suppose $f(x) + \langle x^N - 1 \rangle \in \mathbb{Z}[x]/\langle x^N - 1 \rangle$, where $f(x) = \sum_{n=0}^{d} a_n x^n$ for some $d \geq 0$ and $a_n \in \mathbb{Z}$ (all but finitely many a_n are 0, since $f(x) \in \mathbb{Z}[x]$ has finite degree).
If $d < N$, there is nothing to do: $f(x)$ itself already has degree at most $N - 1$.
If $d \geq N$, group the terms of $f(x)$ by residue mod N:

$$f(x) = \sum_{r=0}^{N-1} \Big(\sum_{k \geq 0} a_{kN+r}\Big) x^{kN+r},$$

where each inner sum $\sum_{k\geq 0} a_{kN+r}$ is a *finite* sum, since $a_n = 0$ for $n > d$.
Since $x^{kN+r} \equiv x^r \pmod{\langle x^N - 1\rangle}$ for every $k \geq 0$, and coset addition/multiplication is computed termwise, we get:

$$f(x) + \langle x^N - 1\rangle = \sum_{r=0}^{N-1} (\sum_{k\geq 0} a_{kN+r}) x^r + \langle x^N - 1\rangle.$$

Writing $b_r := \sum_{k\geq 0} a_{kN+r} \in \mathbb{Z}$ for $r = 0, 1, \ldots, N-1$, this shows:

$$f(x) + \langle x^N - 1\rangle = (b_0 + b_1 x + \cdots + b_{N-1} x^{N-1}) + \langle x^N - 1\rangle,$$

so every coset has a representative of degree at most $N - 1$.

Uniqueness of the low-degree representative.
Suppose $g_1(x), g_2(x) \in \mathbb{Z}[x]$ both have degree at most $N - 1$ and represent the same coset, i.e.

$$g_1(x) + \langle x^N - 1\rangle = g_2(x) + \langle x^N - 1\rangle \quad \Longrightarrow \quad g_1(x) - g_2(x) \in \langle x^N - 1\rangle.$$

Then $g_1(x) - g_2(x) = h(x)(x^N - 1)$ for some $h(x) \in \mathbb{Z}[x]$.
Suppose $h(x) \neq 0$. Since $\mathbb{Z}[x]$ is an integral domain (as $\mathbb{Z}$ is), degrees add under multiplication:

$$\deg(h(x)(x^N - 1)) = \deg h(x) + \deg(x^N - 1) = \deg h(x) + N \geq N.$$

But $\deg(g_1 - g_2) \leq N - 1 < N$ (or $g_1 - g_2 = 0$), which is a contradiction unless $h(x) = 0$.
Hence $h(x) = 0$, so $g_1(x) = g_2(x)$: the representative of degree at most $N - 1$ is unique.

Conclusion. Combining existence and uniqueness, every element of $\mathbb{Z}[x]/\langle x^N - 1\rangle$ can be written *uniquely* as

$$b_0 + b_1 x + \cdots + b_{N-1} x^{N-1} + \langle x^N - 1\rangle, \qquad b_i \in \mathbb{Z},$$

so

$$\mathbb{Z}[x]/\langle x^N - 1\rangle = \left\{ b_0 + b_1 x + \cdots + b_{N-1} x^{N-1} + \langle x^N - 1\rangle \;\middle|\; b_0, \ldots, b_{N-1} \in \mathbb{Z} \right\}.$$

2 Lattices

2.1 Motivation

Before introducing lattices, we can consider an interesting problem: We are given that $y \in \mathbb{R}$ is the root of a cubic polynomial with *small integer* coefficients $ax^3 + bx^2 + cx + d$, where $a, b, c, d \in \mathbb{Z}$, and y is approximated to be $\hat{y} \approx 2.25992$. How can we find a closed form for the polynomial $ax^3 + bx^2 + cx + d$? (i.e. the approximated integer values of a, b, c, d)

To solve this problem, we may need $|\epsilon|$ to have a small value in the matrix-vector equation below:

$$\begin{pmatrix} a & b & c & d \end{pmatrix} \begin{pmatrix} \hat{y}^3 \\ \hat{y}^2 \\ \hat{y} \\ 1 \end{pmatrix} =: \epsilon$$

This is because for the real root y, we have $ay^3 + by^2 + cy + d = 0$, so if $\hat{y}$ is the approximated value for y, then the value of $a\hat{y}^3 + b\hat{y}^2 + c\hat{y} + d$ will be close to 0.

By modifying the above equation, we have:

$$\begin{pmatrix} a & b & c & d \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 & \hat{y}^3 \\ 0 & 1 & 0 & 0 & \hat{y}^2 \\ 0 & 0 & 1 & 0 & \hat{y} \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} a & b & c & d & \epsilon \end{pmatrix} \tag{1}$$

Now we assume that we have a machine that works like below:

Input: A matrix M with rows $r_1, r_2, \ldots, r_m$

Output: The "shortest" element r in the $\mathbb{Z}$-span of $r_1, r_2, \ldots, r_m$.

The $\mathbb{Z}$-span is a definition associated with *lattices*, introduced later. It is the set of row vectors of the form $a_1 r_1 + \cdots + a_m r_m$, where $a_i \in \mathbb{Z}$, $\forall i = 1, 2, \ldots, m$. The "shortest" element r means that r has the smallest *Euclidean norm* in this set.

We can use this machine in Equation 1 above: Let $M = \begin{pmatrix} 1 & 0 & 0 & 0 & \hat{y}^3 \\ 0 & 1 & 0 & 0 & \hat{y}^2 \\ 0 & 0 & 1 & 0 & \hat{y} \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}$, then we know that the vector on the RHS is exactly a linear combination of the rows of M, with coefficients $a, b, c, d \in \mathbb{Z}$. Thus $\begin{pmatrix} a & b & c & d & \epsilon \end{pmatrix}$ lies in the $\mathbb{Z}$-span of the rows of M.

By inputting M into the machine, we may get the "shortest" version of $\begin{pmatrix} a & b & c & d & \epsilon \end{pmatrix}$, but this doesn't necessarily lead to small $|\epsilon|$. For example, if we take $a, b, c = 0, d = 1$, then $\begin{pmatrix} a & b & c & d & \epsilon \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 \end{pmatrix}$. The norm of this vector is $\sqrt{2}$, which is indeed small, but $|\epsilon| = 1$ is too large. So we need to modify this approach.

If we consider a new matrix:

$$M' = \begin{pmatrix} 1 & 0 & 0 & 0 & 10000\hat{y}^3 \\ 0 & 1 & 0 & 0 & 10000\hat{y}^2 \\ 0 & 0 & 1 & 0 & 10000\hat{y} \\ 0 & 0 & 0 & 1 & 10000 \end{pmatrix}$$

Now the RHS of the equation becomes $\begin{pmatrix} a & b & c & d & 10000\epsilon \end{pmatrix}$, and inputting M' into the machine gives the result $\begin{pmatrix} 1 & -3 & 3 & -3 & 0.099995 \end{pmatrix}$.

In this case, $|\epsilon| = 0.99995 \times 10^{-5}$ is indeed very small, so we may check if the values of a, b, c, d work in the polynomial:

$$x^3 - 3x^2 + 3x - 3 = (x-1)^3 - 2 = 0 \Rightarrow y = \sqrt[3]{2} + 1 \approx 2.25992 = \hat{y}$$

Hence, $x^3 - 3x^2 + 3x - 3$ is the polynomial we want to find!

Then why doesn't M work, but M' does? This is because we put a multiplier of 10000 in the last column of M', which makes the output row vector have 10000ϵ instead of ϵ. When the machine calculates the norm of this row vector, $|\epsilon|$ will have more significance compared to the small a, b, c, d. Therefore, the problem we met for M is less likely to happen, and the output of the machine is more likely to be the vector we wanted, meaning ϵ is small in absolute value.

2.2 Basics of Lattices

2.2.1 Definition of Lattices

Definition 2.1. A *lattice* is the integer span of a set of linearly independent vectors $v_1, \ldots, v_m \in \mathbb{R}^{1\times n}$:

$$L = \mathbb{Z}_{v_1} + \mathbb{Z}_{v_2} + \cdots + \mathbb{Z}_{v_m}.$$

- The set $\{v_1, \ldots, v_m\}$ is called a *basis* for L.
- The matrix $B \in \mathbb{R}^{m\times n}$ with rows given by $v_1, \ldots, v_m$ is called a *basis matrix* for L.
- m is called the *rank* of L.
- If every element of L has integer entries, then L is called an *integral or (integer) lattice.*

Example 2.2. If $v_1 = (1,0), v_2 = (0,1)$, then $L = \mathbb{Z}(1,0) + \mathbb{Z}(0,1)$ consists of all integer points in the plane (i.e. $\mathbb{Z}^2$).
This lattice may have other bases, for example $\{(3,2),(1,1)\}$. We can show that $L' := \mathbb{Z}(3,2) + \mathbb{Z}(1,1) = L$:

$$(1,0) = (3,2) - 2 \times (1,1), \quad (0,1) = -(3,2) + 3 \times (1,1).$$

Thus, $(1,0)$ and $(0,1)$ lie in the $\mathbb{Z}$-span of $\{(3,2),(1,1)\}$, so $L \subseteq L'$. Similarly, $L' \subseteq L$. Hence, $L' = L$. (i.e. $\mathbb{Z}(3,2) + \mathbb{Z}(1,1) = \mathbb{Z}(1,0) + \mathbb{Z}(0,1) = \mathbb{Z}^2$)
We can see that in Figure 1 above, the lattices of these 2 bases is the same set of points in $\mathbb{R}^2$, which is the set of all integer points.

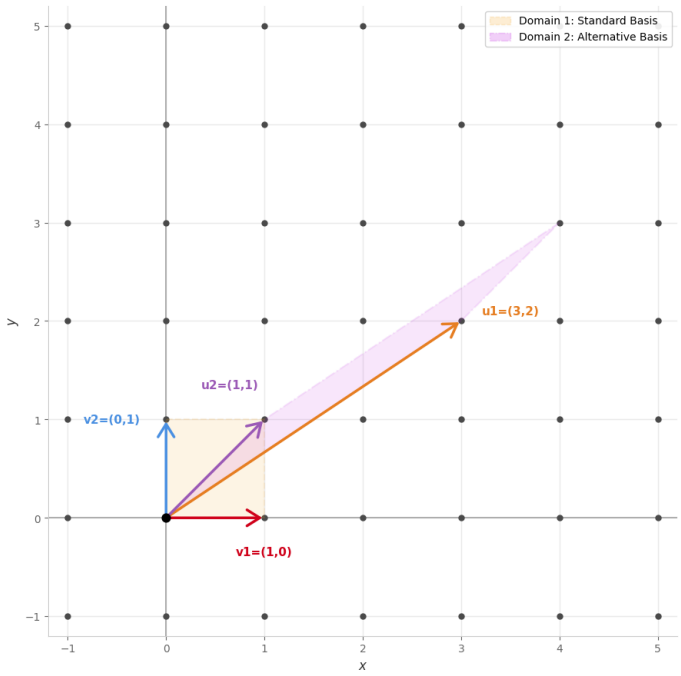


Figure 1: Equivalent Lattice: $\mathbb{Z}^2 = \mathbb{Z}(1,0) + \mathbb{Z}(0,1) = \mathbb{Z}(3,2) + \mathbb{Z}(1,1)$

Example 2.3. Another example of lattices not equal to $\mathbb{Z}^2$: $L = \mathbb{Z}(2,0) + \mathbb{Z}(1,\sqrt{3})$ is called the *Hexagonal Lattice*.In Figure 2 below, we can see that this lattice is not *integral* as $(1,\sqrt{3}) \in L$, but it is not an integer vector.

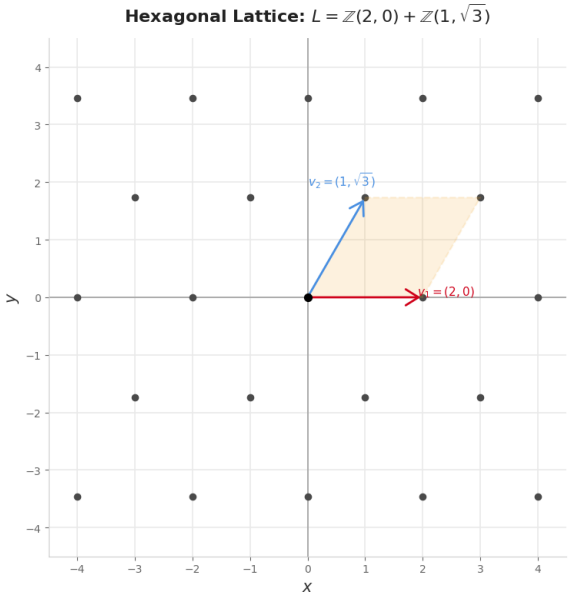


Figure 2: Hexagonal Lattice: $L = \mathbb{Z}(2,0) + \mathbb{Z}(1,\sqrt{3})$

2.2.2 Propositions of Lattices

Proposition 2.4. Suppose B is a basis matrix for a lattice L defined as above. Then $v \in L$ if and only if there exists $x \in \mathbb{Z}^{1\times m}$ such that $v = xB$.

Proof. ($\Longrightarrow$) Assume $v \in L$, then there exists $k_1, \dots k_m \in \mathbb{Z}$ such that $v = k_1v_1 + \cdots + k_mv_m$ by Definition. Hence,

$$v = \begin{pmatrix} k_1 & k_2 & \dots & k_m \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{pmatrix} = xB,$$

where $x := \begin{pmatrix} k_1 & k_2 & \dots & k_m \end{pmatrix} \in \mathbb{Z}^{1\times m}$.

($\Longleftarrow$) Assume $x = \begin{pmatrix} k_1 & k_2 & \dots & k_m \end{pmatrix} \in \mathbb{Z}^{1\times m}$, with $k_1, \dots k_m \in \mathbb{Z}$.

Let $v = xB \in \mathbb{Z}^{1\times m}$. Hence,

$$v = \begin{pmatrix} k_1 & k_2 & \dots & k_m \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{pmatrix} = k_1v_1 + \cdots + k_mv_m \in L.$$

□

Proposition 2.5. Let B and B' be $m \times n$ basis matrices for lattices L and L' respectively. Then $L = L'$ if and if only there exists $U \in GL(m, \mathbb{Z})$ with $B' = UB$.

Proof. Suppose L has basis vector $v_1, \dots, v_m$, which are rows of B. Similarly, L' has basis vector $w_1, \dots, w_m$, which are rows of B'.

($\Longrightarrow$) Assume $L = L'$, then every basis vector w_i of L' is in L, so it can be written as $w_i = \sum_{j=1}^{m} k_{ij}v_j$ with $k_{ij} \in \mathbb{Z}$, for $i, j = 1, \dots, m$.

Then let U be a $m \times m$ matrix with integer entries $U_{ij} = k_{ij}$ and $B' = UB$.

Similarly, every basis vector v_i of L is in L', then $B = VB'$ for some $m \times m$ matrix V with integer entries.

Combining above two equations, we get:

$$B = VB' = VUB \Rightarrow (I_m - VU)B = 0$$

Since rows of B are linearly independent, B is full row-rank, so the only way to satisfy $(I_m - VU)B = 0$ is $I_m - VU = 0$.

Hence, $VU = I_m \Rightarrow \det(V)\det(U) = \det(I_m) = 1$.

Since all entries in U and V are integers, $\det(V)$ and $\det(U)$ both are integers, so $\det(U) = \det(V) = \pm 1$.

Therefore, $U \in GL(m, \mathbb{Z})$ and $B' = UB$.

($\Longleftarrow$) Assume there exists $U \in GL(m, \mathbb{Z})$ such that $B' = UB$.

Suppose $w \in L'$, then $w = yB'$ for some $y \in \mathbb{Z}^{1\times m}$, by Proposition 2.4.

Thus, $w = yB' = y(UB) = (yU)B$, where $yU \in \mathbb{Z}^{1\times m}$, so $w \in L$ by Proposition 2.4.

Hence, $L' \subseteq L$.

Conversely, we may get $L \subseteq L'$ by similar steps using U^{-1}. This works because U^{-1} is an invertible matrix with integer entries just as U, by formulae $U^{-1} = \frac{1}{\det(U)} adj(U)$ and $\det(U) = \pm 1$.

In conclusion, $L = L'$. □

Remark 2.6.

1. $GL(m, \mathbb{Z})$ is the set of $m \times m$ matrices with integer entries, which are invertible and have determinant equal to ± 1.

2. There is an alternative way to think of this proposition, using *elementary row operations*: each such operation can be represented by left-multiplication by an invertible matrix. If we make some *restrictions* to operations, each of these representing matrices may be in $GL(m, \mathbb{Z})$ and multiply together to get U. This alternative approach may be introduced later, in Example 2.23.

Definition 2.7. The *determinant* (or *covolume*) of a full rank lattice L is $\det(L) := |\det(B)|$, where B is any basis matrix of L.

Remark 2.8.

1. This is definition is *well-defined*, because for B, B' representing the same lattice L, $B' = UB$ with $\det(U) = \pm 1$ by Proposition 2.5, so $|\det(B')| = |\det(U)||\det(B)| = |\det(B)|$.

2. The *full-rank* lattice here means that $m = n$ as defined in Definition 2.1. It is necessary because $\det(B)$ exists for square matrices.

   It is called *rank deficient* if $m < n$, and we will introduce a more general definition of determinant of lattice later in Definition 2.15 to include this case.

Definition 2.9. Let $L \subseteq \mathbb{R}^{1\times n}$ with basis $\{v_1, \ldots, v_n\}$. The *fundamental domain* or *fundamental region* for L corresponding to this basis is the set

$$\mathcal{F}(v_1, \ldots, v_n) = \{\sum_{i=1}^{n} t_i v_i \mid 0 \le t_i < 1\}.$$

Example 2.10. Consider the lattice $L = \mathbb{Z}(2, 3) + \mathbb{Z}(-2, 2) = \mathbb{Z}(4, 1) + \mathbb{Z}(6, -1)$.
Since $\begin{pmatrix} 4 & 1 \\ 6 & -1 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ -2 & 2 \end{pmatrix}$ and $\det \begin{pmatrix} 1 & -1 \\ 1 & -2 \end{pmatrix} = -1$, so these 2 bases indeed represent the same lattice by Proposition 2.5.
We can see two *fundamental domains* with respect to these bases in Figure 3 below: One thing to notice is that we didn't include $t_i = 1$ in the definition, so the "far"

edges and the "furthest" point are not included in the fundamental domain.

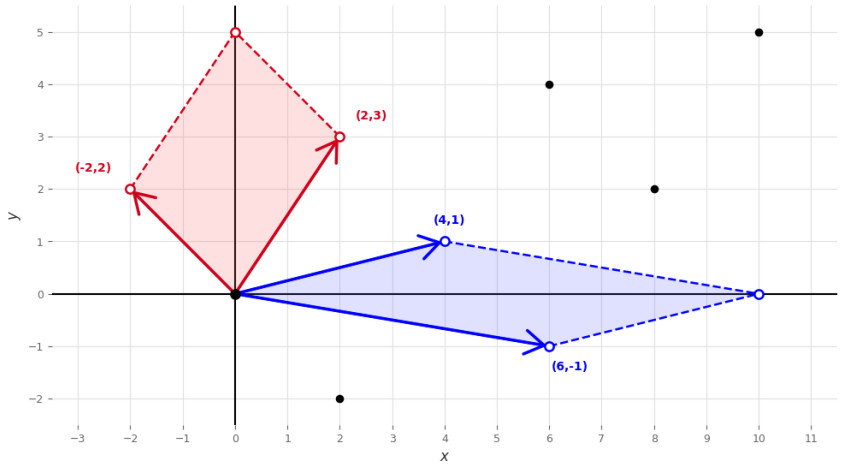


Figure 3: $L = \mathbb{Z}(2,3) + \mathbb{Z}(-2,2) = \mathbb{Z}(4,1) + \mathbb{Z}(6,-1)$

Theorem 2.11. Let $L \subseteq \mathbb{R}^n$ be a full rank lattice, and let $\mathcal{F}$ be a fundamental domain for L. Then every vector $w \in \mathbb{R}^n$ can be written uniquely in the form

$$w = f + v$$

where $f \in \mathcal{F}$ and $v \in L$.

Proof. First, we prove existence: let $v_1, \ldots, v_n$ be basis vectors of L.

These are linearly independent, and since there are n of them, they form a basis of $\mathbb{R}^n$.

For every $w \in \mathbb{R}^n$, there exist $\alpha_i \in \mathbb{R}$ such that $w = \sum_{i=1}^n \alpha_i v_i$.

Now define $a_i := \lfloor \alpha_i \rfloor$, the greatest integer smaller than or equal to α_i, and define $t_i := \alpha_i - \lfloor \alpha_i \rfloor$.

Thus $\alpha_i = t_i + a_i$, with $a_i \in \mathbb{Z}$ and $0 \leq t_i < 1$. Hence, we get:

$$w = \sum_{i=1}^{n} (t_i + a_i) v_i = \underbrace{\sum_{i=1}^{n} t_i v_i}_{\in \mathcal{F}} + \underbrace{\sum_{i=1}^{n} a_i v_i}_{\in L}$$

Let $f := \sum_{i=1}^n t_i v_i$ and $v := \sum_{i=1}^n a_i v_i$; then $w = f + v$.

Next, we prove uniqueness. Suppose $w = f + v = f' + v'$, with $f, f' \in \mathcal{F}$ and $v, v' \in L$.

Let $f = \sum_{i=1}^n t_i v_i$ and $f' = \sum_{i=1}^n t'_i v_i$, where $t_i, t'_i \in [0,1)$.

Then $v - v' = f' - f = \sum_{i=1}^n (t'_i - t_i) v_i \in L$, so $t'_i - t_i \in \mathbb{Z}$.

Since $t_i, t'_i \in [0,1)$, we have $t'_i - t_i \in (-1,1)$, so it must be 0, giving $t'_i = t_i$ for all i, and hence $f' = f$.

It follows that $v' = w - f' = w - f = v$. Therefore f and v are unique. □

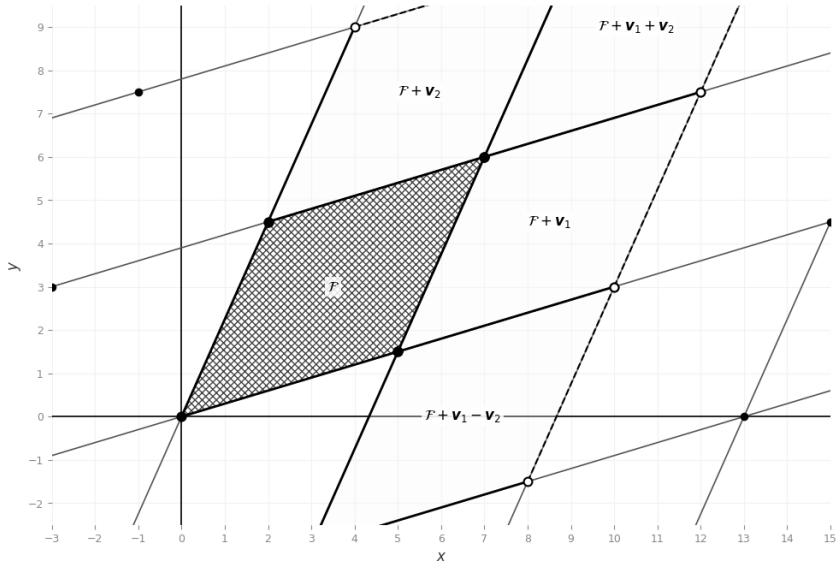


Figure 4: Union of translations of $\mathcal{F}$ by vectors in L exactly covers $\mathbb{R}^n$

Remark 2.12. This theorem shows that the union of translations of the fundamental domain $\mathcal{F}$ covers $\mathbb{R}^n$ without overlapping, i.e. it is a disjoint union:

$$\mathbb{R}^n = \bigcup_{v \in L} (\mathcal{F} + v)$$

Here $\mathcal{F} + v := \{f + v \mid f \in \mathcal{F}\}$, and disjointness across distinct $v \in L$ is exactly the uniqueness part of the theorem: if $w \in (\mathcal{F} + v) \cap (\mathcal{F} + v')$ for $v \neq v'$, then $w = f+v = f'+v'$ for some $f, f' \in \mathcal{F}$, contradicting uniqueness of the decomposition unless $v = v'$.
See Figure 4 above.

Proposition 2.13. Let $L \subseteq \mathbb{R}^n$ be a lattice with basis $\{v_1, \ldots, v_n\}$, and let

$$\mathcal{F} = \{t_1 v_1 + \cdots + t_n v_n : t_i \in [0, 1) \text{ for all } i\}$$

be the associated fundamental domain. Then

$$\det L = \mathrm{Vol}(\mathcal{F}).$$

Proof. By definition,

$$\mathrm{Vol}(\mathcal{F}) = \int_{\mathcal{F}} dx_1\, dx_2 \cdots dx_n.$$

Consider the change of variables from $x = (x_1, \ldots, x_n)$ to $t = (t_1, \ldots, t_n)$ given by

$$x = t_1 v_1 + \cdots + t_n v_n,$$

where $v_j = (v_j^1, v_j^2, \dots, v_j^n) \in \mathbb{R}^n$ for each j. Equivalently, each coordinate of x is

$$x_i = \sum_{j=1}^{n} t_j v_j^i, \qquad i = 1, \dots, n.$$

The Jacobian matrix of this transformation is

$$J = \left(\frac{\partial x_i}{\partial t_j}\right)_{i,j} = \begin{pmatrix} v_1^1 & \dots & v_n^1 \\ \vdots & \ddots & \vdots \\ v_1^n & \dots & v_n^n \end{pmatrix} = B^T,$$

where B is the $n \times n$ basis matrix of L whose rows are $v_1, \dots, v_n$.

Since $\det J = \det(B^T) = \det B$, the change-of-variables formula gives

$$dx_1\, dx_2 \cdots dx_n = |\det B|\, dt_1\, dt_2 \cdots dt_n.$$

Under this substitution, x ranges over $\mathcal{F}$ exactly when $t_j \in [0, 1)$ for each j, i.e. when t ranges over $[0, 1)^n$. Hence

$$\mathrm{Vol}(\mathcal{F}) = |\det B| \int_{[0,1)^n} dt_1\, dt_2 \cdots dt_n = |\det B| \cdot 1 = |\det B|.$$

Since $\det L = |\det B|$ for any basis matrix B of L, we conclude $\mathrm{Vol}(\mathcal{F}) = \det L$. □

This proposition lets us interpret $\det L$ geometrically:

- In $\mathbb{R}^2$, $\det L$ is the *area* of the fundamental domain, a parallelogram with sides v_1, v_2.
- In $\mathbb{R}^3$, $\det L$ is the *volume* of the fundamental domain, a parallelepiped with edges v_1, v_2, v_3.

Viewing $v_1, \dots, v_n$ as the edges of an n-dimensional parallelepiped, elementary geometric intuition suggests that, among all bases with the edge lengths $\|v_1\|, \dots, \|v_n\|$ fixed, the volume $\mathrm{Vol}(\mathcal{F})$ is largest when the v_i are pairwise orthogonal, in which case $\mathrm{Vol}(\mathcal{F}) = \prod_{i=1}^{n} \|v_i\|$. This motivates the following inequality below:

Proposition 2.14 (Hadamard's Inequality)**.** Let L be a lattice and let $v_1, \dots, v_n$ be any basis for L. Then

$$\det L \le \|v_1\|\|v_2\| \cdots \|v_n\|,$$

with equality if and only if $v_1, \dots, v_n$ are pairwise orthogonal.

Proof. Step 1: Find Pairwise Orthogonal Basis Vectors.

Apply the Gram–Schmidt Process (Without Normalizing) in *Algebra 3* to $v_1, \dots, v_n$, defining $v_1^*, \dots, v_n^*$ recursively by

$$v_1^* := v_1, \quad w_i := \sum_{j=1}^{i-1} \frac{\langle v_i, v_j^* \rangle}{\langle v_j^*, v_j^* \rangle} v_j^*, \quad v_i^* := v_i - w_i, \quad i = 2, \dots, n.$$

By construction, the vectors $v_1^*, \ldots, v_n^*$ are pairwise orthogonal, and for each i we may write:

$$v_i = v_i^* + w_i, \quad \text{where } w_i \in \operatorname{span}(v_1^*, \ldots, v_{i-1}^*) \text{ and } \langle v_i^*, w_i \rangle = 0. \tag{$*$}$$

Step 2: Prove $\det L = \prod_{i=1}^n \|v_i^*\|$.

From the Gram–Schmidt recursion,

$$v_1^* = v_1, \qquad v_i^* = v_i - \sum_{j=1}^{i-1} \frac{\langle v_i, v_j^* \rangle}{\langle v_j^*, v_j^* \rangle} v_j^* \quad (i \geq 2),$$

rearranging the second equation for v_i gives

$$v_i = v_i^* + \sum_{j=1}^{i-1} \underbrace{\frac{\langle v_i, v_j^* \rangle}{\langle v_j^*, v_j^* \rangle}}_{=: U_{ij}} v_j^*, \qquad i = 2, \ldots, n,$$

and for $i = 1$ we have $v_1 = v_1^*$.

In this case, let $U_{ii} = 1$ for all i and $U_{ij} = 0$ for $j > i$, every v_i is expressed as

$$v_i = \sum_{j=1}^{n} U_{ij}\, v_j^*, \qquad i = 1, \ldots, n,$$

where $U = (U_{ij})$ is clearly *lower triangular*.

Let B denote the $n \times n$ matrix with rows $v_1, \ldots, v_n$, and B^* the matrix with rows $v_1^*, \ldots, v_n^*$.

The relation $v_i = \sum_j U_{ij} v_j^*$ is exactly the matrix equation

$$B = U B^*.$$

Since U is lower triangular with 1's on the diagonal, $\det U = \prod_{i=1}^n U_{ii} = 1$.

Taking determinants of $B = UB^*$,

$$\det B = \det(U) \cdot \det(B^*) = \det(B^*),$$

and hence

$$\det L = |\det B| = |\det B^*|.$$

Now we need to compute $|\det B^*|$: Since $v_1^*, \ldots, v_n^*$ are pairwise orthogonal, the matrix $B^*(B^*)^T$ has (k, l) entry $\langle v_k^*, v_l^* \rangle$, which is 0 for $k \neq l$ and $\|v_k^*\|^2$ for $k = l$. Hence,

$$B^*(B^*)^T = \operatorname{diag}(\|v_1^*\|^2, \ldots, \|v_n^*\|^2).$$

Taking determinants of both sides,

$$\det(B^*) \cdot \det((B^*)^T) = \prod_{i=1}^{n} \|v_i^*\|^2.$$

Since $\det((B^*)^T) = \det(B^*)$, $(\det(B^*))^2 = \prod_{i=1}^n \|v_i^*\|^2 \Rightarrow |\det B^*| = \prod_{i=1}^n \|v_i^*\|$.

Therefore,

$$\det L = \prod_{i=1}^{n} \|v_i^*\|.$$

Step 3: Prove $\|v_i^*\| \leq \|v_i\|$ *for* $\forall\ i$.

By $(*)$, $v_i = v_i^* + w_i$ with $\langle v_i^*, w_i \rangle = 0$. By the Pythagorean theorem, we have:

$$\|v_i\|^2 = \|v_i^*\|^2 + \|w_i\|^2 \geq \|v_i^*\|^2.$$

Hence, $\|v_i^*\| \leq \|v_i\|$, with equality if and only if $\|w_i\| = 0$, i.e. $w_i = 0$.

In conclusion,

$$\det L = \prod_{i=1}^{n} \|v_i^*\| \leq \prod_{i=1}^{n} \|v_i\| = \|v_1\|\|v_2\| \cdots \|v_n\|.$$

Moreover, equality holds if and only if $w_i = 0$ for every $i = 2, \ldots, n$. (i.e. if and only if $v_1, \ldots, v_n$ are already pairwise orthogonal) □

The method for calculating $\det B^*$ is quite helpful, and we may use similar approach to define a more general definition for determinant of lattice:

In Definition 2.7, we require lattice L to be full rank ($m = n$), but there will be cases for *rank deficient* lattice:

Let $L \subseteq \mathbb{R}^n$, with basis matrix $B \in \mathbb{R}^{m \times n}$, basis vector $v_1, \ldots, v_m \in \mathbb{R}^{1 \times n}$ and rank of $L = m < n$. In this case, L lies inside an subspace V with dimension m, and $V = \text{span}(v_1, \ldots, v_m)$, which is *isomorphic* to $\mathbb{R}^m$. Moreover, $\det L$ should be the volume of a fundamental domain in V.

For example, $L \subseteq \mathbb{R}^3$ with $B = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, then $L \subseteq V = \{(x, y, 0) \mid x, y \in \mathbb{R}\} \cong \mathbb{R}^2$. Thus, $\det L$ is the *area* of fundamental domain based on $(1, 0, 0)$ and $(0, 1, 0)$ at a plane, instead of the *volume* in $\mathbb{R}^3$.

Generally, we have linearly independent vector $\{v_1, \ldots, v_m\}$, then we can extend it to be the basis for $\mathbb{R}^n$: $\{v_1, \ldots, v_m, v_{m+1}, \ldots, v_n\}$.

By *Gram-Schmidt Process* (With Normalizing), we can get a orthonormal basis $\{v_1^*, \ldots, v_m^*, v_{m+1}^*, \ldots, v_n^*\}$, associated with $\{v_1, \ldots, v_m, v_{m+1}, \ldots, v_n\}$.

By similar approach in the Step 2 Proof of Proposition 2.14, we may have a lower triangular matrix $U \in \mathbb{R}^{n \times n}$ such that $A = UA^*$, where A is consisted with rows $\{v_1, \ldots, v_n\}$ and A^* is consisted with rows $\{v_1^*, \ldots, v_n^*\}$.

We only need the left upper $m \times m$ part of U, defined it to be $R \in \mathbb{R}^{m \times m}$, so $U = \begin{pmatrix} R & 0_{m \times (n-m)} \\ * & * \end{pmatrix}$. Also, let $Q := A^* \in \mathbb{R}^{n \times n}$.

$$\text{Since } A = \begin{pmatrix} v_1 \\ \vdots \\ v_m \\ \vdots \\ v_n \end{pmatrix} = \begin{pmatrix} R & 0_{m \times (n-m)} \\ * & * \end{pmatrix} \begin{pmatrix} v_1^* \\ \vdots \\ v_m^* \\ \vdots \\ v_n^* \end{pmatrix} = Q \Rightarrow B = \begin{pmatrix} v_1 \\ \vdots \\ v_m \end{pmatrix} = R \begin{pmatrix} v_1^* \\ \vdots \\ v_m^* \end{pmatrix} \in \mathbb{R}^{m \times m},$$

we have

$$\begin{pmatrix} v_1 \\ \vdots \\ v_m \end{pmatrix} = (R \mid 0_{m\times(n-m)}) \begin{pmatrix} v_1^* \\ \vdots \\ v_m^* \\ \vdots \\ v_n^* \end{pmatrix}$$

Hence,

$$\underbrace{B}_{\in\mathbb{R}^{m\times n}} = \underbrace{(R \mid 0_{m\times(n-m)})}_{\in\mathbb{R}^{m\times n}} \underbrace{Q}_{\in\mathbb{R}^{n\times n}}.$$

The determinant of lattice L is defined to be $|\det R|$. However, it is complicated to compute it every time, we can continue to simplify this calculation:

Since the rows of Q are orthonormal basis, then Q is orthogonal and $QQ^T = I_n$.

Thus,

$$BB^T = (R \mid 0)QQ^T \begin{pmatrix} R^T \\ 0 \end{pmatrix} = (R \mid 0) \begin{pmatrix} R^T \\ 0 \end{pmatrix} = RR^T.$$

Hence,

$$\det(BB^T) = \det(RR^T) = \det(R)\det(R^T) = \det(R)^2.$$

Therefore,

$$\det L = |\det(R)| = \sqrt{\det(BB^T)}.$$

Definition 2.15. Let L be a lattice with basis matrix B. Then the *determinant* of L is

$$\det L = \sqrt{\det(BB^T)}.$$

This definition works well for both full-rank and rank-deficient lattices.

2.2.3 An Alternative Characterisation of Lattices

Definition 2.16. A subgroup L of $\mathbb{R}^n$ is said to be *discrete* if there exists $\epsilon > 0$ such that, for every $v \in L$, we have

$$\{u \in L \mid \|u - v\| < \epsilon\} = \{v\}$$

This is actually a definition of *uniformly discrete* subgroup, since ϵ can be chosen *uniformly* for all $v \in L$. For subgroups of $\mathbb{R}^n$, these two notations coincide.

Motivation: In this section, we want to show that lattices are the same as *discrete additive subgroups* of $\mathbb{R}^n$. First, we need to prove that the length of any non-zero vector in a lattice L is bounded below, and we will again use *Gram-Schmidt Orthogonalisation*.

Proposition 2.17. Let L be a lattice with $m \times n$ basis matrix B, and let $\hat{B}$ be the Gram-Schmidt orthogonalisation of B. Write v_i for the rows of B and $\hat{v}_i$ for the rows

of $\hat{B}$. Then the *shortest* non-zero vector in L has length *at least* $\min_{1\leq i\leq m}\|\hat{v}_i\|$.

Proof. Let y be an arbitrary *non-zero* element of L, then there exists non-zero $x=(x_1,\ldots,x_m)\in\mathbb{Z}^m$ such that $y=xB$, by Proposition 2.4.

Let j be the maximal value in $\{1,2,\ldots,m\}$ such that $x_j\neq 0$, so $xB=\sum_{i=1}^{j}x_iv_i$.

Then, we may consider

$$|\langle xB,\hat{v}_j\rangle|=|\langle\sum_{i=1}^{j}x_iv_i,\hat{v}_j\rangle|=|\sum_{i=1}^{j}x_i\langle v_i,\hat{v}_j\rangle|$$

Recall the Gram-Schmidt Orthogonalisation:

- $\hat{v}_j$ is orthogonal to $\text{span}(\hat{v}_1,\ldots,\hat{v}_{j-1})$, for all $j=2,\ldots,m$.
- $\text{span}\{v_1,\ldots,v_j\}=\text{span}\{\hat{v}_1,\ldots,\hat{v}_j\},\forall 1\leq j\leq m$
- $v_j=\hat{v}_j+\sum_{i=1}^{j-1}\frac{\langle v_j,\hat{v}_i\rangle}{\langle\hat{v}_i,\hat{v}_i\rangle}\hat{v}_i$,for $j=2,\ldots,m$, and $v_1=\hat{v}_1$.

Hence, $\langle v_i,\hat{v}_j\rangle=0$ for $i<j$, as $v_i\in\text{span}\{v_1,\ldots,v_{j-1}\}=\text{span}\{\hat{v}_1,\ldots,\hat{v}_{j-1}\}$ and $\hat{v}_j$ is orthogonal to $\text{span}(\hat{v}_1,\ldots,\hat{v}_{j-1})$.

Thus,

$$|\langle xB,\hat{v}_j\rangle|=|\sum_{i=1}^{j}x_i\langle v_i,\hat{v}_j\rangle|=|x_j\langle v_j,\hat{v}_j\rangle|=|x_j||\langle v_j,\hat{v}_j\rangle|.$$

Since $v_j=\hat{v}_j+\sum_{i=1}^{j-1}\frac{\langle v_j,\hat{v}_i\rangle}{\langle\hat{v}_i,\hat{v}_i\rangle}\hat{v}_i$, then $\langle v_j,\hat{v}_j\rangle=\langle\hat{v}_j,\hat{v}_j\rangle+\sum_{i=1}^{j-1}\frac{\langle v_j,\hat{v}_i\rangle}{\langle\hat{v}_i,\hat{v}_i\rangle}\langle\hat{v}_i,\hat{v}_j\rangle=\langle\hat{v}_j,\hat{v}_j\rangle$.

Hence,

$$|\langle xB,\hat{v}_j\rangle|=|x_j||\langle v_j,\hat{v}_j\rangle|=|x_j|\|\hat{v}_j\|^2.$$

By *Cauchy's Inequality*, we have $|\langle xB,\hat{v}_j\rangle|\leq\|xB\|\|\hat{v}_j\|$.

Therefore,

$$\|y\|=\|xB\|\geq\frac{|\langle xB,\hat{v}_j\rangle|}{\|\hat{v}_j\|}=|x_j|\|\hat{v}_j\|\geq\|\hat{v}_j\|\geq\min_{1\leq i\leq m}\|\hat{v}_i\|$$

□

Theorem 2.18. Let L be a subgroup of $\mathbb{R}^n$. Then L is a lattice if and only if it is a *discrete subgroup* of $\mathbb{R}^n$.

Proof. ($\Longrightarrow$) Assume L is a lattice, then choose $\epsilon<\min_{1\leq i\leq m}\|\hat{v}_i\|$, as in the notation of Proposition 2.17.

Then for all $v\in L$, if $u\neq v\in L$, then $u-v\neq 0$ and $u-v\in L$, so $\|u-v\|\geq\min_{1\leq i\leq m}\|\hat{v}_i\|>\epsilon$.

Thus, $\{\|u-v\|<\epsilon\}$ and $u\in L$ force $u=v$. Therefore, $\{u\in L\mid\|u-v\|<\epsilon\}=\{v\}$, so L is a discrete subgroup of $\mathbb{R}^n$.

($\Longleftarrow$) *Step 1: Define a New Lattice L'.*

Assume L is a discrete subgroup of $\mathbb{R}^n$.

Let V be the $\mathbb{R}$-subspace of $\mathbb{R}^n$ spanned by L, with dimension m. ($m\leq n$)

Choose a basis $u_1, \ldots, u_m$ of V consisting of vectors in L, and form the lattice:

$$L' = \mathbb{Z}_{u_1} + \cdots + \mathbb{Z}_{u_m}.$$

Since L is a subgroup, then $x \in L' \Rightarrow x := \sum_{i=1}^{m} k_i u_i$ for some $k_i \in \mathbb{Z} \Rightarrow x \in L$.
Hence, $L' \subseteq L$.

Step 2: Prove index $[L : L']$ is finite.

Here, we let $[L : L']$ denotes the number of distinct cosets of L' in L.

To prove this, we need to consider the number of representatives for distinct cosets, and also the size of factor group $L/L' = \{v + L' \mid x \in L'\}$. This factor group exists because addition is always commutative in $\mathbb{R}^n$.

Let $\{v_i\}_{i \in I}$ be a set of representative for cosets of L' in L, where $|I| = [L : L']$.

Let $\mathcal{F}$ be the fundamental domain of L' in V, by Theorem 2.11, we have:

$$v_i = x_i + v_i' \text{ for some } x_i \in \mathcal{F}, v_i' \in L'.$$

Then $x_i = v_i - v_i' \in L \cap \mathcal{F}$. Since L is discrete and $\mathcal{F}$ is bounded, $L \cap \mathcal{F}$ is finite, so the number of $\{x_i\}$ is finite.

If $x_i = x_j$, then $v_i - v_i' = v_j - v_j' \Rightarrow v_i - v_j = v_i' - v_j' \in L'$.

Then $v_i + L' = v_j + L' \in L/L' \Rightarrow i = j$, as the v_i are defined to be distinct coset representatives.

Therefore, the index $[L : L'] = |I| \leq |L \cap \mathcal{F}|$ is finite.

Step 3: Prove $L \subseteq \frac{1}{q}L$.

Let $[L : L'] =: q \in \mathbb{N}$ and for any $x \in L$, $x + L' \in L/L'$, which is a factor group has q elements.

Since $\langle x + L' \rangle$ is a subgroup of L/L' and $|L/L'| = q$, order of $x + L'$ divides q by Lagrange's Theorem.

Then, $q(x + L')$ is equal to "zero" in L/L' under addition, so $q(x + L') = qx + L' = L'$.

Hence, $qx \in L' \Rightarrow qL \subseteq L'$.

For all $x \in L$, $qx = a_1 u_1 + \cdots + a_m u_m \in L'$, for some $a_i \in \mathbb{Z}$.

$\Rightarrow x = a_1 \frac{u_1}{q} + \cdots + a_m \frac{u_m}{q} \in \mathbb{Z}_{\frac{u_1}{q}} + \cdots + \mathbb{Z}_{\frac{u_m}{q}} =: \frac{1}{q}L'$, which is also a lattice in V.

Thus, $L' \subseteq L \subseteq \frac{1}{q}L'$.

Step 4: Prove L is a lattice.

By Step 3, $L' \subseteq L \subseteq \frac{1}{q}L'$, and both L' and $\frac{1}{q}L'$ are free abelian groups of rank m (with bases $u_1, \ldots, u_m$ and $\frac{u_1}{q}, \ldots, \frac{u_m}{q}$ respectively).

Sandwiching L between two rank-m free abelian groups forces L itself to be free abelian of rank m, by the structure theorem for subgroups of finitely generated free abelian groups (we will not go into the details of this theory here; see Wikipedia for background).

Hence $L = \mathbb{Z}w_1 + \cdots + \mathbb{Z}w_m$ for some $w_1, \ldots, w_m \in L$, and since these span V with $m = \dim V$, they form a basis of V.

Therefore L is a lattice. □

2.2.4 A Digression on Linear Independence

Our original Definition 2.1 requires L to be generated by linearly independent vectors, which is clearly necessary. Otherwise, $\mathbb{Z} + \mathbb{Z}\sqrt{2} \subseteq \mathbb{R}$ would be a lattice, but this group is not discrete, since it is possible to find points arbitrarily close to 0. Therefore, it doesn't satisfy Theorem 2.18 above.

However, if we restrict attention to integer lattices, then it is possible to remove the linear independence hypothesis: *the $\mathbb{Z}$-span of a set of integer vectors is a lattice.*

Proposition 2.19. Any subgroup of $\mathbb{Z}^n$ is free, of rank at most n.

We will not go through the proof here (it proceeds by induction on n, using a short exact sequence argument); if you're interested, have a try yourself, or see e.g. here for background.

In the language of lattices, we have proved:

Proposition 2.20. The $\mathbb{Z}$-span of a set of vectors in $\mathbb{Z}^n$ (or, more generally, in $\mathbb{Q}^n$) is a lattice.

Remark 2.21.

- The set of vectors used to generate L does not need to be finite:

  Any (even infinite) collection of vectors in $\mathbb{Z}^n$ generates a subgroup of $\mathbb{Z}^n$, so the proposition still applies and this subgroup is automatically a lattice.

- The result also extends to vectors with rational entries:

  If $v_1, \ldots, v_k \in \mathbb{Q}^n$, pick a common denominator q so that $qv_1, \ldots, qv_k \in \mathbb{Z}^n$.

  Then $L' = \mathbb{Z}(qv_1) + \cdots + \mathbb{Z}(qv_k)$ is a lattice in $\mathbb{Z}^n$ by the Proposition above.

  Since $L = \frac{1}{q}L'$, scaling a basis of L' by $\frac{1}{q}$ gives a basis of L. So L is a lattice too.

While this result is theoretically useful, it is not constructive, and so one has to be careful. For example, it is not necessarily the case that a spanning set can be reduced to obtain a basis.

In practice, a lattice is often given as $L = \mathbb{Z}v_1 + \cdots + \mathbb{Z}v_m$ with $m > n$ vectors in $\mathbb{Z}^n$. The result tells us L is a lattice, but the v_i themselves are too many to be a basis. Finding an actual basis is a separate, harder problem:

Example 2.22. Consider the lattice L spanned by the vectors $v_1 = (2, 0), v_2 = (0, 3), v_3 = (1, 1)$, so $m = 3 > 2 = n$.
Now, L sits inside $\mathbb{Z}^2$ and contains the vectors

$$2v_1 + v_2 - 3v_3 = (1, 0), \qquad v_1 + v_2 - 2v_3 = (0, 1).$$

Hence L contains both standard basis vectors of $\mathbb{Z}^2$, so $L = \mathbb{Z}^2$.

However, no two-element subset of $\{v_1, v_2, v_3\}$ has $\mathbb{Z}$-span equal to $\mathbb{Z}^2$.

2.2.5 Finding a Basis from a Generating Set

Motivation: Unlike vector spaces, a generating set for a lattice cannot always be reduced to a basis simply by removing redundant vectors. As seen in Example 2.22, a lattice may be generated by a dependent set of vectors such that no proper subset generates the same lattice. Consequently, finding a basis for a lattice requires a different approach.

Suppose a lattice L is given as the $\mathbb{Z}$-span of vectors

$$v_1, \ldots, v_m \in \mathbb{R}^n.$$

If these vectors are arranged as the rows of a matrix A, then

$$L = \operatorname{span}_{\mathbb{Z}}\{v_1, \ldots, v_m\}.$$

There are efficient algorithms for extracting a basis from such a generating set.

One important method is the *Hermite Normal Form* (HNF). Roughly speaking, HNF plays a role analogous to row echelon form in linear algebra, but with an important *restriction*: we are only allowed to perform *integral row operations*.

These are precisely the operations corresponding to left multiplication by a matrix in $\mathbf{GL}(m, \mathbb{Z})$. The allowed operations are:

- **swapping two rows;**
- **multiplying a row by -1;**
- **adding an integer multiple of one row to another.**

Each of these operations preserves the lattice spanned by the rows, since every new row remains an integer linear combination of the original rows and vice versa. By repeatedly applying such operations, one can transform the matrix into a simpler form whose nonzero rows constitute a basis for the lattice.

We will not give a formal definition of Hermite Normal Form here, but instead illustrate the process through an example.

Example 2.23. Let L be the lattice spanned by the rows of the following matrix:

$$\begin{pmatrix} 2 & 4 & -3 & -5 \\ 1 & -1 & -3 & 3 \\ -2 & 5 & -5 & -4 \\ 0 & -3 & 1 & 4 \end{pmatrix}.$$

We want to find a basis for L. To do this, we iteratively try to simplify the rows of this matrix by performing elementary row operations, where each operation corresponds to left multiplication by an element of $\mathrm{GL}(4, \mathbb{Z})$.

For example,

$$\begin{pmatrix} 2 & 4 & -3 & -5 \\ 1 & -1 & -3 & 3 \\ -2 & 5 & -5 & -4 \\ 0 & -3 & 1 & 4 \end{pmatrix} \xrightarrow{r_3' = r_3 + r_1} \begin{pmatrix} 2 & 4 & -3 & -5 \\ 1 & -1 & -3 & 3 \\ 0 & 9 & -8 & -9 \\ 0 & -3 & 1 & 4 \end{pmatrix},$$

$$\begin{pmatrix} 2 & 4 & -3 & -5 \\ 1 & -1 & -3 & 3 \\ 0 & 9 & -8 & -9 \\ 0 & -3 & 1 & 4 \end{pmatrix} \xrightarrow{\substack{r_1' = r_1 - 2r_2 \\ r_3' = r_3 + 3r_4}} \begin{pmatrix} 0 & 6 & 3 & -11 \\ 1 & -1 & -3 & 3 \\ 0 & 0 & -5 & 3 \\ 0 & -3 & 1 & 4 \end{pmatrix},$$

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & -1 & -3 & 3 \\ 0 & 0 & -5 & 3 \\ 0 & -3 & 1 & 4 \end{pmatrix} \xrightarrow{\text{permute}} \begin{pmatrix} 1 & -1 & -3 & 3 \\ 0 & -3 & 1 & 4 \\ 0 & 0 & -5 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Thus, L has rank 3, and a basis for L is given by the non-zero rows of the final matrix:

$$\{(1, -1, -3, 3),\ (0, -3, 1, 4),\ (0, 0, -5, 3)\}.$$

Why do these "restrictions" work?

When simplifying a generating matrix for a lattice, every row operation must preserve the lattice generated by the rows.

If a matrix A is transformed into another matrix B, then there must exist an integer matrix $U \in \mathrm{GL}(m, \mathbb{Z})$ such that $B = UA$. Since U is invertible over $\mathbb{Z}$, we can also recover A from B by $A = U^{-1}B$, where U^{-1} also has integer entries. Thus, A and B generate exactly the same lattice.

A standard result from linear algebra states that an integer matrix U has an integer inverse (equivalently, $U \in \mathrm{GL}(m, \mathbb{Z})$) if and only if $\det(U) = \pm 1$. Hence, we only allow elementary row operations whose corresponding matrices satisfy this condition.

Let U_1, U_2, U_3 denote the elementary matrices corresponding to the three allowed row operations:

- Swapping two rows. Then $\det(U_1) = -1$.
- Multiplying a row by -1. Then $\det(U_2) = -1$.
- Adding an integer multiple of one row to another. Then $\det(U_3) = 1$, regardless of the multiplier.

Therefore, each of these elementary matrices belongs to $\mathrm{GL}(m, \mathbb{Z})$, so every operation is reversible using another integer row operation and hence preserves the lattice.

On the other hand, suppose we were allowed to multiply a row by an arbitrary integer $k \in \mathbb{Z}$, the corresponding elementary matrix would satisfy $\det(U) = k$, so $\det(U^{-1}) = \frac{1}{k}$.

If $|k| > 1$, this is not an integer, so U^{-1} cannot have integer entries and $U \notin \mathrm{GL}(m, \mathbb{Z})$: this operation cannot be reversed using integer row operations, and in general changes the lattice generated by the rows.

This is exactly why $k = -1$ is the only non-trivial scalar allowed.

Hence, the restriction to the three elementary row operations above is exactly what guarantees that the lattice remains unchanged throughout the reduction process.

2.3 Operations on Lattices

2.3.1 Sums & Intersections

Definition 2.24. Let L and L' be lattices. Then their *sum* is

$$L + L' := \{v + w \mid v \in L, w \in L'\}$$

and their *intersection* is

$$L \cap L' := \{v \mid v \in L \text{ and } w \in L'\}.$$

Remark 2.25.

1. $L + L'$ may not be lattice. For example, $\mathbb{Z}, \mathbb{Z}\sqrt{2} \subseteq R$ are both lattices, but $\mathbb{Z} + \mathbb{Z}\sqrt{2} = \mathbb{R}$ is not a lattice.
2. If $L, L' \subseteq \mathbb{Q}^n$, then $L + L'$ is again a lattice, since vectors in this subgroup of $\mathbb{R}^n$ are discrete.

Example 2.26. Let $L, L' \subseteq \mathbb{R}^4$ have basis matrices

$$B = \begin{pmatrix} 2 & 4 & -2 & 2 \\ 0 & 4 & 4 & 5 \\ 5 & 1 & 4 & 2 \end{pmatrix}, \qquad B' = \begin{pmatrix} 1 & 1 & -2 & -4 \\ 0 & 4 & 2 & 4 \\ 1 & 4 & 5 & -2 \end{pmatrix}.$$

To find a basis for the sum $L + L'$, we first stack the two basis matrices together to obtain a generating matrix:

$$A = \begin{pmatrix} 2 & 4 & -2 & 2 \\ 0 & 4 & 4 & 5 \\ 5 & 1 & 4 & 2 \\ 1 & 1 & -2 & -4 \\ 0 & 4 & 2 & 4 \\ 1 & 4 & 5 & -2 \end{pmatrix}.$$

Since $L + L'$ is a subgroup of $\mathbb{Z}^4$, its rank is at most 4, so the six rows of A cannot all belong to a basis. We therefore apply similar row operations, as in Example 2.23: By several steps of calculations (Exercise!), we will find a basis matrix for the lattice

$L + L'$ is

$$\begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 3 \end{pmatrix}.$$

2.3.2 The Dual of a Lattice

Definition 2.27. Let $L \subseteq \mathbb{R}^n$ be a lattice. The *dual* of L is

$$L^* := \{x \in \operatorname{span}_{\mathbb{R}} L \mid \langle x, v\rangle \in \mathbb{Z} \text{ for all } v \in L\}$$

where $\langle -, -\rangle$ is the usual inner-product on $\mathbb{R}^n$ and $\operatorname{span}_{\mathbb{R}}(L)$ is the smallest real vector space containing L.
Equivalently,

$$\operatorname{span}_{\mathbb{R}}(L) = \left\{ \sum_{i=1}^{n} a_i v_i : a_i \in \mathbb{R},\ v_i \in \mathcal{B} \right\},$$

where $\mathcal{B} = \{v_1, \ldots, v_n\}$ is any basis of L.

Remark 2.28. If L is an integer lattice, then we always have $L \subseteq L^*$. Since $v, w \in L$ with all integer elements, then $\langle v, w\rangle \in \mathbb{Z}$.

It seems that this definition is different from the *dual* of vector spaces that we learnt in *Algebra 3*.

Let us consider a new definition:

$$\operatorname{Hom}_{\mathbb{Z}}(L, \mathbb{Z}) = \{f : L \to \mathbb{Z} \mid f \text{ is } \mathbb{Z}\text{-linear}\},$$

where "$\mathbb{Z}$-linear" means that $f(av + bw) = af(v) + bf(w), \forall a, b \in \mathbb{Z}, v, w \in L$.

This is the same as

$$\{f \in (\operatorname{span}_{\mathbb{R}} L)^* \mid f(v) \in \mathbb{Z} \text{ for all } v \in L\} =: L^\dagger,$$

where $(\operatorname{span}_{\mathbb{R}} L)^*$ is the vector space dual of $\operatorname{span}_{\mathbb{R}} L$.

We claim that $L^* \cong L^\dagger$ as abelian groups. To show this, we may first consider the map of vector spaces

$$\theta : \operatorname{span}_{\mathbb{R}} L \to (\operatorname{span}_{\mathbb{R}} L)^*, \qquad \theta(x) = \langle x, -\rangle.$$

This map is an isomorphism because

- Injective: $\theta(x) = \theta(y) \Rightarrow \langle x, v\rangle = \langle y, v\rangle$, for all vector $v \Rightarrow \langle x - y, v\rangle = 0$
  $\Rightarrow \|x - y\|^2 = 0$ if $v = x - y \Rightarrow x - y = 0 \Rightarrow x = y$.
- Surjective: Let $V = \operatorname{span}_{\mathbb{R}} L$. Since V is a finite-dimensional vector space, we know that $\dim(V) = \dim(V^*)$. By the Rank-Nullity Theorem, any injective linear map

between vector spaces of the same finite dimension is automatically surjective.

Then we claim θ restricts to a bijection $L^* \to L^\dagger$:

- Injective: Clearly true because it is the restriction of θ, which is injective.
- Surjective: Let $f \in L^\dagger$, then $f \in (\operatorname{span}_{\mathbb{R}} L)^*$ and $f(v) \in \mathbb{Z}, \forall v \in L$.

  Since θ is an isomorphism of vector spaces, $\exists x \in \operatorname{span}_{\mathbb{R}} L$ such tha $\theta(x) = f$.

  Hence, $\langle x, v\rangle = \theta(x)(v) = f(v) \in \mathbb{Z}$, for all $v \in L \Rightarrow x \in L^*$.

Therefore, $L^* \cong L^\dagger$.

Example 2.29. Let L have basis matrix

$$B = \begin{pmatrix} 2 & -1 & 5 \\ 2 & 2 & 2 \end{pmatrix}.$$

We check directly that $x_1 = \begin{pmatrix} 1/6 & 1/3 & 0 \end{pmatrix}$ lies in L^*:
Since any element in L is a linear combination of integer coefficient, then it is only essential to prove $\langle x_1, (2,-1,5)\rangle, \langle x_1, (2,2,2)\rangle \in \mathbb{Z}$ for $x_1 \in L^*$.

$$\langle x_1, (2,-1,5)\rangle = \frac{2}{6} - \frac{1}{3} + 0 = \frac{1}{3} - \frac{1}{3} = 0 \in \mathbb{Z}, \langle x_1, (2,2,2)\rangle = \frac{2}{6} + \frac{2}{3} + 0 = \frac{1}{3} + \frac{2}{3} = 1 \in \mathbb{Z}.$$

Similarly for $x_2 = \begin{pmatrix} 0 & 1/6 & -1/6 \end{pmatrix}$:

$$\langle x_2, (2,-1,5)\rangle = 0 - \frac{1}{6} - \frac{5}{6} = -1 \in \mathbb{Z}, \langle x_2, (2,2,2)\rangle = 0 + \frac{2}{6} - \frac{2}{6} = 0 \in \mathbb{Z}.$$

So both $x_1, x_2 \in L^*$. Then we may guess what is the basis vector of L^*? Is it $\{x_1, x_2\}$ in this Example? We will explain this later in Example 2.35.

In *Algebra 3*, duality of vector spaces is *inclusion-reversing*, this is the same for lattices:

Proposition 2.30. If L and M are lattices of the same rank with $L \subseteq M$, then $L^* \supseteq M^*$.

Proof. Let $x \in M^*$, so $x \in \operatorname{span}_{\mathbb{R}} M$ and $\langle x, v\rangle \in \mathbb{Z}$ for all $v \in M$.

Since $L \subseteq M$ and L, M have the same rank, $\operatorname{span}_{\mathbb{R}} L = \operatorname{span}_{\mathbb{R}} M$ (as a subspace of $\operatorname{span}_{\mathbb{R}} M$ having the same dimension must be all of $\operatorname{span}_{\mathbb{R}} M$), so $x \in \operatorname{span}_{\mathbb{R}} L$.

Since $L \subseteq M$, we also have $\langle x, v\rangle \in \mathbb{Z}$ for every $v \in L$ (as $L \subseteq M$).

Hence $x \in L^*$, so $M^* \subseteq L^*$. □

Remark 2.31. The rank condition is essential, otherwise the case of below example may happen.

Example 2.32. Let $L, M \subseteq \mathbb{R}^3$ have respective basis matrices

$$B_L = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad B_M = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

We will show that L and M are self-dual, and deduce that $L^* \not\supseteq M^*$, despite $L \subseteq M$:
The lattice L has basis $(1,0,0),\ (0,1,0)$, so

$$L = \{(a,b,0) : a,b \in \mathbb{Z}\} = \mathbb{Z}^2,$$

and

$$\operatorname{span}_{\mathbb{R}}(L) = \{(x,y,0) : x,y \in \mathbb{R}\} = \mathbb{R}^2.$$

Let $(x,y,0) \in \operatorname{span}_{\mathbb{R}}(L)$. Then $(x,y,0) \in L^*$ if and only if

$$\langle (x,y,0),(a,b,0)\rangle = ax + by \in \mathbb{Z}, \forall a,b \in \mathbb{Z}.$$

Taking $(a,b) = (1,0)$ gives $x \in \mathbb{Z}$, and taking $(a,b) = (0,1)$ gives $y \in \mathbb{Z}$. Hence,

$$L^* = \{(x,y,0) : x,y \in \mathbb{Z}\} = L = \mathbb{Z}^2.$$

Similarly,$M = \mathbb{Z}^3$, and $\operatorname{span}_{\mathbb{R}}(M) = \mathbb{R}^3$.
Let $(x,y,z) \in \mathbb{R}^3$. Then $(x,y,z) \in M^*$ if and only if

$$\langle (x,y,z),(a,b,c)\rangle = ax + by + cz \in \mathbb{Z}$$

for every $a,b,c \in \mathbb{Z}$.
Taking $(a,b,c) = (1,0,0)$, $(0,1,0)$ and $(0,0,1)$ gives $x,y,z \in \mathbb{Z}$.Hence,

$$M^* = \mathbb{Z}^3 = M.$$

Therefore,

$$L = \mathbb{Z}^2 \subsetneq \mathbb{Z}^3 = M.$$

However,

$$L^* = \mathbb{Z}^2 \not\supseteq \mathbb{Z}^3 = M^*.$$

In conclusion, although $L \subseteq M$, the inclusion is not reversed after taking duals.

Then, what if L and M has *different* rank?

Proposition 2.33. If $L \subseteq M$ are lattices (of possibly different rank), then

$$L^* \supseteq M^* \cap \operatorname{span}_{\mathbb{R}} L.$$

Proof. Let $x \in M^* \cap \operatorname{span}_{\mathbb{R}} L$. Then $x \in \operatorname{span}_{\mathbb{R}} L$, which is the first condition needed for $x \in L^*$.

It remains to check $\langle x, v\rangle \in \mathbb{Z}$ for all $v \in L$:

Since $x \in M^*$, we know $\langle x, w\rangle \in \mathbb{Z}$ for *every* $w \in M$. Since $L \subseteq M$, every $v \in L$ is in particular an element $w \in M$, so $\langle x, v\rangle \in \mathbb{Z}$ for all $v \in L$. Hence $x \in L^*$. □

Next, we may be interested in how to determine the elements in the dual of a lattice:

Theorem 2.34. Let L be a lattice with (rank × ambient dimension) basis matrix B. Then L^* is a lattice, and a basis matrix for L^* is given by

$$(BB^T)^{-1}B.$$

In particular, L and L^* have the same rank.

Proof. Let L has rank m inside $\mathbb{R}^n$, so B is $m \times n$ of full row rank m and BB^T is an $m \times m$ symmetric matrix which is invertible. We want to show that

$$L^* = \text{rowspan}_{\mathbb{Z}}((BB^T)^{-1}B).$$

($\supseteq$) Suppose $v = y(BB^T)^{-1}B$ for some $y \in \mathbb{Z}^m$, then $y(BB^T) \in \mathbb{R}^m$, so v is a real linear combination of the rows of B and $v \in \text{span}_{\mathbb{R}} L$.

If $w \in L$, then $w = xB$ for some $x \in \mathbb{Z}^m$. Hence,

$$\langle v, w\rangle = vw^T = y(BB^T)^{-1}B(xB)^T = y(BB^T)^{-1}BB^Tx^T = y(BB^T)^{-1}(BB^T)x^T = yx^T = \langle y, x\rangle \in \mathbb{Z},$$

since $x, y \in \mathbb{Z}^m$.

So $v \in L^*$, and then $L^* \supseteq \text{rowspan}_{\mathbb{Z}}((BB^T)^{-1}B)$.

($\subseteq$) Let $v \in L^*$. Since $v \in \text{span}_{\mathbb{R}} L = \text{span}_{\mathbb{R}}\{\text{rows of } B\}$, we may write $v = xB$ for some $x \in \mathbb{R}^n$.

For each basis vector w of L (a row of B), we may have $\langle v, w\rangle \in \mathbb{Z}$ by definition of L^*. Combining these over all m rows gives $vB^T \in \mathbb{Z}^m$. Then

$$v = xB = x(BB^T)(BB^T)^{-1}B = (xB)B^T \cdot (BB^T)^{-1}B = (vB^T) \cdot (BB^T)^{-1}B.$$

Since $vB^T \in \mathbb{Z}^m$, v is an element of $\text{rowspan}_{\mathbb{Z}}((BB^T)^{-1}B)$, so $L^* \subseteq \text{rowspan}_{\mathbb{Z}}((BB^T)^{-1}B)$.

Therefore, $L^* = \text{rowspan}_{\mathbb{Z}}((BB^T)^{-1}B)$ is proved.

We know from above that the m rows of $(BB^T)^{-1}B$ are obtained from the linearly independent rows of B by left-multiplying by the invertible matrix $(BB^T)^{-1}$.

Since $(BB^T)^{-1}$ is invertible, left multiplication by $(BB^T)^{-1}$ preserves linear independence, so the rows of $(BB^T)^{-1}B$ are *linearly independent*.

Therefore, $L^* = \text{rowspan}_{\mathbb{Z}}((BB^T)^{-1}B)$ is indeed a lattice of rank m, i.e. L^* is a lattice of the same rank as L. □

Example 2.35. Continuing the previous example with

$$B = \begin{pmatrix} 2 & -1 & 5 \\ 2 & 2 & 2 \end{pmatrix},$$

we compute L^* from the formula, checking it against the vectors found above. First,

$$BB^T = \begin{pmatrix} 2^2+(-1)^2+5^2 & 2\cdot 2+(-1)\cdot 2+5\cdot 2 \\ 2\cdot 2+(-1)\cdot 2+5\cdot 2 & 2^2+2^2+2^2 \end{pmatrix} = \begin{pmatrix} 30 & 12 \\ 12 & 12 \end{pmatrix}.$$

Then $\det(BB^T) = 30\cdot 12 - 12\cdot 12 = 360 - 144 = 216$, and

$$(BB^T)^{-1} = \frac{1}{216}\begin{pmatrix} 12 & -12 \\ -12 & 30 \end{pmatrix} = \begin{pmatrix} 1/18 & -1/18 \\ -1/18 & 5/36 \end{pmatrix}.$$

So

$$(BB^T)^{-1}B = \begin{pmatrix} 1/18 & -1/18 \\ -1/18 & 5/36 \end{pmatrix}\begin{pmatrix} 2 & -1 & 5 \\ 2 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 0 & -\frac{1}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{1}{3} & 0 \end{pmatrix}.$$

Therefore, we can find a basis for L^*, which is

$$\beta_1 := \{\begin{pmatrix} 0 & -\frac{1}{6} & \frac{1}{6} \end{pmatrix}, \begin{pmatrix} \frac{1}{6} & \frac{1}{3} & 0 \end{pmatrix}\}.$$

Then, we can compare the lattice of basis $\beta_2 := \{\begin{pmatrix} 1/6 & 1/3 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1/6 & -1/6 \end{pmatrix}\}$ found earlier with L^* and see whether they are the same.

- If β_1 and β_2 are both bases of L^*, then by the methods we use in Example 2.23, two matrices formed by these 2 bases will differ by left-multiplication by a matrix in $GL(2,\mathbb{Z})$.
- Comparing the two bases directly: the second vector of β_1 equals the first vector of β_2, and the first vector of β_1 equals -1 times the second vector of β_2. So $\beta_1 = M\beta_2$ with

  $$M = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad \det M = 1,$$

  which is integral with determinant ± 1, hence $M \in GL(2,\mathbb{Z})$. So β_1 and β_2 are both bases of L^*.
- Therefore, L^* is exactly the lattice formed by β_2, not merely a lattice containing it.

Proposition 2.36. $(L^*)^* = L$.

Proof. Let B be a basis matrix for L, so by Theorem 2.34, $B^* := (BB^T)^{-1}B$ is a basis matrix for L^*.

Applying this theorem again to the lattice L^* with basis matrix B^*, a basis matrix for

$(L^*)^*$ is

$$(B^*(B^*)^T)^{-1}B^*.$$

We first need to compute $B^*(B^*)^T$:

$$B^*(B^*)^T = (BB^T)^{-1}B \cdot \left[(BB^T)^{-1}B\right]^T = (BB^T)^{-1}BB^T((BB^T)^{-1})^T$$

Since $((BB^T)^{-1})^T = ((BB^T)^T)^{-1}$ and $(BB^T)^{-1}$ is symmetric:

$$B^*(B^*)^T = (BB^T)^{-1}(BB^T)(BB^T)^{-1} = (BB^T)^{-1}.$$

Hence $(B^*(B^*)^T)^{-1} = BB^T$, so a basis matrix for $(L^*)^*$ is

$$(BB^T) \cdot B^* = (BB^T)(BB^T)^{-1}B = B.$$

So $(L^*)^*$ has the same basis matrix B as L, and hence $(L^*)^* = L$. □

2.3.3 Duality of Sums & Intersections

Theorem 2.37. Let L and M be *full-rank* lattices. Then $(L \cap M)^* = L^* + M^*$ and $(L + M)^* = L^* \cap M^*$.

Proof. **(1)** For the first equation, we have:

- $L \cap M \subseteq L \Rightarrow (L \cap M)^* \supseteq L^*$ by Proposition 2.30.
- $L \cap M \subseteq M \Rightarrow (L \cap M)^* \supseteq M^*$ by Proposition 2.30.

Thus, $(L \cap M)^* \supseteq L^* + M^*$.
Conversely,

- $L^* \subseteq L^* + M^* \Rightarrow L \supseteq (L^* + M^*)^*$.
- $M^* \subseteq L^* + M^* \Rightarrow M \supseteq (L^* + M^*)^*$.

Hence, $L \cap M \supseteq (L^* + M^*)^* \Rightarrow (L \cap M)^* \subseteq L^* + M^*$
Therefore, $(L \cap M)^* = L^* + M^*$, the first equation is proved.
(2) For the second equation, let L be L^* and M be M^* in the first equation:

$$(L^* \cap M^*)^* = (L^*)^* + (M^*)^* = L + M.$$

Taking the duals of both sides gives:

$$L^* \cap M^* = (L + M)^*.$$

□

Remark 2.38. The condition of rank is again essential. For the general case, we have:

If L and M are lattices (no essentially full-rank), then

$$(L \cap M)^* = (L^* + M^*) \cap \operatorname{span}_{\mathbb{R}} L \cap M.$$

In particular, for full-rank lattices L and M, we may have the result $L \cap M = (L^* + M^*)^*$, which helps us to calculate the intersection of lattices through calculating duality:

Example 2.39. Let L and M have basis matrices

$$B_L = \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix}, \qquad B_M = \begin{pmatrix} 1 & 4 \\ 5 & -1 \end{pmatrix}.$$

Step 1: Compute B_L^* and B_M^*.
$B_L B_L^T = \begin{pmatrix} 1+1 & 2-1 \\ 2-1 & 4+1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 5 \end{pmatrix}$, with determinant 9.
Hence, $(B_L B_L^T)^{-1} = \frac{1}{9} \begin{pmatrix} 5 & -1 \\ -1 & 2 \end{pmatrix}$. Then,

$$B_L^* = (B_L B_L^T)^{-1} B_L = \frac{1}{9} \begin{pmatrix} 5 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix} = \frac{1}{3} \begin{pmatrix} 1 & -2 \\ 1 & 1 \end{pmatrix}.$$

Similarly for B_M^*, we will get

$$B_M^* = (B_M B_M^T)^{-1} B_M = \frac{1}{21} \begin{pmatrix} 1 & 5 \\ 4 & -1 \end{pmatrix}.$$

Step 2: Compute the basis vector B^* of $L^* + M^*$. We need to apply the restricted row operations to this matrix:

$$\frac{1}{21} \begin{pmatrix} 7 & -14 \\ 7 & 7 \\ 1 & 5 \\ 4 & -1 \end{pmatrix}.$$

After several steps of calculation, we will get the result of

$$B^* = \frac{1}{21} \begin{pmatrix} 1 & 5 \\ 0 & 7 \\ 0 & 0 \\ 0 & 0 \end{pmatrix},$$

Hence, $L^* + M^*$ has basis matrix

$$\frac{1}{21} \begin{pmatrix} 1 & 5 \\ 0 & 7 \end{pmatrix}.$$

Step 3: Compute $(L^* + M^*)^* = L \cap M$.

Since $B^* = \frac{1}{21}\begin{pmatrix} 1 & 5 \\ 0 & 7 \end{pmatrix}$, then

$$B^*(B^*)^T = \frac{1}{441}\begin{pmatrix} 1+25 & 0+35 \\ 0+35 & 0+49 \end{pmatrix} = \frac{1}{441}\begin{pmatrix} 26 & 35 \\ 35 & 49 \end{pmatrix},$$

with $\det(\begin{smallmatrix} 26 & 35 \\ 35 & 49 \end{smallmatrix}) = 49$, so

$$(B^*(B^*)^T)^{-1} = 441 \cdot \frac{1}{49}\begin{pmatrix} 49 & -35 \\ -35 & 26 \end{pmatrix} = \begin{pmatrix} 441 & -315 \\ -315 & 234 \end{pmatrix}.$$

Then

$$(B^*(B^*)^T)^{-1}B^* = \begin{pmatrix} 441 & -315 \\ -315 & 234 \end{pmatrix} \cdot \frac{1}{21}\begin{pmatrix} 1 & 5 \\ 0 & 7 \end{pmatrix} = \frac{1}{21}\begin{pmatrix} 441 & 0 \\ -315 & 63 \end{pmatrix} = \begin{pmatrix} 21 & 0 \\ -15 & 3 \end{pmatrix}.$$

So $L \cap M$ has basis matrix $\begin{pmatrix} 21 & 0 \\ -15 & 3 \end{pmatrix}$.

Step 4: Reduce to a nicer basis.

As with any lattice, we may replace this basis with any other related by a matrix in $GL(2, \mathbb{Z})$. Let $s_1 = (21, 0)$, $s_2 = (-15, 3)$, and set

$$t_1 := s_1 + s_2 = (6, 3), \qquad t_2 := -2s_1 - 3s_2 = (-42 + 45,\ 0 - 9) = (3, -9).$$

The transformation matrix $\begin{pmatrix} 1 & 1 \\ -2 & -3 \end{pmatrix}$ has determinant $1 \cdot (-3) - 1 \cdot (-2) = -1$.

Hence,

$$L \cap M = \mathbb{Z}(6, 3) + \mathbb{Z}(3, -9),$$

i.e. another basis matrix for $L \cap M$, with smaller entries, is

$$\begin{pmatrix} 6 & 3 \\ 3 & -9 \end{pmatrix}.$$

3 Latticed-Based Cryptography

3.1 Short Vectors in Lattices

3.1.1 Fundamental Lattice Problems

- **The Shortest Vector Problem (SVP):** Find a shortest non-zero vector in a lattice L, i.e. find a non-zero vector $v \in L$ that minimizes the Euclidean norm $\|v\|$.
- **The Closest Vector Problem (CVP):** Given a vector $w \in \mathbb{R}^m$ that is not in L, find a vector $v \in L$ that is closest to w, i.e. find a vector $v \in L$ that minimizes the Euclidean norm $\|w - v\|$.

- **Shortest Basis Problem (SBP):** Find a basis $v_1, \ldots, v_n$ for a lattice that is shortest in some sense. For example, we might require that

$$\max_{1 \leq i \leq n} \|v_i\| \quad \text{and} \quad \sum_{i=1}^{n} \|v_i\|^2$$

be minimized. There are thus many different versions of **SBP**, depending on *how to measure the "size" of the basis.*

- **Approximate Shortest Vector Problem (apprSVP):** Let $\psi(n)$ be a function of n. In a lattice L of dimension n, find a non-zero vector v that is no more than $\psi(n)$ times longer than a shortest non-zero vector v_{shortest} in L. Mathematically speaking, we want to find a non-zero vector $v \in L$ such that

$$\|v\| \leq \psi(n)\|v_{\text{shortest}}\|.$$

- **Approximate Closest Vector Problem (apprCVP):** This is similar to **apprSVP**, but finding a vector that is an approximate solution to **CVP**, i.e. for a given vector $w \in \mathbb{R}^m$, find a vector $v \in L$ such that

$$\|w - v\| \leq \psi(n)\|w - v_{\text{closest}}\|,$$

where $\psi(n)$ and L are defined the same as in **apprSVP**, and v_{closest} is the solution of **CVP**.

Remark 3.1.

- In **SVP**, we said "a shortest vector" instead of "the shortest vector", because there may be more than 1 shortest vector in a lattice, i.e. the solution is not unique.

  For example, in $\mathbb{Z}^2$, the four vectors $(0, \pm 1)$ and $(\pm 1, 0)$ are solutions to **SVP**. The same applies to **CVP** as well.

- In **apprSVP**, each choice of $\psi(n)$ gives a different solution.

  For example, one might ask for an algorithm that finds a non-zero $v \in L$ satisfying

$$\|v\| \leq 3\sqrt{n}\|v_{\text{shortest}}\| \quad \text{or} \quad \|v\| \leq 2^{\frac{n}{2}}\|v_{\text{shortest}}\|.$$

  Clearly an algorithm that solves the former is much stronger than one that solves the latter, but even the latter may be useful if the dimension n is not too large.

3.1.2 Hermite's Theorem & Minkowski's Theorem

Theorem 3.2 (Hermite's Theorem)**.** Every lattice L of dimension n contains a non-zero $v \in L$ satisfying $\|v\| \leq \sqrt{n}\det(L)^{\frac{1}{n}}$.

We will see the proof of this theorem after introducing *Minkowski's Theorem*. Now we will discuss some new concepts related to this theorem:

- For a given dimension n, *Hermite's constant* γ_n is the smallest value such that every lattice L of dimension n contains a non-zero vector $v \in L$ such that $\|v\|^2 \leq \gamma_n \det(L)^{\frac{2}{n}}$, which is the same as $\|v\| \leq \sqrt{\gamma_n}\det(L)^{\frac{1}{n}}$.

  By Theorem 3.2, we have $\gamma_n \leq n$.

  For the exact values of γ_n, we only know them when $1 \leq n \leq 8$ and $n = 24$, where the Leech lattice gives $\gamma_{24} = 4$.

  For larger n, we approximate it by $\frac{n}{2\pi e} \leq \gamma_n \leq \frac{n}{\pi e}$.

- In addition, the *Leech Lattice* is an even, unimodular lattice Λ_{24} in 24-dimensional Euclidean space E_{24}, where "even" means the square of the length of each non-zero vector in Λ_{24} is an even integer, and "unimodular" means that it is generated by a 24×24 basis matrix with determinant 1.

  With such properties, we know that $\det \Lambda_{24} = 1$ and $\min_{v \neq 0, v \in \Lambda_{24}} \|v\| = 2$, so

  $$\gamma_{24} \geq \frac{\min_{v \neq 0, v \in \Lambda_{24}} \|v\|^2}{\det \Lambda_{24}^{\frac{2}{24}}} = 4.$$

  Hence, we can conclude that $\gamma_{24} = 4$.

- We can also connect this theorem with Hadamard's Inequality 2.14 above and consider the *Hadamard Ratio*:

  For a basis $\mathcal{B} = \{v_i\}_{i=1}^n$ of a lattice L of dimension n, let the Hadamard ratio of this basis be

  $$\mathcal{H}(\mathcal{B}) := \left(\frac{\det(L)}{\prod_{i=1}^n \|v_i\|}\right)^{\frac{1}{n}}.$$

  By Hermite's Theorem, we have $\|v_i\| \leq \sqrt{n}\det(L)^{\frac{1}{n}}$, so $\prod_{i=1}^n \|v_i\| \leq \prod_{i=1}^n \sqrt{n}\det(L)^{\frac{1}{n}} = n^{\frac{n}{2}} \det(L)$.

  By Hadamard's Inequality, we have $\det(L) \leq \prod_{i=1}^n \|v_i\|$.

  Hence, combining these 2 inequalities, we get

  $$n^{-\frac{n}{2}} \leq \frac{\det(L)}{\prod_{i=1}^n \|v_i\|} \leq 1.$$

  Thus, $n^{-\frac{1}{2}} \leq \mathcal{H}(\mathcal{B}) \leq 1$, where the closer $\mathcal{H}(\mathcal{B})$ is to 1, the more *orthogonal* the basis vectors are.

- Moreover, we call the *reciprocal* of the Hadamard ratio $\frac{1}{\mathcal{H}(\mathcal{B})}$ the *orthogonality defect*, where $1 \leq \frac{1}{\mathcal{H}(\mathcal{B})} \leq n^{\frac{1}{2}}$.

The smaller the value of the orthogonality defect is (i.e. $\frac{1}{\mathcal{H}(\mathcal{B})} \to 1$), the more orthogonal the basis vectors are.

Next, before introducing Minkowski's Theorem, we need to recall some definitions from *Analysis 4*:

Definition 3.3.

1. For any $\mathbf{a} \in \mathbb{R}^n$ and any $R > 0$, the (closed) ball of radius R centered at $\mathbf{a}$ is the set $B_R(\mathbf{a}) = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x} - \mathbf{a}\| \leq R\}$.
2. Let S be a subset of $\mathbb{R}^n$.
   - S is **bounded** if the lengths of the vectors in S are bounded. Equivalently, S is bounded if there is a radius R such that S is contained within the ball $B_R(\mathbf{0})$.
   - S is **symmetric** if for every point $\mathbf{a}$ in S, the negation $-\mathbf{a}$ is also in S.
   - S is **convex** if whenever two points $\mathbf{a}$ and $\mathbf{b}$ are in S, then the entire line segment connecting $\mathbf{a}$ to $\mathbf{b}$ lies completely in S.
   - S is **closed** if it has the following property: If $\mathbf{a} \in \mathbb{R}^n$ is a point such that every ball $B_R(\mathbf{a})$ contains a point of S, then $\mathbf{a}$ is in S.

Remark 3.4. The definition of *closed* subset is actually the same as the result that S is closed if and only if every limit point of S is in S, where a limit point of S is defined by "If $\mathbf{a} \in \mathbb{R}^n$ is a point such that every ball $B_R(\mathbf{a})$ contains a point of S, then $\mathbf{a}$ is a limit point of S."

Theorem 3.5 (Minkowski's Theorem)**.** Let $L \subset \mathbb{R}^n$ be a lattice of dimension n and let $S \subset \mathbb{R}^n$ be a *bounded symmetric convex* set whose volume satisfies:

$$\mathrm{Vol}(S) > 2^n \det(L).$$

Then S contains a non-zero lattice vector.
If S is also *closed*, then it is sufficient to take $\mathrm{Vol}(S) \geq 2^n \det(L)$.

Proof. **(Part 1) Strict Larger Volume Case**

Let $\mathcal{F}$ be a fundamental domain for L. Then by Theorem 2.11, for every $a \in S$, it can be written uniquely as

$$a = v_a + w_a, \text{ for some } v_a \in \mathcal{L}, w_a \in \mathcal{F}.$$

Now, we shrink S by a factor of 2: $\frac{1}{2}S = \{\frac{1}{2}a \mid a \in S\}$.
Consider a map: $\theta : \frac{1}{2}S \to \mathcal{F}$ by $\theta(\frac{1}{2}a) = w_{\frac{1}{2}a}$.
Shrinking S by a factor of 2 changes its volume by a factor of 2^n as S is in dimension n,

so

$$\text{Vol}(\frac{1}{2}S) = \frac{1}{2^n}\,\text{Vol}(S) > \det(L) = \text{Vol}(\mathcal{F}).$$

If the map θ is *injective*, then it will be one-to-one from $\frac{1}{2}S$ to $\text{Im}\,\theta$, so $\text{Vol}(\frac{1}{2}S) = \text{Vol}(\text{Im}\,\theta)$.

Since the image is always in the codomain $\mathcal{F}$, $\text{Vol}(\text{Im}\,\theta) \leq \text{Vol}(\mathcal{F})$.

Hence, if θ is injective, then $\text{Vol}(\frac{1}{2}S) \leq \text{Vol}(\mathcal{F})$.

However, we have proved above that $\text{Vol}(\frac{1}{2}S) > \text{Vol}(\mathcal{F})$, so θ is *not injective*.

Then, there exist $\frac{1}{2}a_1, \frac{1}{2}a_2 \in \frac{1}{2}S$ (with $a_1, a_2 \in S$, $a_1 \neq a_2$) such that they correspond to the same image in $\mathcal{F}$:

$$\frac{1}{2}a_1 = v_1 + w, \frac{1}{2}a_2 = v_2 + w, \text{ where } v_1, v_2 \in L, w \in \mathcal{F}.$$

Hence,

$$\frac{1}{2}a_1 - \frac{1}{2}a_2 = v_1 - v_2 \in L.$$

Since $a_2 \in S$, then $-a_2$ is in S as S is symmetric.

Since $\frac{1}{2}a_1 - \frac{1}{2}a_2$ represents the midpoint of the line segment connecting a_1 and $-a_2$, both of which are in S, then by convexity this midpoint is in S.

In conclusion, we have found a non-zero vector $v_1 - v_2$ such that

$$v_1 - v_2 \in S \cap L.$$

(Part 2) Equal Volume Case Assume S is closed and allow $\text{Vol}(S) = 2^n \det(L)$.

For each $k \in \mathbb{Z}$ and $k \geq 1$, we expand S by a factor of $(1 + \frac{1}{k})$.

Then we will have $(1 + \frac{1}{k})S := \{(1 + \frac{1}{k})a \mid a \in S\}$ with

$$\text{Vol}((1 + \frac{1}{k})S) = (1 + \frac{1}{k})^n\,\text{Vol}(S) > \text{Vol}(S) \geq 2^n \det(L), \forall k \geq 1.$$

Applying the statement proved in Part 1 to $(1+\frac{1}{k})S$, then there exists some $v_k \neq 0$ such that $v_k \in (1 + \frac{1}{k})S \cap L$.

We claim that $bS \subseteq aS$ if $0 < b \leq a$. To prove this, let $x \in bS$, then $\exists \alpha \in S$ such that $x = b\alpha$. Let $\beta := \frac{b}{a}\alpha$; since $0 < \frac{b}{a} \leq 1$, β is on the line segment connecting 0 and α.

Because S is symmetric and convex, $0 = \frac{\alpha+(-\alpha)}{2}$ is the midpoint of the line segment of α and $-\alpha$ in S, thus $0 \in S$.

Hence, $\beta \in S$ as S is convex.

Therefore, the claim is true.

This statement gives that

$$S \subseteq (1 + \frac{1}{k+1})S \subseteq (1 + \frac{1}{k})S \subseteq \cdots \subseteq 2S, \forall k \geq 1.$$

Since $v_k \in (1 + \frac{1}{k})S \cap L, \forall k \geq 1$, then $v_k \in 2S \cap L, \forall k \geq 1$, so $\{v_k\}_{k=1}^{\infty}$ is a sequence of non-zero vectors in $2S$ and L.

Since $2S$ is bounded and L is discrete, there are finitely many distinct vectors in this sequence, so there exists v that appears infinitely often in this sequence.

Hence, we have

$$v \in \bigcap_{k=1}^{\infty}(1+\frac{1}{k})S \cap L.$$

Now, we want to prove $\bigcap_{k=1}^{\infty}(1+\frac{1}{k})S = S$. Since $S \subseteq \bigcap_{k=1}^{\infty}(1+\frac{1}{k})S$, we only need to prove $\bigcap_{k=1}^{\infty}(1+\frac{1}{k})S \subseteq S$.

Take $x \in \bigcap_{k=1}^{\infty}(1+\frac{1}{k})S$, then $x = (1+\frac{1}{k})y_k$ for some $y_k \in S$, for all $k \geq 1$.

Thus, $y_k = \frac{x}{(1+\frac{1}{k})} \to x$ as $k \to \infty$ and $\{y_k\}_{k=1}^{\infty}$ is a sequence in S, so x is a *limit point* of S and $x \in S$ as S is closed.

Therefore, we have $\bigcap_{k=1}^{\infty}(1+\frac{1}{k})S \subseteq S$ and so $\bigcap_{k=1}^{\infty}(1+\frac{1}{k})S = S$.

In conclusion, we can find a non-zero $v \in S \cap L$. □

The proof of *Hermite's Theorem* is just a simple application of *Minkowski's Theorem*.

Proof. Let $L \subseteq \mathbb{R}^n$ and S be the hypercube in $\mathbb{R}^n$, centred at 0, whose sides have length $2B$:

$$S = \{(x_1, \ldots, x_n) \in \mathbb{R}^n \mid -B \leq x_i \leq B, \forall 1 \leq i \leq n\}.$$

Then the set S is symmetric, closed, convex and bounded with volume $\mathrm{Vol}(S) = (2B)^n$.

Take $B := \det(L)^{\frac{1}{n}}$, then $\mathrm{Vol}(S) = 2^n \det(L)$. Apply Theorem 3.5 to S so that there exists $0 \neq a \in S \cap L$.

Write $a := (a_1, \ldots, a_n) \in \mathbb{R}^n$, then by definition of S, $\|a\| = \sqrt{{a_1}^2 + \cdots + {a_n}^2} \leq \sqrt{nB^2} = \sqrt{n}B = \sqrt{n}\det(L)^{\frac{1}{n}}$. □

3.1.3 The Gaussian Heuristic

In the proof of Hermite's Theorem, we apply Minkowski's Theorem to a hypercube. It is possible to improve the constant appearing in Hermite's Theorem by applying *Minkowski's Theorem* to a *hypersphere*. To do this, we need to know the volume of this "ball" in $\mathbb{R}^n$, and we need to recall the gamma function from calculus, as cited in Section 7.5.3 of [2]:

- (Definition of Gamma Function) $\Gamma(s)$ is defined on $s > 0$ by

  $$\Gamma(s) = \int_0^{\infty} t^s e^{-t} \frac{dt}{t},$$

  which is convergent for all $s > 0$.

- (Special Values of $\Gamma(s)$)

  1.$\Gamma(1) = 1, \Gamma(\frac{1}{2}) = \sqrt{\pi}$ and $\Gamma(s+1) = s\Gamma(s)$.

  2. For all integers $n \geq 1$, we have $\Gamma(n+1) = n!$.

- (Stirling's Formula) For *larger values* of s, we have $\Gamma(1+s)^{\frac{1}{s}} \approx \frac{s}{e}$.

We need to introduce the gamma function here, because it appears when we calculate the volume of a ball in $\mathbb{R}^n$:

Theorem 3.6. Let $\mathbb{B}_R(a)$ be a ball of radius R, centred at a in $\mathbb{R}^n$. Then the volume of $\mathbb{B}_R(a)$ is

$$\mathrm{Vol}(\mathbb{B}_R(a)) = \frac{\pi^{\frac{n}{2}} R^n}{\Gamma(1+\frac{n}{2})} \tag{2}$$

For *large values* of n, we can approximate the volume by using Stirling's Formula:

$$\mathrm{Vol}(\mathbb{B}_R(a))^{\frac{1}{n}} = \frac{\pi^{\frac{1}{2}} R}{\sqrt{\Gamma(1+\frac{n}{2})^{\frac{2}{n}}}} \approx \frac{\pi^{\frac{1}{2}} R}{\sqrt{n/2e}} = \sqrt{\frac{2\pi e}{n}} R \tag{3}$$

Now we know the volume of $\mathbb{B}_R(0)$, and it is closed, symmetric, convex and bounded, so we can apply *Minkowski's Theorem* to this ball with a specifically chosen R:

Take $R > 0$ such that $\mathrm{Vol}(\mathbb{B}_R(0)) \geq 2^n \det(L)$; then there exists a non-zero lattice vector $v \in \mathbb{B}_R(0)$, i.e. $\|v\| \leq R$.

For larger values of n, we require $\mathrm{Vol}(\mathbb{B}_R(a))^{\frac{1}{n}} \approx \sqrt{\frac{2\pi e}{n}} R \geq 2 \det(L)^{\frac{1}{n}}$, which means we need to choose R such that

$$R \gtrapprox \sqrt{\frac{2n}{\pi e}} \det(L)^{\frac{1}{n}}.$$

Hence, for larger values of n, there exists a non-zero $v \in L$ satisfying

$$\|v\| \lessapprox \sqrt{\frac{2n}{\pi e}} \det(L)^{\frac{1}{n}}.$$

Comparing this result with Theorem 3.2, we have improved the estimate by a factor of $\sqrt{\frac{2}{\pi e}} \approx 0.484$.

Although the exact length of a shortest non-zero lattice vector is generally unknown in high dimensions, we can obtain a useful heuristic using a probabilistic argument.

Let $L \subset \mathbb{R}^n$ be a lattice with fundamental domain F, where $\mathrm{Vol}(F) = \det(L)$. For a large Euclidean ball $\mathbb{B}_R(0) = \{x \in \mathbb{R}^n : \|x\| \leq R\}$, the number of lattice points contained in the ball is approximately

$$\#(\mathbb{B}_R(0) \cap L) \approx \frac{\mathrm{Vol}(\mathbb{B}_R(0))}{\det(L)}.$$

The intuition behind this approximation is that the translates $v + F, v \in L$, form a tiling of $\mathbb{R}^n$ without overlapping (except along their boundaries), as proved in Theorem 2.11.

Consequently, each lattice point corresponds to one copy of the fundamental domain. For a sufficiently large ball, almost all of these copies lie either entirely inside or entirely outside the ball; only those intersecting the boundary are counted imperfectly.

Since the boundary region becomes negligible compared with the volume of the ball as R increases, the approximation becomes increasingly accurate.

Example 3.7. For the standard lattice $L = \mathbb{Z}^2$, we have $\det(\mathbb{Z}^2) = 1$, so

$$\#\big(\mathbb{B}_R(0) \cap \mathbb{Z}^2\big) \approx \mathrm{Area}(\mathbb{B}_R(0)) = \pi R^2.$$

Equivalently,

$$\#\left\{(x,y)\in\mathbb{Z}^2 : x^2+y^2\le R^2\right\} = \pi R^2 + (\text{error term}).$$

Estimating the growth of the error term is the classical *Gauss circle problem.*

In higher dimensions, the problem becomes more difficult and the approximation becomes less accurate. As the dimension n increases, the error contributed by lattice points near the boundary of the ball becomes relatively larger, and the estimate does not become reliable until the radius R is sufficiently large. Consequently, the approximation

$$\#\{v\in L : \|v\|\le R\} \approx \frac{\mathrm{Vol}(\mathbb{B}_R(0))}{\mathrm{Vol}(F)} \tag{4}$$

may be rather inaccurate when the dimension is high and the radius R is not sufficiently large.

However, we can still ask for the value of R that makes the RHS of Equation 4 equal to 1, which helps us to find the *first non-zero lattice vector* in $\mathbb{B}_R(0)$, i.e. the shortest vector in the lattice.

Assume n is large, then $\mathrm{Vol}(\mathbb{B}_R(a)) \approx (\frac{2\pi e}{n})^{\frac{n}{2}} R^n$ by Equation 3.

Hence, RHS$= 1 \approx \frac{(\frac{2\pi e}{n})^{\frac{n}{2}} R^n}{\det(L)} \Rightarrow (\frac{2\pi e}{n})^{\frac{n}{2}} R^n \approx \det(L) \Rightarrow R \approx \sqrt{\frac{n}{2\pi e}}\det(L)^{\frac{1}{n}}$.

This leads to the following heuristic:

Definition 3.8. Let L be a lattice of dimension n. The *Gaussian expected shortest length* is defined by

$$\sigma(L) = \sqrt{\frac{n}{2\pi e}}(\det L)^{1/n}.$$

The *Gaussian heuristic* states that the length of a shortest non-zero vector in a "randomly chosen" lattice satisfies

$$\|v_{\text{shortest}}\| \approx \sigma(L).$$

More precisely, for every fixed $\varepsilon > 0$, there exists an integer N such that whenever $n \ge N$, a randomly chosen lattice of dimension n satisfies

$$(1-\varepsilon)\sigma(L) \le \|v_{\text{shortest}}\| \le (1+\varepsilon)\sigma(L).$$

This heuristic predicts the typical length of the shortest non-zero lattice vector and is widely used in the analysis of high-dimensional lattices and lattice-based cryptography, although it is not a rigorous theorem for arbitrary lattices.

Remark 3.9. For smaller values of n, it is better to use the exact volume of $\mathbb{B}_R(0)$ given by Equation 2, in which case the Gaussian expected shortest length is

$$\sigma(L) = \frac{(\Gamma(1+\frac{n}{2})\cdot\det L)^{1/n}}{\sqrt{\pi}}.$$

When n is small, it is more accurate to use the above equation for the Gaussian expected shortest length than Definition 3.8, but when n becomes larger, their difference becomes quite small.

Example 3.10. Let $(m_1, \ldots, m_n, S)$ be a *subset-sum problem*, which is explained in 7.2 in [2]. The associated lattice will be $L_{M,S}$ generated by the rows of a $(n+1) \times (n+1)$ matrix with $\det(L_{M,S}) = 2^n S$. Also, in 7.2, we have the result $S^{\frac{1}{n}} \approx 4$. Hence, we may approximate the Gaussian expected shortest length:

$$\begin{aligned}
\sigma(L_{M,S}) &= \sqrt{\frac{n+1}{2\pi e}} (\det L_{M,S})^{1/(n+1)} \\
&= \sqrt{\frac{n+1}{2\pi e}} (2^n S)^{1/(n+1)} \\
&\approx \sqrt{\frac{n}{2\pi e}} \cdot 2S^{1/n} \\
&\approx \sqrt{\frac{n}{2\pi e}} \cdot 8 \\
&\approx 1.936\sqrt{n}.
\end{aligned}$$

We will find that the Gaussian heuristic is useful in quantifying the difficulty of locating short vectors in lattices. In particular, if the actual shortest vector of a particular lattice L is significantly shorter than $\sigma(L)$, then lattice reduction algorithms such as **LLL** seem to have a much easier time locating the shortest vector, since such a vector stands out as an unusually short outlier rather than being just another vector of typical length.

A similar argument leads to a Gaussian heuristic for **CVP**. Thus, if $L \subset \mathbb{R}^n$ is a random lattice of dimension n and $w \in \mathbb{R}^n$ is a random point, then we expect that the lattice vector $v \in L$ closest to w satisfies

$$\|v - w\| \approx \sigma(L).$$

And just as for **SVP**, if L contains a point that is significantly closer than $\sigma(L)$ to w, then lattice reduction algorithms have an easier time solving **CVP**, as that point is again distinguishable from the typical spacing of lattice points near w.

3.2 Babai's Algorithm

Suppose a lattice $L \subseteq \mathbb{R}^n$ has basis $\{v_1, \ldots, v_n\}$ that is pairwise orthogonal, i.e. $\langle v_i, v_j \rangle = 0$ for $i \neq j$.

- For **SVP**, for any $v \in L$, there exist $a_1, \ldots, a_n \in \mathbb{Z}$ such that $v = \sum_{i=1}^n a_i v_i$. Then, $\|v\|^2 = \langle \sum_{i=1}^n a_i v_i, \sum_{i=1}^n a_i v_i \rangle = \sum_{i,j=1}^n a_i a_j \langle v_i, v_j \rangle = \sum_{i=1}^n {a_i}^2 \|v_i\|^2$.

  If we want to minimize $\|v\|$, then we need to minimize ${a_i}^2$ for each $i = 1, \ldots, n$, without setting all a_i equal to 0. For example, set $a_i = 1$ and $a_j = 0$ for $j \neq i$ and this will give all possible minimal cases, i.e. the shortest vectors will be in $\{\pm v_1, \ldots, \pm v_n\}$.

- For **CVP**, suppose $w \in \mathbb{R}^n$. Since $\{v_i\}_{i=1}^n$ are linearly independent and have size n, then they are a basis for $\mathbb{R}^n$.

Then, $w = \sum_{i=1}^{n} t_i v_i$, for some $t_i \in \mathbb{R}$, and then the squared distance between w and $v \in L$ is

$$\|v - w\|^2 = \|\sum_{i=1}^{n}(a_i - t_i)v_i\|^2 = \sum_{i=1}^{n}(a_i - t_i)^2\|v_i\|^2.$$

To minimize $\|v - w\|^2$, we can take each $a_i \in \mathbb{Z}$ to be the integers closest to the corresponding t_i.

The above method for **CVP** works because we are dealing with a pairwise orthogonal basis (called a "good" basis). What will happen if the basis is arbitrary?

A basis $\{v_1, \ldots, v_n\}$ for a lattice L determines a fundamental domain $\mathcal{F}$, and the translates of $\mathcal{F}$ by the elements of L cover the entire space $\mathbb{R}^n$ by Theorem 2.11.

Then, for any $w \in \mathbb{R}^n$, there exists $v \in L$ such that $w \in \mathcal{F} + v$. In this case, we take the vertex of parallelepiped $\mathcal{F} + v$ that is closest to w as our *hypothetical solution to CVP*.

To find the closest vertex, we may write

$$w = v + \epsilon_1 v_1 + \cdots + \epsilon_n v_n \quad \text{for some } 0 \leq \epsilon_1, \ldots, \epsilon_n < 1.$$

Then, define $\hat{\epsilon}_i = \begin{cases} 0 & \text{if } \epsilon_i < \frac{1}{2} \\ 1 & \text{if } \epsilon_i \geq \frac{1}{2} \end{cases}$ for each $i = 1, \ldots, n$, and the closest vertex will be $\hat{v} := v + \hat{\epsilon}_1 v_1 + \cdots + \hat{\epsilon}_n v_n$.

The above works well if the basis vectors are *fairly orthogonal* ("Good" Basis), but this will not work for the ones that are *nearly parallel or opposite* ("Bad" Basis). This is because the parallelepiped $\mathcal{F} + v$ is too flat or narrow, so that the non-lattice vector w may be much closer to some lattice point outside the parallelepiped, compared to the closest vertex, just like in Figure 5 below.

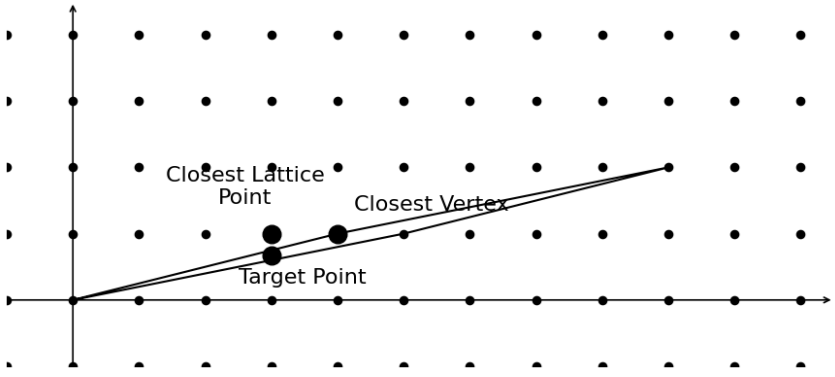


Figure 5: Babai's algorithm works poorly if the basis is "bad".

To make this more intuitive, let us just set $L = \mathbb{Z}^2$ in 2D in the example below:

Example 3.11. Let $\{v_1, v_2\} = \{(1,1),(0,1))\}$, $\{\tilde{v}_1, \tilde{v}_2\} = \{(4,1),(5,1)\}$ and take $w = (3, \frac{2}{3}) \in \mathbb{R}^2$, then clearly we can find that the closest vector in L is $(3,1)$ and $\{v_1, v_2\}$ is the "good" basis, $\{\tilde{v}_1, \tilde{v}_2\}$ is the "bad" basis, like in Figure 6.
If we apply Babai's Algorithm to $\{v_1, v_2\}$, we will have $v = (3,0) \in L$ and $w = v + \frac{2}{3}v_2$,

then the closest vertex will be $\hat{v} = v + v_2 = (3, 1)$, which matches the closest lattice point.

However, for $\{\tilde{v}_1, \tilde{v}_2\}$, we have $w = \frac{1}{3}\tilde{v}_1 + \frac{1}{3}\tilde{v}_2$. Babai's algorithm rounds both coefficients to zero and outputs $\hat{v} = 0\tilde{v}_1 + 0\tilde{v}_2 = (0, 0)$, which is not the closest lattice point.

Notice that the closest vertex of the fundamental parallelepiped is actually $\tilde{v}_1 = (4, 1)$, rather than the vertex selected by Babai's rounding, which does not match $(3, 1)$ either.

This discrepancy occurs because the bad basis produces a highly skewed fundamental parallelepiped, so coordinate-wise rounding no longer agrees with the Euclidean nearest-neighbour search.

This example shows that the success of Babai's algorithm depends strongly on the choice of basis: a good basis makes coordinate rounding reliable, while a bad basis may lead to an incorrect closest lattice point or even incorrect closest vertex.

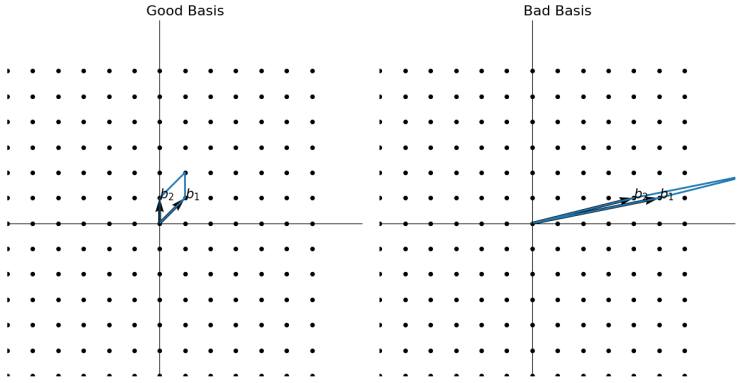


Figure 6: "Good" basis and "Bad" Basis in $\mathbb{Z}^2$

Now, we will introduce the *Babai's Closest Vertex Algorithm*:

Theorem 3.12. Let $L \subseteq \mathbb{R}^n$ be a lattice with basis $v_1, \ldots, v_n$, and let $w \in \mathbb{R}^n$ be an arbitrary vector. If the vectors are *sufficiently orthogonal* to one another, then the following algorithm solves CVP.

Write $w = t_1 v_1 + t_2 v_2 + \cdots + t_n v_n$ with $t_1, \ldots, t_n \in \mathbb{R}$.
Set $a_i = \lfloor t_i \rceil$ for $i = 1, 2, \ldots, n$.
Return the vector $v = a_1 v_1 + a_2 v_2 + \cdots + a_n v_n$.

In general, if the vectors in the basis are reasonably orthogonal to one another, then the algorithm solves some version of apprCVP, but if the basis vectors are highly non-orthogonal, then the vector returned by the algorithm is generally far from the lattice vector that is closest to w.

Remark 3.13. The notation $\lfloor t_i \rceil$ used in Babai's algorithm denotes the nearest integer to t_i, rather than the usual floor function. In other words,

$$\lfloor t_i \rceil = \operatorname{argmin}_{a \in \mathbb{Z}} |t_i - a| = a_i \text{ if } a_i - \frac{1}{2} \leq t_i < a_i + \frac{1}{2}.$$

Thus, the algorithm rounds each coefficient t_i to the closest integer independently.

Let us see a harder example in $\mathbb{R}^2$ applying Babai's Algorithm to two bases of the same lattice:

Example 3.14. Let $L \subset \mathbb{R}^2$ be the lattice given by 2 bases

$$v_1 = (137, 312) \quad \text{and} \quad v_2 = (215, -187).$$

$$v_1' = (1975, 438) = 5v_1 + 6v_2 \quad \text{and} \quad v_2' = (7548, 1627) = 19v_1 + 23v_2.$$

We are going to use Theorem 3.12 to find a vector in L that is close to the vector

$$w = (53172, 81743).$$

Let us start with $\{v_1, v_2\}$. The first step is to express w as a linear combination of v_1 and v_2 using real coordinates. Thus we need to find $t_1, t_2 \in \mathbb{R}$ such that

$$w = t_1 v_1 + t_2 v_2.$$

This gives the two linear equations

$$53172 = 137t_1 + 215t_2 \quad \text{and} \quad 81743 = 312t_1 - 187t_2.$$

Solving the equations gives:

$$t_1 \approx 296.85 \quad \text{and} \quad t_2 \approx 58.15.$$

Babai's algorithm tells us to round t_1 and t_2 to the nearest integer:

$$\lfloor t_1 \rceil = 297 \quad \text{and} \quad \lfloor t_2 \rceil = 58.$$

Hence,

$$v = 297(137, 312) + 58(215, -187) = (53159, 81818) \in L,$$

which is the closest lattice vector to w, and we can compute their distance:

$$\|v - w\| \approx 76.12$$

is indeed quite small.

We now try to solve the same closest vector problem in the same lattice, but using the new basis $\{v_1', v_2'\}$:

The system of linear equations $(53172, 81743) = (t_1, t_2)\begin{pmatrix} 1975 & 438 \\ 7548 & 1627 \end{pmatrix}$ has the solution

$$(t_1, t_2) \approx (5722.66, -1490.34),$$

so we set

$$v' = 5723v_1' - 1490v_2' = (56405, 82444) \in L.$$

However, v' is not particularly close to w, since

$$\|v' - w\| \approx 3308.12.$$

This is to be expected, since $\{v_1, v_2\}$ are *fairly orthogonal* to one another, while $\{v_1', v_2'\}$ are *highly non-orthogonal*, which can be shown by the Hadamard ratio

$$\mathcal{H}(v_1, v_2) = (\frac{\det(L)}{\|v_1\|\|v_2\|})^{1/2} \approx 0.977, \text{ which is reasonably close to 1.}$$

$$\mathcal{H}(v_1', v_2') = (\frac{\det(L)}{\|v_1'\|\|v_2'\|})^{1/2} \approx 0.077, \text{ which is much smaller than 1.}$$

3.3 Cryptosystems based on Hard Lattice Problems

The computational difficulty of lattice problems such as the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP) provides a natural foundation for constructing public-key cryptosystems.

The main idea behind lattice-based cryptography is to create a situation where a legitimate user can efficiently solve a lattice problem using secret information, while an attacker without this information is required to solve a computationally difficult problem.

In a public-key cryptosystem, each user has a pair of keys: a public key and a private key. The public key is made available to everyone and is used for encryption, whereas the private key is kept secret and is used for decryption.

The security of such systems relies on the existence of a one-way operation: it should be easy to perform encryption using the public key, but computationally difficult to recover the original message without knowledge of the private key.

Figure 7 shows how a public key cryptosystem works, and we can conclude that the generation of the pair of keys $(Pub_B, Priv_B)$ is important.

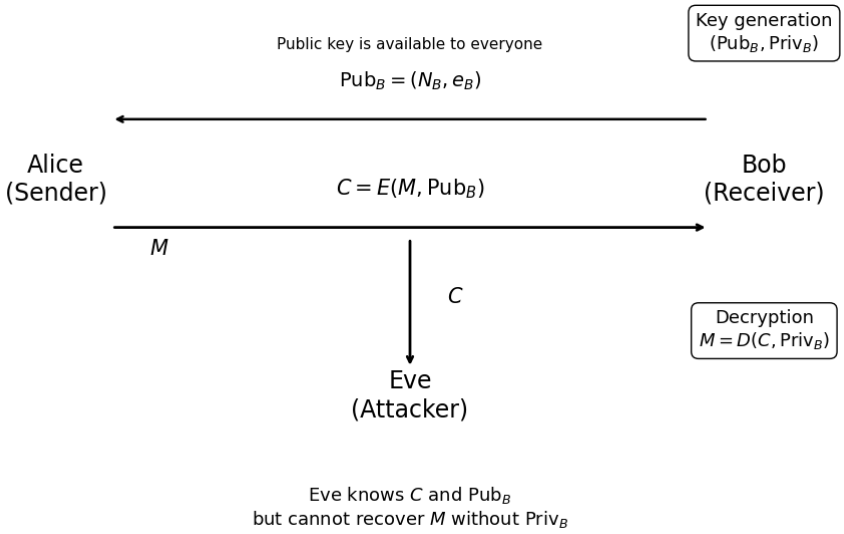


Figure 7: Public Key Cryptosystem under RSA

Traditional public-key cryptosystems, such as RSA [Chapter 3.2 in [2]] and Elliptic Curve Cryptography (ECC) [Chapter 6.5 in [2]], are based on the hardness of integer factorisation and discrete logarithm problems. In contrast, lattice-based cryptosystems rely on the difficulty of solving lattice problems in high dimensions. Since no efficient quantum algorithms are currently known for solving these problems, lattice-based cryptography has become one of the most promising approaches for post-quantum cryptography.

During the mid-1990s, several lattice-based cryptosystems were introduced, with the most influential early examples being the Ajtai–Dwork cryptosystem, the GGH cryptosystem, and the NTRU cryptosystem.

- **The Ajtai–Dwork Cryptosystem.**

  The Ajtai–Dwork cryptosystem was one of the first constructions to provide a connection between the security of a cryptosystem and worst-case lattice problems. Ajtai and Dwork showed that breaking their system efficiently would allow certain worst-case lattice problems to be solved in polynomial time. This provided an important theoretical foundation for lattice-based cryptography.

  However, the original construction was not practical because of its extremely large key size, which grows as $O(n^4)$. Later analysis also showed that practical implementations of the system were vulnerable to attacks.

- **The GGH Cryptosystem.**

  The GGH cryptosystem, proposed by Goldreich, Goldwasser, and Halevi, is a direct application of lattice geometry. Its security is based on the difficulty of the Closest Vector Problem.

  The private key is a good basis of a lattice, while the public key is a bad basis of the same lattice. During encryption, a message is represented as a lattice vector using the public basis, and a small random perturbation is added to hide the lattice point. The resulting ciphertext is therefore close to an unknown lattice vector.

The legitimate receiver, who knows the good basis, can apply Babai's algorithm to efficiently recover the closest lattice point and hence decrypt the message. An attacker with only the bad basis would need to solve the CVP in the lattice, which is assumed to be computationally difficult.

The public key consists of an entire lattice basis matrix, resulting in a key size of approximately $O(n^2 \log n)$ bits. However, later work showed that lattice reduction algorithms could exploit weaknesses in the original construction, reducing its practical security.

- **The NTRU Cryptosystem.**

  The NTRU cryptosystem, introduced by Hoffstein, Pipher, and Silverman, is based on polynomial rings and convolution multiplication. Although it is naturally described algebraically, its security can also be interpreted in terms of lattice problems.

  The underlying NTRU lattice has dimension $n = 2N$, and the security of the system is related to both the Shortest Vector Problem and the Closest Vector Problem. Recovering the private key corresponds to finding a shortest vector in the lattice, while recovering the plaintext can be viewed as solving CVP.

  A key advantage of NTRU is its use of special algebraic structure. The public key is represented using a circulant matrix, meaning that each row is obtained by rotating the previous row. Therefore, only a small amount of information is required to describe the public key, giving a key size of approximately $O(n \log n)$, which is significantly smaller than that of GGH.

Among these early lattice-based cryptosystems, the GGH construction provides the most direct illustration of how lattice geometry can be used for encryption.

Therefore, we first study the GGH cryptosystem in detail before introducing the algebraic structures, such as convolution rings, that underlie the NTRU cryptosystem.

3.4 The GGH Public Key Cryptosystem

To explain the idea behind the GGH public key cryptosystem in detail, we separate it into three steps:

Here, we assume Bob wants to send a message to Alice and avoid Eve knowing what it is, so Alice needs to generate a pair of keys and share the public key with Bob.

Step 1: Key Generation of Alice

- Alice begins by choosing a set of linearly independent and *reasonably orthogonal* vectors $\{v_1, \ldots, v_n\}$.

  To do this, one way is to fix a parameter d and choose coordinates of $\{v_i\}_{i=1}^n$ randomly between $-d$ and d, then check the orthogonality of the chosen basis by computing its *Hadamard ratio* and verify the value is close to 1.

  Hence, the private key of Alice is chosen as

$$Priv_A = \{v_1, \ldots, v_n\}.$$

- Let $V \in \mathbb{R}^{n\times n}$ be the matrix whose rows are $\{v_1, \ldots, v_n\}$, and $L \subset \mathbb{R}^n$ be the lattice generated by $\{v_1, \ldots, v_n\}$.

  Next, choose a matrix $U \in GL(\mathbb{Z}, n)$, i.e. $\det(U) = \pm 1$ and all entries of U are integers. We can create U as a product of a large number of randomly chosen elementary matrices, which are restricted as in Exercise 2.23.

  Then, $W := UV$ is a matrix consisting of rows $\{w_1, \ldots, w_n\}$, which is a new basis of L by Proposition 2.5.

  Moreover, Alice can compute the *Hadamard ratio* of this basis again and verify this value is close to 0, i.e. $\{w_1, \ldots, w_n\}$ is a "bad" basis of L.

  Hence, the public key of Alice is chosen as

$$Pub_A = \{w_1, \ldots, w_n\}.$$

Step 2: Encryption

Now, Bob wants to send a message to Alice and he has the public key of Alice $\{w_1, \ldots, w_n\}$, then he may first select a small vector $m = \begin{pmatrix} m_1 & \ldots & m_n \end{pmatrix} \in \mathbb{R}^n$ with $m_i \in \mathbb{Z}$ as the plaintext. (e.g. a binary vector).

Bob also needs to choose a small *random* perturbation vector $r \in \mathbb{R}^n$. For example, he may choose the coordinates of r randomly between $-\delta$ and δ, where δ is a fixed public parameter and $\delta \ll d$.

Then, the ciphertext sent to Alice is the vector

$$e = mW + r = \sum_{i=1}^{n} m_i w_i + r.$$

Notice that $mW \in L$ but $e \notin L$, and e is quite close to mW since r is small.

Step 3: Decryption

When Alice receives the ciphertext e, she has the private key $\{v_1, \ldots, v_n\}$.

Then, by applying Babai's Algorithm (Theorem 3.12) directly, Alice can use the "good" basis $\{v_1, \ldots, v_n\}$ to find the closest vector to e in L. Denote it by v. From the information provided above, $v = mW$, so $m = vW^{-1}$ is recovered by Alice.

Moreover, Eve only has the public key $\{w_1, \ldots, w_n\}$ and the ciphertext e, so it is hard for her to recover m using Babai's Algorithm, since $\{w_1, \ldots, w_n\}$ is a "bad" basis. Therefore, recovering m without the private key is computationally difficult for Eve.

We can summarize the above process of this algorithm by the table below:

Alice	Bob
Key creation	
Choose a good basis $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$. Choose an integer matrix U satisfying $\det(U) = \pm 1$. Compute a bad basis $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_n$ as the rows of $W = UV$. Publish the public key $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_n$.	
Encryption	
	Choose small plaintext vector $\boldsymbol{m}$. Choose random small vector $\boldsymbol{r}$. Use Alice's public key to compute $\boldsymbol{e} = m_1\boldsymbol{w}_1 + \cdots + m_n\boldsymbol{w}_n + \boldsymbol{r}$. Send the ciphertext $\boldsymbol{e}$ to Alice.
Decryption	
Use Babai's algorithm to compute the vector $\boldsymbol{v} \in L$ closest to $\boldsymbol{e}$. Compute $\boldsymbol{v}W^{-1}$ to recover $\boldsymbol{m}$.	

Table 1: The GGH cryptosystem

Below we illustrate the GGH cryptosystem with a three-dimensional example. All numerical calculations in this example are verified using Python, and the implementation is provided in Appendix A.1.

Example 3.15. As before, we again work with Alice, Bob, and Eve:
Alice chooses the following private good basis:

$$v_1 = (-97, 19, 19), \qquad v_2 = (-36, 30, 86), \qquad v_3 = (-184, -64, 78).$$

The lattice L spanned by v_1, v_2, v_3 has determinant

$$\det(L) = 859516,$$

and the Hadamard ratio of the basis is

$$\mathcal{H}(v_1, v_2, v_3) = \left(\frac{\det(L)}{\|v_1\|\|v_2\|\|v_3\|}\right)^{1/3} \approx 0.74620.$$

Alice generates her public key by multiplying her private basis by a unimodular

matrix

$$U = \begin{pmatrix} 4327 & -15447 & 23454 \\ 3297 & -11770 & 17871 \\ 5464 & -19506 & 29617 \end{pmatrix},$$

where

$$\det(U) = -1.$$

The resulting public basis is

$$\begin{aligned} w_1 &= (-4179163, -1882253, 583183), \\ w_2 &= (-3184353, -1434201, 444361), \\ w_3 &= (-5277320, -2376852, 736426). \end{aligned}$$

The Hadamard ratio of the public basis is very small:

$$\mathcal{H}(w_1, w_2, w_3) = \left(\frac{\det(L)}{\|w_1\|\|w_2\|\|w_3\|}\right)^{1/3} \approx 0.0000208.$$

Bob encrypts the plaintext

$$m = (86, -35, -32)$$

using the randomly chosen perturbation

$$r = (-4, -3, 2).$$

The ciphertext is computed as

$$e = mW + r,$$

where W is the matrix whose rows are the public basis vectors:

$$W = \begin{pmatrix} -4179163 & -1882253 & 583183 \\ -3184353 & -1434201 & 444361 \\ -5277320 & -2376852 & 736426 \end{pmatrix}.$$

Therefore,

$$\begin{aligned} e &= (86, -35, -32) \begin{pmatrix} -4179163 & -1882253 & 583183 \\ -3184353 & -1434201 & 444361 \\ -5277320 & -2376852 & 736426 \end{pmatrix} + (-4, -3, 2) \\ &= (-79081427, -35617462, 11035473). \end{aligned}$$

To decrypt, Alice applies Babai's algorithm using her private basis. She first expresses the ciphertext as a real linear combination of the private basis:

$$e \approx 81878.97v_1 - 292300.00v_2 + 443815.04v_3.$$

Rounding the coefficients gives

$$(81879, -292300, 443815),$$

and hence the closest lattice vector is

$$\begin{aligned} v &= 81879v_1 - 292300v_2 + 443815v_3 \\ &= (-79081423, -35617459, 11035471). \end{aligned}$$

The plaintext is recovered by writing v in terms of the public basis:

$$v = 86w_1 - 35w_2 - 32w_3.$$

Now suppose Eve tries to decrypt the ciphertext using only the public basis. Applying Babai's algorithm with the public basis gives

$$e \approx 75.76w_1 - 34.52w_2 - 24.18w_3.$$

After rounding, Eve obtains

$$v' = 76w_1 - 35w_2 - 24w_3,$$

where

$$v' = (-79508353, -35809745, 11095049).$$

This lattice vector is close to e, but it corresponds to the incorrect plaintext

$$(76, -35, -24) \neq (86, -35, -32).$$

The difference between the errors is

$$\|e - v\| \approx 5.39, \qquad \|e - v'\| \approx 472004.09.$$

Therefore, Babai's algorithm succeeds with Alice's private good basis but fails with Eve's public bad basis.

To further illustrate the behaviour of the GGH cryptosystem, we implement the above example in Python and perform multiple random trials. We fix Alice's private key as the good basis $\{v_1, v_2, v_3\}$, while generating a random unimodular matrix U to produce the public basis. The matrix U is chosen such that the resulting public basis has a sufficiently small Hadamard ratio, making it a bad basis for Babai's algorithm.

In each trial, the plaintext m and the perturbation vector r are randomly generated with small entries. The encryption and decryption processes are then performed using the public and private bases respectively. The experiment shows that, for this three-dimensional lattice, the chosen private basis provides stable decryption: all 30 random trials successfully recover the original plaintext.

Furthermore, the implementation is extended to encrypt string messages. The input

string is first transformed into numerical arrays and partitioned into 3×1 vectors so that the same private basis V can be applied. After performing the GGH encryption and decryption procedures on each block, the resulting vectors are converted back into the original string. This shows that the cryptosystem can handle general message inputs beyond numerical examples.

As an example, the string message `"Lattice-Based Cryptography"` is used as the input. The output of the implementation is shown below:

```
Original Message:
Lattice-Based Cryptography
--------------------------------------
The array of message (input):
[array([ 76,  97, 116]), array([116, 105,  99]), array([101,  45,
    66]), array([ 97, 115, 101]), array([100,  32,  67]), array([114,
    121, 112]), array([116, 111, 103]), array([114,  97, 112]), array
    ([104, 121,   0])]
--------------------------------------
Encrypted ciphertext:
[array([ 2659522,   626856, -1131218]), array([ 4320830,  1024328,
    -1839792]), array([ 4153637,   986585, -1769246]), array([
    3394644,   803251, -1444922]), array([ 4230580,  1004865,
    -1802024]), array([ 4108027,   972713, -1748796]), array([
    4272605,  1012567, -1819152]), array([ 4328325,  1025256,
    -1842712]), array([ 3475325,   829216, -1481518])]
--------------------------------------
Decrypted result:
[array([ 76,  97, 116]), array([116, 105,  99]), array([101,  45,
    66]), array([ 97, 115, 101]), array([100,  32,  67]), array([114,
    121, 112]), array([116, 111, 103]), array([114,  97, 112]), array
    ([104, 121,   0])]
--------------------------------------
Plaintext:
Lattice-Based Cryptography
```

The decrypted output exactly matches the original input message, confirming that the GGH encryption and decryption procedures can successfully recover general string messages using the chosen three-dimensional private basis. The complete Python implementation and additional computational details are provided in Appendix A.2.

Remark 3.16.

- The GGH cryptosystem is insecure in low dimensions since efficient lattice reduction algorithms can recover good bases.

  For example, Gauss's algorithm solves the two-dimensional case, while the **LLL algorithm** generalises this idea to higher dimensions.

- GGH is a probabilistic cryptosystem because the random perturbation r produces different ciphertexts for the same plaintext.

Reusing the same perturbation or sending the same plaintext with different perturbations may introduce security risks.

- An alternative GGH construction uses

$$e = rW + m,$$

where Alice first finds rW using Babai's algorithm and then recovers the plaintext by

$$m = e - rW.$$

PS: I have created an interactive page where you can choose to play as Alice, Bob, or Eve, which may be helpful for learning how the GGH cryptosystem works. See the GGH Interactive.

3.5 NTRU Public Key Cryptosystem

3.5.1 Convolution Polynomial Rings

In this subsection, we will introduce a special type of polynomial factor ring that is used in *the NTRU public key cryptosystem.*

Definition 3.17. For a fixed positive integer N, the ring of convolution polynomials of rank N is the factor ring

$$R = \frac{\mathbb{Z}[x]}{\langle x^N - 1\rangle}.$$

Similarly, the ring of convolution polynomials (modulo q) is the factor ring

$$R_q = \frac{(\mathbb{Z}/q\mathbb{Z})[x]}{\langle x^N - 1\rangle}.$$

Recall Example 1.41: we know that every element of R or R_q has a unique representative of the form

$$a_0 + a_1x + a_2x^2 + \cdots + a_{N-1}x^{N-1}$$

with $a_i \in \mathbb{Z}$ or $\mathbb{Z}/q\mathbb{Z}$ respectively.

It is usually more convenient to represent a polynomial $a(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{N-1}x^{N-1}$ with the vector formed by its coefficients

$$\begin{pmatrix} a_0 & a_1 & a_2 & \dots & a_{N-1} \end{pmatrix} \in \mathbb{Z}^N \text{ or } (\mathbb{Z}/q\mathbb{Z})^N.$$

Next, we consider the addition and multiplication in R or R_q:

For addition, the rule for polynomials just corresponds to the one for vectors:

$$a(x)+b(x) \leftrightarrow \begin{pmatrix} a_0+b_0 & a_1+b_1 & a_2+b_2 & \dots & a_{N-1}+b_{N-1} \end{pmatrix} =: \begin{pmatrix} c_0 & c_1 & c_2 & \dots & c_{N-1} \end{pmatrix}.$$

For multiplication, we write it as $\star$ in R or R_q:

Proposition 3.18. The product of two polynomials $a(x), b(x) \in R$ is given by the formula

$$a(x) \star b(x) =: c(x) \text{ with } c_k = \sum_{0 \le i,j \le N-1, i+j=k} a_i b_j.$$

The product of $a(x), b(x) \in R_q$ is given by similar formula, except that the value of c_k is reduced modulo q.

Proof. Easy to prove by the product of finite sums. Exercise. □

Remark 3.19. The convolution product of two vectors is given by

$$\begin{pmatrix} a_0 & a_1 & a_2 & \dots & a_{N-1} \end{pmatrix} \star \begin{pmatrix} b_0 & b_1 & b_2 & \dots & b_{N-1} \end{pmatrix} = \begin{pmatrix} c_0 & c_1 & c_2 & \dots & c_{N-1} \end{pmatrix}$$

where c_k is defined in Proposition 3.18.

Clearly there is a natural map from R to R_q by reducing each coefficient modulo q; we may now consider a map from R_q to R:

Definition 3.20. Let $a(x) \in R_q$, the *centre-lift* of $a(x)$ to R is the *unique* polynomial $a'(x) \in R$ satisfying $a'(x) \pmod q = a(x)$, whose coefficients are chosen to be

$$-\frac{q}{2} < a'_i \le \frac{q}{2}.$$

For example, if $q = 2$, then $-1 < a'_i \le 1 \Rightarrow a'_i = 0, 1$, so the centre-lift of $a(x)$ is a binary polynomial.

One thing to be noticed is that the sum or product of the centre-lifts may *not* be equal to the centre-lift of the sum or product. We can see this from the below example:

Example 3.21. Let $N = 5, q = 7$, choose $a(x) = 5 + 3x - 6x^2 + 2x^3 + 4x^4, b(x) = 3 + 5x^2 - 6x^3 + 3x^4 \in R_7$.
Then the coefficients for the centre-lifts are in $\{-3, -2, \dots, 2, 3\}$:

$$a'(x) = -2 + 3x + x^2 + 2x^3 - 3x^4,$$

$$b'(x) = 3 - 2x^2 + x^3 + 3x^4.$$

Hence,

$$a'(x) \star b'(x) = 20x + 10x^2 - 11x^3 - 14x^4 \in R,$$

$$\text{Centre-lift of } (a \star b) = -x + 3x^2 + 3x^3 \in R_7.$$

We can see that they are not equal, although they are congruent modulo 7.

Proposition 3.22. Let q be prime. Then $a(x) \in R_q$ has a multiplicative inverse if

and only if

$$\gcd(a(x), x^N - 1) = 1 \in (\mathbb{Z}/q\mathbb{Z})[x].$$

If the above equation is true, then the inverse $a(x)^{-1} \in R_q$ can be computed using the extended Euclidean algorithm (Proposition 2.46 in [2]) to find polynomials $u(x), v(x) \in (\mathbb{Z}/q\mathbb{Z})[x]$ satisfying

$$a(x)u(x) + (x^N - 1)v(x) = 1.$$

Then $a(x)^{-1} = u(x)$ in R_q.

We will not go through the proof of this proposition, you can see Proposition 7.45 in [2] if interested.

Example 3.23. Let $N = 5, q = 2$, performing polynomial division:

$$\begin{aligned} x^5 + 1 &= x(x^4 + x + 1) + (x^2 + x + 1), \\ x^4 + x + 1 &= (x^2 + x)(x^2 + x + 1) + 1. \end{aligned}$$

Since the remainder is 1, $\gcd(x^4 + x + 1, x^5 + 1) = 1$. Substituting back gives:

$$\begin{aligned} 1 &= (x^4 + x + 1) + (x^2 + x)(x^2 + x + 1) \\ &= (x^4 + x + 1) + (x^2 + x)((x^5 + 1) + x(x^4 + x + 1)) \\ &= (x^4 + x + 1)(x^3 + x^2 + 1) + (x^5 + 1)(x^2 + x). \end{aligned}$$

Reducing modulo $(x^5 - 1)$, we obtain:

$$(1 + x + x^4)^{-1} = 1 + x^2 + x^3 \in R_2.$$

Remark 3.24. The ring R_q is well-defined even when q is composite:

- When $q = p^k$ is a prime power, finding $a(x)^{-1} \in R_q$ is typically done by first computing the inverse in R_p, and then iteratively lifting the result to R_{p^2}, R_{p^4}, and so forth.
- Generally, if $q = q_1 q_2 \cdots q_r$ where each q_i is coprime, one can compute the inverses in each R_{q_i} individually and combine them using the Chinese Remainder Theorem.

3.5.2 NTRUEncrypt

We will now introduce the NTRU public key cryptosystem, *NTRUEncrypt*, which is most naturally described by convolution polynomial rings, but the underlying hard mathematical problems can be interpreted as *SVP or CVP*.

Step 1: Public Parameter Creation Fix an integer $N \geq 1$, and two moduli p and q satisfying N is a prime and $\gcd(N, q) = \gcd(p, q) = 1$.

Let R, R_p and R_q be defined by

$$R = \frac{\mathbb{Z}[x]}{\langle x^N - 1\rangle}, R_p = \frac{(\mathbb{Z}/p\mathbb{Z})[x]}{\langle x^N - 1\rangle} \text{ and } R_q = \frac{(\mathbb{Z}/q\mathbb{Z})[x]}{\langle x^N - 1\rangle}.$$

We also need to introduce a new notation for polynomial $a(x)$:

Definition 3.25. For any positive integer d_1 and d_2, we let

$$\mathcal{T}(d_1, d_2) = \left\{ a(x) \in \mathbb{R} : \begin{array}{l} a(x) \text{ has } d_1 \text{ coefficients equal to } 1, \\ a(x) \text{ has } d_2 \text{ coefficients equal to } -1, \\ a(x) \text{ has all other coefficients equal to } 0 \end{array} \right\}.$$

Polynomials in $\mathcal{T}(d_1, d_2)$ are called *ternary* or *trinary* polynomials, since they only have 0 and ± 1 as coefficients. (Analogue to *binary* polynomials, which have only 0 and 1 as coefficients.)

Now, we are prepared with all parameters needed for NTRUEncrypt. Again, we assume Bob wants to send a message to Alice and avoid Eve knowing what it is, so Alice needs to generate a pair of keys and share the public key with Bob.

Step 2: Key Generation Alice chooses the pulic parameter (N, p, q, d) with satisfying conditions required above, then choose two polynomials randomly

$$f(x) \in \mathcal{T}(d+1, d) \quad \text{and} \quad g(x) \in \mathcal{T}(d, d).$$

She computes the inverses of $f(x)$:

$$F_q(x) = f(x)^{-1} \pmod{q} \in R_q \quad \text{and} \quad F_p(x) = f(x)^{-1} \pmod{p} \in R_p.$$

(If either inverse fails to exist, Alice will choose a new $f(x)$.)

Hence, the *public key* of Alice is the polynomial

$$h(x) = F_q(x) \star g(x) \in R_q,$$

and the *private key* is a pair of polynomials

$$(f(x), F_p(x)).$$

Step 3: Encryption

Suppose Bob has a message $m(x) \in R$ with coefficients $-\frac{p}{2} < m_i \leq \frac{p}{2}$, which means $m(x)$ is the centre-lift of a polynomial in R_p.

Then he chooses a random polynomial $r(x) \in \mathcal{T}(d, d)$ and computes

$$e(x) :\equiv ph(x) \star r(x) + m(x) \pmod{q}. \tag{5}$$

Therefore, Bob's ciphertext $e(x)$ is in R_q.

Step 4: Decryption

When Alice receives Bob's ciphertext $e(x) \in R_q$, she computes

$$a(x) :\equiv f(x) \star e(x) \pmod{q}.$$

Then she centre lifts $a(x)$ to $a'(x) \in R$ and computes

$$b(x) :\equiv F_p(x) \star a'(x) \pmod{p}.$$

Next, we want to show that $b(x) = m(x)$ with properly chosen parameters:

Proposition 3.26. If the NTRUEncrypt parameters (N, p, q, d) are chosen to satisfy

$$q > (6d + 1)p,$$

then the polynomial $b(x)$ computed by Alice is equal to the message $m(x)$ of Bob.

Proof. Firstly, we calculate more about $a(x)$:

$$\begin{aligned} a(x) &:\equiv f(x) \star e(x) \pmod{q} \\ &\equiv f(x) \star \big(ph(x) \star r(x) + m(x)\big) \pmod{q} \\ &\equiv pf(x) \star F_q(x) \star g(x) \star r(x) + f(x) \star m(x) \pmod{q} \\ &\equiv pg(x) \star r(x) + f(x) \star m(x) \pmod{q}. \end{aligned}$$

Evaluating $a'(x) = pg(x) \star r(x) + f(x) \star m(x)$ directly over R, the maximum coefficient magnitude is bounded by:

$$p(2d) + (2d + 1)\frac{p}{2} = (3d + \frac{1}{2})p.$$

If parameters satisfy $q > (6d + 1)p$, every coefficient of $a'(x)$ has absolute value strictly smaller than $\frac{q}{2}$.

Thus, center-lifting of $a(x) \pmod{q}$ recovers $a'(x)$ exactly in R, and then $b(x)$ is computed as

$$\begin{aligned} b(x) &:\equiv F_p(x) \star a'(x) \pmod{p} \\ &\equiv F_p(x) \star \big(pg(x) \star r(x) + f(x) \star m(x)\big) \pmod{p} \\ &\equiv F_p(x) \star f(x) \star m(x) \pmod{p} \\ &\equiv m(x) \pmod{q}. \end{aligned}$$

In conclusion, $b(x)$ and $m(x)$ are the same modulo p, i.e. we can compute the original message $m(x)$ by centre-lifting $b(x) \in R_p$ to R. □

We can summarize the above process of this algorithm by the table below:

Public parameter creation	
A trusted party chooses public parameters (N, p, q, d) with N and p prime, $\gcd(p, q) = \gcd(N, q) = 1$, and $q > (6d + 1)p$.	
Alice	**Bob**
Key creation	
Choose private $f \in \mathcal{T}(d + 1, d)$ that is invertible in R_q and R_p. Choose private $g \in \mathcal{T}(d, d)$. Compute F_q, the inverse of f in R_q. Compute F_p, the inverse of f in R_p. Publish the public key $h = F_q \star g$.	
Encryption	
	Choose plaintext $m \in R_p$. Choose a random $r \in \mathcal{T}(d, d)$. Use Alice's public key h to compute $e \equiv pr \star h + m \pmod{q}$. Send ciphertext e to Alice.
Decryption	
Compute $f \star e \equiv pg \star r + f \star m \pmod{q}$. Center-lift to $a = pg \star r + f \star m \in R$, and compute $m \equiv F_p \star a \pmod{p}$.	

Table 2: The NTRU public key cryptosystem

Remark 3.27.

- The condition $q > (6d + 1)p$ guarantees decryption never fails, but smaller values of q can be used in practice. Decryption failures are unlikely for smaller q because the positive and negative coefficients of $g(x)$ and $r(x)$ (and similarly $f(x)$ and $m(x)$) rarely align exactly. *Choosing smaller q increases efficiency and reduces public key size.*

  However, estimating decryption failure probability becomes a delicate problem. Decryption failure probability should be very small (e.g., $< 2^{-80}$), as failures can leak private key information to attackers.

- NTRUEncrypt is a probabilistic cryptosystem: a single plaintext $m(x)$ has many possible encryptions $p \cdot r(x) \star h(x) + m(x)$, depending on the random choice of $r(x)$.

Sending the same message twice with different random elements is inadvisable just as for GGH cryptosystem in Remark 3.16. Similarly, reusing the same random element for different plaintexts is dangerous.

- The polynomial $f(x) \in \mathcal{T}(d+1, d)$ has small coefficients, but its inverse $F_q(x) \in R_q$ tends to have randomly and uniformly distributed coefficients modulo q (experimentally observed).

  For example, if we choose $N = 11$, $q = 73$, and $f(x) = x^{10} + x^8 - x^3 + x^2 - 1 \in \mathcal{T}(3, 2)$, the inverse $F_q(x)$ has *random-looking* coefficients. (In cryptography, an output is *random-looking* if a computationally bounded adversary cannot distinguish it from pure uniform noise.) In practice, the public key $h(x) \equiv F_q(x) \star g(x) \pmod q$ and ciphertext $e(x) \equiv p \cdot r(x) \star h(x) + m(x) \pmod q$ also appear randomly distributed modulo q.

- Lattice-based cryptosystems are motivated by high speed compared to discrete logarithm and factorization-based systems. The most time-consuming part of encryption/decryption is the convolution product.

  General convolution $a \star b$ requires N^2 multiplications. NTRUEncrypt convolutions ($r \star h$, $f \star e$, $F_p \star a$) involve ternary polynomials, requiring only $\approx \frac{2}{3}N^2$ additions/subtractions (no multiplications). Encryption and decryption take $O(N^2)$ steps, with each step being extremely fast.

Below we present an small numerical example with NTRUEncrypt. All numerical calculations in this example are verified using Python, and the implementation is provided in Appendix A.3.

Example 3.28. Let $(N, p, q, d) = (7, 3, 41, 2)$. Since $q = 41 > 39 = (6d+1)p$, correct decryption is guaranteed.
Alice chooses private keys:

$$f(x) = x^6 - x^4 + x^3 + x^2 - 1 \in \mathcal{T}(3, 2), \quad g(x) = x^6 + x^4 - x^2 - x \in \mathcal{T}(2, 2).$$

She computes inverses and publishes public key $h(x) \in R_q$:

$$F_q(x) = f(x)^{-1} \bmod q = 8x^6 + 26x^5 + 31x^4 + 21x^3 + 40x^2 + 2x + 37$$

$$F_p(x) = f(x)^{-1} \bmod p = x^6 + 2x^5 + x^3 + x^2 + x + 1$$

$$h(x) = F_q(x) \star g(x) \bmod q = 20x^6 + 40x^5 + 2x^4 + 38x^3 + 8x^2 + 26x + 30$$

Bob encrypts $m(x) = -x^5 + x^3 + x^2 - x + 1$ using $r(x) = x^6 - x^5 + x - 1$:

$$e(x) \equiv p \cdot r(x) \star h(x) + m(x) \equiv 31x^6 + 19x^5 + 4x^4 + 2x^3 + 40x^2 + 3x + 25 \pmod q$$

Alice decrypts:

1. Compute $f(x) \star e(x) \pmod q$ and center-lift to R:

$$a'(x) = (x^6+10x^5+33x^4+40x^3+40x^2+x+40) = x^6+10x^5-8x^4-x^3-x^2+x-1.$$

2. Compute $F_p(x) \star a(x) \pmod p$ and center-lift to recover $m(x)$:

$$m(x) = \text{lift}_p(2x^5 + x^3 + x^2 + 2x + 1) = -x^5 + x^3 + x^2 - x + 1.$$

3.5.3 Mathematical Problems for NTRUEncrypt

Although the coefficients of the public key $h(x)$ appear to be random integers modulo q, there exists a hidden relationship involving polynomials $f(x)$ and $g(x)$ with small coefficients:

$$f(x) \star h(x) \equiv g(x) \pmod q.$$

Thus, finding the private key from the public key reduces to solving the core recovery problem:

The NTRU Key Recovery Problem.
Given $h(x)$, find ternary polynomials $f(x)$ and $g(x)$ satisfying:

$$f(x) \star h(x) \equiv g(x) \pmod q$$

The solution $(f(x), g(x))$ to this problem is not unique, since if $(f(x), g(x))$ is a solution, then the pair $(x^k \star f(x), x^k \star g(x))$ is also a solution for any $0 \leq k < N$.

The polynomial $x^k \star f(x)$ is called a *rotation* of $f(x)$ as its coefficients have been rotated by k positions cyclically. Moreover, decryption using a rotated key $x^k \star f(x)$ yields a rotated plaintext $x^k \star m(x)$.

Generally, any polynomial pair $(f(x), g(x))$ with sufficiently small coefficients, satisfying the key equation for The NTRU Key Recovery Problem, serves as a NTRU decryption key.

We expect the NTRU Key Recovery Problem to be a hard mathematical problem. Otherwise, Eve as an attacker may be able to recover the private key from the public key, leading to failure of the NTRU cryptosystem.

In the next section, we will prove that solving this problem is almost equivalent to solving SVP in a certain class of lattices.

3.5.4 NTRUEncrypt as a Lattice-Based Cryptosystem

Let $h(x) = h_0 + h_1 x + \cdots + h_{N-1}x^{N-1}$ be an NTRUEncrypt public key. Let $h \in \mathbb{R}^{N \times N}$ be a matrix whose rows consist of cyclic permutations of the coefficients of $h(x)$:

$$h = \begin{pmatrix} h_0 & h_1 & \dots & h_{N-1} \\ h_{N-1} & h_0 & \dots & h_{N-2} \\ \vdots & \vdots & \ddots & \vdots \\ h_1 & h_2 & \dots & h_0 \end{pmatrix}.$$

Then let M_h^{NTRU} be a $2N \times 2N$ block matrix composed of four $N \times N$ blocks:

$$M_h^{\text{NTRU}} = \begin{pmatrix} I_N & h \\ 0 & qI_N \end{pmatrix}.$$

We define the lattice spanned by the rows of M_h^{NTRU} to be the *NTRU Lattice* L_h^{NTRU} associated with $h(x)$.

Moreover, we identify any pair of polynomials $a(x) = \sum_{i=0}^{N-1} a_i x^i$ and $b(x) = \sum_{i=0}^{N-1} b_i x^i$ in $R = \mathbb{Z}[x]/(x^N - 1)$ with a vector in $\mathbb{Z}^{2N}$:

$$(a, b) = (a_0, a_1, \ldots, a_{N-1}, b_0, b_1, \ldots, b_{N-1}) \in \mathbb{Z}^{2N}.$$

Proposition 3.29 (Embedding the Private Key into the Lattice)**.** Let $f(x)$ and $g(x)$ be the secret key polynomials and $h(x)$ be the public key generated by them. Since $f(x) \star h(x) \equiv g(x) \pmod{q}$, there exists a polynomial $u(x)$ such that $f(x) \star h(x) = g(x) + qu(x)$. Then:

$$(f, -u)M_h^{\text{NTRU}} = (f, g),$$

which proves that the target key vector (f, g) lies directly inside the NTRU lattice L_h^{NTRU}.

Proof. Multiplying $(f, -u)$ by the left block column $\begin{pmatrix} I_N \\ 0 \end{pmatrix}$ gives f. Multiplying $(f, -u)$ by the right block column computes $f(x) \star h(x) - qu(x) = g(x)$.

Since $(f, -u) \in \mathbb{Z}^{2N}$, the vector (f, g) is an integer linear combination of the lattice basis rows, so $(f, g) \in L_h^{\text{NTRU}}$. □

Proposition 3.30 (Geometric Properties of the NTRU Lattice)**.** For standard parameter choices ($p = 3$, $d \approx N/3$, $q \approx 2pN$):

1. **Determinant:** $\det(L_h^{\text{NTRU}}) = q^N$.
2. **Target Length:** $\|(f, g)\| \approx \sqrt{4d} \approx 1.155\sqrt{N}$.
3. **Gaussian Heuristic Prediction:** The expected length of the shortest vector in an average $2N$-dimensional lattice of this determinant is:

$$\sigma(L_h^{\text{NTRU}}) \approx \sqrt{\frac{Nq}{\pi e}} \approx 0.838N.$$

Proof.

1. M_h^{NTRU} is block upper-triangular, so its determinant is the product of the diagonal blocks: $\det(M_h^{\text{NTRU}}) = \det(I_N) \cdot \det(qI_N) = q^N$.
2. f and g each contain roughly d entries of 1 and d entries of -1. Thus, $\|(f, g)\|^2 \approx d(1)^2 + d(-1)^2 + d(1)^2 + d(-1)^2 = 4d$.
3. Substituting dimension $n = 2N$ and determinant $\det(L) = q^N$ into the n-dimensional Gaussian heuristic formula $\sigma(L) \approx \sqrt{\frac{n}{2\pi e}}(\det L)^{1/n}$ yields $\sigma(L_h^{\text{NTRU}}) \approx \sqrt{\frac{Nq}{\pi e}}$.

□

These properties reveal why NTRU key recovery is equivalent to lattice reduction:

- **Anomalously Short Vector:** The key vector (f, g) has length $\mathcal{O}(\sqrt{N})$, whereas an average vector in the lattice has length $\mathcal{O}(N)$. The target vector (f, g) is shorter than expected by a factor of $\mathcal{O}(1/\sqrt{N})$.
- **Reduction to apprSVP:** Finding (f, g) corresponds to solving the Shortest Vector Problem (SVP) in L_h^{NTRU}. In fact, finding any vector within an approximation factor $\gamma \approx N^\epsilon$ (where $\epsilon < 1/2$) yields a working decryption key.
- **Algorithmic Hardness:** Standard LLL achieves an approximation factor of 2^N in polynomial time, which is too weak to find (f, g) for large N. Block-reduction algorithms such as BKZ-LLL improve the approximation factor to $\beta^{2N/\beta}$ using blocksize β, but their running time grows exponentially in β.

Thus, for sufficiently large dimension N, solving apprSVP to locate (f, g) becomes computationally infeasible, guaranteeing the lattice-based security of NTRUEncrypt.

To analyze how an attacker might attempt to find such short vectors in practice, we now introduce the fundamental lattice reduction tools.

3.6 Lattice Reduction Algorithms

In higher dimensions, solving the Shortest Vector Problem (SVP) or Closest Vector Problem (CVP) exactly becomes computationally intractable. Lattice reduction algorithms seek to find an *optimal basis*—a basis whose vectors are as short and as nearly orthogonal as possible.

3.6.1 Gaussian Lattice Reduction in Dimension 2

In dimension $n = 2$, an algorithm due to Gauss provides an exact and efficient solution to SVP. The core idea mimics the Euclidean algorithm: we iteratively subtract integer multiples of the shorter basis vector from the longer one to reduce its length.

Given a basis $\{v_1, v_2\}$ for a two-dimensional lattice $L \subset \mathbb{R}^2$, we assume $\|v_1\| \leq \|v_2\|$. (Otherwise, swap v_1 and v_2.)

To make v_2 more orthogonal to v_1, we first consider its orthogonal component:

$$v_2^* = v_2 - \frac{v_1 \cdot v_2}{\|v_1\|^2} v_1.$$

Although v_2^* is orthogonal to v_1, it is generally not a lattice vector because $\frac{v_1 \cdot v_2}{\|v_1\|^2}$ is not necessarily an integer. Therefore, instead of subtracting this real multiple of v_1, we subtract the nearest integer multiple. Let

$$m = \left\lfloor \frac{v_1 \cdot v_2}{\|v_1\|^2} \right\rceil,$$

and define

$$v_2' = v_2 - m v_1.$$

Since $m \in \mathbb{Z}$, v_2' remains a lattice vector. Choosing the nearest integer m makes v_2' as close as possible to the orthogonal vector v_2^*.

Gaussian Lattice Reduction Algorithm

[1] Input a basis $\{v_1, v_2\}$ for a 2-dimensional lattice L

[2] **Loop:**

[3] **If** $\|v_2\| < \|v_1\|$:

[4] Swap v_1 and v_2

[5] Compute $m = \left\lfloor \frac{v_1 \cdot v_2}{\|v_1\|^2} \right\rceil$

[6] **If** $m = 0$:

[7] **Return** reduced basis $\{v_1, v_2\}$

[8] Set $v_2 = v_2 - mv_1$ *[Size Reduction]*

[9] Continue Loop

Note: Upon termination, v_1 is a shortest non-zero vector in L, and the angle θ between v_1 and v_2 satisfies $\frac{\pi}{3} \leq \theta \leq \frac{2\pi}{3}$.

Proposition 3.31 (Correctness of Gaussian Reduction)**.** With conditions and algorithm given above, the vector v_1 is a shortest non-zero vector in L (solving SVP exactly). Furthermore, the angle θ between v_1 and v_2 satisfies $|\cos\theta| \leq \frac{\|v_1\|}{2\|v_2\|}$, implying $\frac{\pi}{3} \leq \theta \leq \frac{2\pi}{3}$.

Proof. When the algorithm terminates with $\|v_2\| \geq \|v_1\|$, $m = 0$ implies:

$$\frac{|v_1 \cdot v_2|}{\|v_1\|^2} \leq \frac{1}{2}.$$

For any arbitrary non-zero lattice vector $v = a_1v_1 + a_2v_2 \in L$ with $a_1, a_2 \in \mathbb{Z}$:

$$\begin{aligned} \|v\|^2 &= a_1^2\|v_1\|^2 + 2a_1a_2(v_1 \cdot v_2) + a_2^2\|v_2\|^2 \\ &\geq a_1^2\|v_1\|^2 - |a_1a_2|\|v_1\|^2 + a_2^2\|v_1\|^2 \\ &= (a_1^2 - |a_1a_2| + a_2^2)\|v_1\|^2. \end{aligned}$$

Since $a_1, a_2 \in \mathbb{Z}$ are not both zero, the quadratic form $a_1^2 - |a_1a_2| + a_2^2 = (a_1 - \frac{1}{2}a_2)^2 + \frac{3}{4}a_2^2 \geq 1$. Thus $\|v\|^2 \geq \|v_1\|^2$, proving v_1 is a shortest vector. □

Example 3.32 (Gaussian Lattice Reduction)**.** Consider the lattice $L \subset \mathbb{R}^2$ spanned by $v_1 = (66586820, 65354729)$ and $v_2 = (6513996, 6393464)$. Initializing with $\|v_1\| > \|v_2\|$, we swap them so $\|v_1\| \leq \|v_2\|$ and iteratively compute $m = \lfloor \frac{v_1 \cdot v_2}{\|v_1\|^2} \rceil$, replacing $v_2 \leftarrow v_2 - mv_1$ and swapping when $\|v_2\| < \|v_1\|$:

Step	v_1	v_2	m
1	$(6513996, 6393464)$	$(66586820, 65354729)$	10
2	$(1446860, 1420089)$	$(6513996, 6393464)$	5
3	$(-720304, -706981)$	$(1446860, 1420089)$	-2
4	$(6252, 6127)$	$(-720304, -706981)$	-115
5	$(-1324, -2376)$	$(6252, 6127)$	-3
6	$(2280, -1001)$	$(-1324, -2376)$	0

Since $m = 0$ at Step 6, the algorithm terminates. The reduced basis is $\{(2280, -1001), (-1324, -2376)\}$, and $v_1 = (2280, -1001)$ solves the Shortest Vector Problem (SVP) for L. We have verified this step-by-step reduction using a Python script in Appendix A.4.

3.6.2 The LLL Lattice Reduction Algorithm

Motivation: For $n > 2$, finding exact shortest vectors is hard. The Lenstra–Lenstra–Lovász (LLL) algorithm (1982) trades exactness for efficiency: it finds a *quasi-orthogonal* (i.e. nearly orthogonal) basis containing a reasonably short vector, in polynomial time.

The Gram–Schmidt basis:

Let $\mathcal{B} = \{v_1, \ldots, v_n\}$ be a basis for a lattice $L \subset \mathbb{R}^n$. Let $\mathcal{B}^* = \{v_1^*, \ldots, v_n^*\}$ be the associated Gram-Schmidt orthogonal basis given by:

$$v_i^* = v_i - \sum_{j=1}^{i-1} \mu_{i,j} v_j^* \quad \text{where} \quad \mu_{i,j} = \frac{v_i \cdot v_j^*}{\|v_j^*\|^2}.$$

Note that $\mathcal{B}^*$ spans $\mathbb{R}^n$, but is generally not a basis for L because $\mu_{i,j} \notin \mathbb{Z}$. However, $\det(L) = \prod_{i=1}^n \|v_i^*\|$ as I proved in *Step 2* of the proof of Proposition 2.14.

What counts as a "reduced" basis?

Definition 3.33 (LLL-Reduced Basis)**.** A basis $\mathcal{B} = \{v_1, \ldots, v_n\}$ for a lattice L is said to be *LLL-reduced* if its associated Gram-Schmidt orthogonal basis $\mathcal{B}^* = \{v_1^*, \ldots, v_n^*\}$ and coefficients $\mu_{i,j}$ satisfy:

1. **Size Condition:**
$$|\mu_{i,j}| \leq \frac{1}{2} \quad \text{for all } 1 \leq j < i \leq n.$$

2. **Lovász Condition:**
$$\|v_i^*\|^2 \geq (\frac{3}{4} - \mu_{i,i-1}^2)\|v_{i-1}^*\|^2 \quad \text{for all } 1 < i \leq n.$$

Remark 3.34.

- The *Size Condition* says each v_i is already as close as an integer shift allows to its own Gram–Schmidt projection v_i^*, so we are changing the basis for the lattice to be as "good" (or orthogonal) as possible.

- The *Lovász Condition* says consecutive Gram–Schmidt lengths $\|v_{i-1}^*\|, \|v_i^*\|$ never drop too sharply.

How to construct an LLL-reduced basis?

To see why the algorithm below works, recall the goal: a basis with short, nearly orthogonal vectors, built only from *integer* combinations of the vectors we already have.

The algorithm processes basis vectors one at a time using a pointer k, starting at $k = 2$ and moving between 2 and n. At every stage, everything to the left of the pointer, $\{v_1, \ldots, v_{k-1}\}$, is already LLL-reduced; the algorithm's job is to bring v_k into line too, in two steps:

Step 1: Size reduction. Subtract integer multiples of $v_{k-1}, \ldots, v_1$ from v_k, one at a time, so each $\mu_{k,j}$ satisfies $|\mu_{k,j}| \leq \frac{1}{2}$. This does not change v_k^* — it only removes the redundant components of v_k along earlier basis directions.

- *Why does this bound always hold?* Processing index j replaces $\mu_{k,j}$ by $\mu_{k,j}^{\text{old}} - \lfloor \mu_{k,j}^{\text{old}} \rceil$. A real number minus its nearest integer always lies in $[-\frac{1}{2}, \frac{1}{2}]$, so the bound holds as soon as $\mu_{k,j}$ is updated.

- *Why must the loop run from $j = k-1$* ***down to*** *1?* Once $\mu_{k,j}$ is fixed into $[-\frac{1}{2}, \frac{1}{2}]$, it must stay that way. Moving on to the smaller index $j-1$ only changes $\mu_{k,m}$ for $m \leq j-1$, via $\mu_{k,m}^{\text{new}} = \mu_{k,m}^{\text{old}} - t'\mu_{j-1,m}$, so $\mu_{k,j}$ itself is never touched again. This is why we work from the largest index down to the smallest: each coefficient, once fixed, is left alone for the rest of the loop. Concretely, for each j in turn we set $t = \lfloor \mu_{k,j} \rceil$, update $v_k \leftarrow v_k - t\, v_j$, and then immediately update $\mu_{k,m}^{\text{new}} = \mu_{k,m}^{\text{old}} - t\, \mu_{j,m}$ for $m < j$ and $\mu_{k,j}^{\text{new}} = \mu_{k,j}^{\text{old}} - t$ — rather than subtracting for every j first and recomputing everything at the end. None of this changes any v_i^*: since v_j (for $j < k$) lies entirely in the span of $v_1, \ldots, v_{k-1}$, it has no component along the direction that defines v_k^*. So the Gram–Schmidt vectors only need to be recomputed once at the start (line 3) and again after each swap (line 12) — never during size reduction.

Step 2: Lovász check. Ask whether v_k^* is reasonably long compared to v_{k-1}^*.

- **Holds:** the vectors are in a sensible (roughly increasing) length order, so $\{v_1, \ldots, v_k\}$ is now reduced too — advance the pointer, $k \leftarrow k+1$.

- **Fails:** v_k^* is too short relative to v_{k-1}^*, so we swap v_{k-1} and v_k. This changes v_{k-1}^* and can undo reduction achieved at earlier positions, so we step back, $k \leftarrow \max(k-1, 2)$, and re-check.

- *Why does swapping fix the failure?* Write $u = v_{k-1}$, $w = v_k$, and decompose w using the old Gram–Schmidt data:

$$w = w^* + \mu_{k,k-1} u^* + (\text{components along } v_1^*, \ldots, v_{k-2}^*).$$

After swapping, the new v_{k-1}^* is the part of w orthogonal to $v_1, \ldots, v_{k-2}$, which the decomposition above shows is $v_{k-1}^{*,\text{new}} = \mu_{k,k-1} u^* + w^*$. Since $u^* = v_{k-1}^*$ and $w^* = v_k^*$

are orthogonal to each other,

$$\|v_{k-1}^{*,\text{new}}\|^2 = \mu_{k,k-1}^2 \|v_{k-1}^*\|^2 + \|v_k^*\|^2.$$

Failure of the Lovász condition says exactly that the right-hand side is $< (\frac{3}{4})\|v_{k-1}^*\|^2$, so

$$\|v_{k-1}^{*,\text{new}}\|^2 < \frac{3}{4}\|v_{k-1}^*\|^2.$$

The swap therefore *strictly shrinks* $\|v_{k-1}^*\|$ by a factor below $\frac{3}{4}$ — this is exactly the progress that makes the potential D (defined precisely below) decrease.

- *Does the swap already guarantee the new position is reduced?* No. The relationship between the new v_{k-1}^* and v_{k-2}^* has not been checked — it was the *old* v_{k-1}^* that satisfied the earlier conditions, not the new one. So we cannot simply move on: the whole loop body — size reduction, then the Lovász test — is re-run from scratch at $k \leftarrow \max(k-1, 2)$, exactly as it would be at any other value of k. The $\max(\cdot, 2)$ only reflects that index 1 has no predecessor, so $k = 2$ is the smallest index at which the check makes sense. If the recheck fails again, another swap and another retreat follow — but each swap still shrinks some $\|v_i^*\|^2$ by a factor below $\frac{3}{4}$, so this cannot go on forever.

The LLL Algorithm

[1] Input a basis $\{v_1, \dots, v_n\}$ for a lattice L

[2] Set $k = 2$

[3] Compute the Gram–Schmidt basis $\{v_1^*, \dots, v_n^*\}$ and initial coefficients $\mu_{i,j}$

[4] **While** $k \le n$:

[5] **For** $j = k-1$ down to 1:

[6] Set $v_k = v_k - \lfloor \mu_{k,j} \rceil v_j$ *[Size Reduction]*

[7] Update $\mu_{k,m}$ for all $1 \le m \le j$

[8] **If** $\|v_k^*\|^2 \ge (\frac{3}{4} - \mu_{k,k-1}^2)\|v_{k-1}^*\|^2$ *[Lovász Condition]*

[9] Set $k = k + 1$

[10] **Else**:

[11] Swap v_{k-1} and v_k *[Swap Step]*

[12] Update affected v_i^* and $\mu_{i,j}$ for indices $k-1$ and k

[13] Set $k = \max(k-1, 2)$

[14] **Return** LLL-reduced basis $\{v_1, \dots, v_n\}$

Note: At each step, $\{v_1^*, \dots, v_n^*\}$ is the Gram–Schmidt orthogonal set associated with the current basis $\{v_1, \dots, v_n\}$, and $\mu_{i,j} = \frac{v_i \cdot v_j^*}{\|v_j^*\|^2}$.

The pointer moves back and forth, but it always makes progress: each swap strictly decreases a potential D, defined precisely in the remark below, so the algorithm cannot go on forever. Once k reaches $n+1$, every vector has passed the Lovász check, and the whole basis is LLL-reduced.

Example 3.35. To see the whole loop in action, take the basis $v_1 = (4, 1)$, $v_2 = (2, 3)$ for a lattice in $\mathbb{Z}^2$. Initially,

$$v_1^* = (4, 1), \qquad \|v_1^*\|^2 = 17, \qquad \mu_{2,1} = \frac{v_2 \cdot v_1^*}{\|v_1^*\|^2} = \frac{2 \cdot 4 + 3 \cdot 1}{17} = \frac{11}{17} \approx 0.647,$$

$$v_2^* = v_2 - \mu_{2,1} v_1^* \approx (-0.588,\, 2.353), \qquad \|v_2^*\|^2 \approx 5.884,$$

and k is set to 2.
On the first visit to $k = 2$, size reduction has only $j = 1$: $\lfloor \mu_{2,1} \rceil = \lfloor 0.647 \rceil = 1$, so

$$v_2 \leftarrow v_2 - v_1 = (2, 3) - (4, 1) = (-2, 2), \qquad \mu_{2,1}^{\text{new}} = 0.647 - 1 = -0.353,$$

which indeed satisfies $|-0.353| \le \frac{1}{2}$. Since v_1 lies along the direction defining v_1^*

only, v_2^* is unchanged. The Lovász check then gives

$$(\frac{3}{4} - \mu_{2,1}^2)\|v_1^*\|^2 = (0.75 - 0.125) \cdot 17 \approx 10.63, \qquad \|v_2^*\|^2 \approx 5.884 \not\geq 10.63,$$

so the condition fails and v_1, v_2 are swapped, giving $v_1 = (-2, 2)$, $v_2 = (4, 1)$. Recomputing Gram–Schmidt from scratch,

$$v_1^* = (-2, 2), \qquad \|v_1^*\|^2 = 8, \qquad \mu_{2,1} = \frac{4 \cdot (-2) + 1 \cdot 2}{8} = -0.75,$$

$$v_2^* = (4, 1) - (-0.75)(-2, 2) = (2.5,\ 2.5), \qquad \|v_2^*\|^2 = 12.5,$$

and k is set back to $\max(2 - 1, 2) = 2$.
On the second visit to $k = 2$, size reduction gives $\lfloor -0.75 \rceil = -1$, so

$$v_2 \leftarrow v_2 - (-1)v_1 = (4, 1) + (-2, 2) = (2, 3), \qquad \mu_{2,1}^{\text{new}} = -0.75 - (-1) = 0.25,$$

again within $[-\frac{1}{2}, \frac{1}{2}]$, with v_2^* unchanged at $(2.5, 2.5)$, $\|v_2^*\|^2 = 12.5$. The Lovász check now gives

$$(\frac{3}{4} - 0.25^2) \cdot 8 = 5.5, \qquad \|v_2^*\|^2 = 12.5 \geq 5.5,$$

so the condition holds, k advances to $3 = n + 1$, and the loop terminates with the reduced basis

$$v_1 = (-2, 2), \qquad v_2 = (2, 3).$$

As a check, det of the original basis is $|4 \cdot 3 - 1 \cdot 2| = 10$, and det of the reduced basis is $|(-2)(3) - (2)(2)| = 10$, confirming the same lattice, while $\|v_1\|^2$ has dropped from 17 to 8 with $\|v_2\|^2 = 13$ unchanged.

Remark 3.36 (Termination and Computational Complexity)**.**

- Define the potential $D = \prod_{i=1}^{n-1} \det(L_i)^2$, where $L_i = \text{span}_{\mathbb{Z}}\{v_1, \ldots, v_i\}$ is the sublattice generated by the first i basis vectors. Recall $\det(L_i)^2 = \prod_{j=1}^{i} \|v_j^*\|^2$.

  A size-reduction step changes no v_j^* and no L_i (it only replaces v_k by an integer shift of itself, which does not change the sublattice spanned by any prefix of the basis), so D is unchanged by size reduction.

- A swap at position k changes only the sublattice L_{k-1}: for $i < k - 1$ the first i vectors are untouched, and for $i \geq k$ the set $\{v_1, \ldots, v_i\}$ is the same set of vectors as before, just reordered, so L_i is the same sublattice.

  Only L_{k-1} genuinely changes, since it now spans $\{v_1, \ldots, v_{k-2}, v_k\}$ instead of $\{v_1, \ldots, v_{k-2}, v_{k-1}\}$. We showed above that a swap makes $\det(L_{k-1})^2 = \|v_{k-1}^{*,\text{new}}\|^2 \cdot \prod_{j<k-1} \|v_j^*\|^2$ shrink by a factor strictly less than $\frac{3}{4}$, so D itself shrinks by that same factor with every swap.

- Although the Gram–Schmidt vectors v_j^* need not have integer (or even rational) coordinates, $\det(L_i)^2$ is still a positive integer for every i. This is because $\det(L_i)^2$ has two equal expressions:

$$\det(L_i)^2 = \prod_{j=1}^{i} \|v_j^*\|^2 = \det(G_i), \qquad \text{where } (G_i)_{a,b} = v_a \cdot v_b \quad (1 \le a, b \le i)$$

  is the *Gram matrix* of the *original* basis vectors $v_1, \ldots, v_i$ (not the v_j^*). Since $v_1, \ldots, v_i \in \mathbb{Z}^n$, every entry $v_a \cdot v_b$ of G_i is an integer, so $\det(G_i)$ is an integer too.

  Since $v_1, \ldots, v_i$ are linearly independent, $\det(G_i) \neq 0$, so in fact $\det(L_i)^2 \ge 1$, and hence $D \ge 1$ always.

- This is enough to bound the number of swaps. Let D_0 be the value of D at the start, and D_m its value after m swaps have occurred (size reduction never changes D, so swaps are the only thing that moves it). Each swap multiplies D by a factor strictly below $\frac{3}{4}$, so after m swaps,

$$D_m < \left(\frac{3}{4}\right)^m D_0.$$

  But D_m is always a positive integer, so $D_m \ge 1$. Combining the two:

$$1 \le D_m < \left(\frac{3}{4}\right)^m D_0 \quad \Longrightarrow \quad m < \frac{\log D_0}{\log(4/3)}.$$

  Since D_0 is a single fixed, finite number determined by the starting basis, this is a fixed finite bound on m — so only finitely many swaps can ever happen, which is exactly what guarantees the algorithm terminates.

  Bounding $\log D_0$ in terms of the size of the input basis makes this bound explicit: $m = \mathcal{O}(n^2 \log B)$ swaps, where $B = \max_i \|v_i\|$, giving polynomial-time termination. For the full derivation of this bound, see [2], Section 7.13.2.

How good is the basis we get?

So far we know the algorithm terminates and produces a basis satisfying the Size and Lovász conditions. We now show that any such basis is automatically a good approximate solution to SVP.

Theorem 3.37 (Properties of an LLL-Reduced Basis)**.** If $\{v_1, \ldots, v_n\}$ is an LLL-reduced basis for a lattice L, then:

1. $\|v_1\| \le 2^{(n-1)/4} (\det L)^{1/n}$.
2. $\|v_1\| \le 2^{(n-1)/2} \min_{0 \neq v \in L} \|v\|$.

Thus, LLL solves apprSVP to within an exponential factor of $\gamma = 2^{(n-1)/2}$ in polynomial time.

Proof. Combining the Lovász condition with the size condition $|\mu_{i,i-1}| \leq \frac{1}{2}$ yields:

$$\|v_i^*\|^2 \geq (\frac{3}{4} - \frac{1}{4})\|v_{i-1}^*\|^2 = \frac{1}{2}\|v_{i-1}^*\|^2 \quad \text{for all } 1 < i \leq n.$$

By induction, this lower bound generalizes to:

$$\|v_j^*\|^2 \leq 2^{i-j}\|v_i^*\|^2 \quad \text{for all } 1 \leq j \leq i \leq n.$$

In particular, setting $j = 1$ gives:

$$\|v_1^*\|^2 \leq 2^{i-1}\|v_i^*\|^2 \implies \|v_i^*\|^2 \geq 2^{-(i-1)}\|v_1\|^2 \quad \text{for all } 1 \leq i \leq n$$

(since $v_1^* = v_1$).

We now complete the proofs of both statements of the theorem:

Proof of (1):

Recall that the determinant of the lattice L is given by the product of the norms of its Gram–Schmidt orthogonal basis vectors:

$$\det L = \prod_{i=1}^{n} \|v_i^*\|.$$

Taking the square of the determinant and applying the bound $\|v_i^*\|^2 \geq 2^{-(i-1)}\|v_1\|^2$ for each term yields:

$$(\det L)^2 = \prod_{i=1}^{n} \|v_i^*\|^2 \geq \prod_{i=1}^{n} (2^{-(i-1)}\|v_1\|^2) = \|v_1\|^{2n} \cdot 2^{-\sum_{i=1}^{n}(i-1)}.$$

Evaluating the arithmetic sum in the exponent:

$$\sum_{i=1}^{n}(i-1) = 0 + 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

Substituting this back gives:

$$(\det L)^2 \geq \|v_1\|^{2n} \cdot 2^{-n(n-1)/2}.$$

Taking the $2n$-th root on both sides:

$$\det L^{1/n} \geq \|v_1\| \cdot 2^{-(n-1)/4} \implies \|v_1\| \leq 2^{(n-1)/4}(\det L)^{1/n}.$$

Proof of (2):

Let $v \in L$ be any non-zero lattice vector. Expressing v in terms of the basis $\{v_1, \ldots, v_n\}$ with integer coefficients gives:

$$v = \sum_{j=1}^{n} a_j v_j, \quad a_j \in \mathbb{Z}.$$

Let k be the largest index such that $a_k \neq 0$ $(1 \leq k \leq n)$. By expanding each v_j into

its Gram–Schmidt decomposition $v_j = v_j^* + \sum_{m=1}^{j-1} \mu_{j,m} v_m^*$, we can write v in terms of the orthogonal basis $\{v_1^*, \ldots, v_n^*\}$:

$$v = \sum_{j=1}^{k} b_j v_j^* \quad \text{where } b_k = a_k.$$

Since $a_k \in \mathbb{Z}$ and $a_k \neq 0$, we have $|b_k| = |a_k| \geq 1$. By orthogonality of the vectors v_j^*, applying the Pythagorean theorem gives:

$$\|v\|^2 = \sum_{j=1}^{k} b_j^2 \|v_j^*\|^2 \geq b_k^2 \|v_k^*\|^2 \geq \|v_k^*\|^2.$$

Using our earlier lower bound $\|v_k^*\|^2 \geq 2^{-(k-1)} \|v_1\|^2$, we obtain:

$$\|v\|^2 \geq 2^{-(k-1)} \|v_1\|^2 \geq 2^{-(n-1)} \|v_1\|^2 \quad (\text{since } k \leq n).$$

Taking square roots yields:

$$\|v\| \geq 2^{-(n-1)/2} \|v_1\| \implies \|v_1\| \leq 2^{(n-1)/2} \|v\|.$$

Since this inequality holds for every non-zero vector $v \in L$, taking the minimum over all non-zero $v \in L$ gives:

$$\|v_1\| \leq 2^{(n-1)/2} \min_{0 \neq v \in L} \|v\|.$$

□

Example 3.38 (LLL reduction)**.** Consider the 6-dimensional lattice L with basis given by the rows of

$$M = \begin{pmatrix} 19 & 2 & 32 & 46 & 3 & 33 \\ 15 & 42 & 11 & 0 & 3 & 24 \\ 43 & 15 & 0 & 24 & 4 & 16 \\ 20 & 44 & 44 & 0 & 18 & 15 \\ 0 & 48 & 35 & 16 & 31 & 31 \\ 48 & 33 & 32 & 9 & 1 & 29 \end{pmatrix}.$$

Applying the LLL algorithm with Lov'asz parameter $\delta = \frac{3}{4}$ gives the reduced basis

$$M_{\text{LLL}} = \begin{pmatrix} 7 & -12 & -8 & 4 & 19 & 9 \\ -20 & 4 & -9 & 16 & 13 & 16 \\ 5 & 2 & 33 & 0 & 15 & -9 \\ -6 & -7 & -20 & -21 & 8 & -12 \\ -10 & -24 & 21 & -15 & -6 & -11 \\ 74 & -9 & -11 & 1 & 31 & 32 \end{pmatrix}.$$

The LLL algorithm was implemented in Python, and the code used to obtain this reduced basis is given in Appendix A.5.

Since LLL uses unimodular row operations,

$$|\det(M)| = |\det(M_{\text{LLL}})| = 777406251.$$

The preservation of the determinant was also verified computationally, see Appendix A.5.
The Hadamard ratio improves from $H(M) \approx 0.46908$ to $H(M_{\text{LLL}}) \approx 0.88824$, indicating that the reduced basis vectors are more nearly orthogonal.
Moreover, the shortest basis vector decreases from approximately 51.913 to 26.739.
These quantities were calculated using the Python implementation in Appendix A.5.
Thus, LLL produces a significantly better basis for the same lattice.
This example also illustrates that the output of the LLL algorithm depends on both the ordering of the input basis and the choice of the Lovász parameter δ.
These effects are investigated computationally in Appendix A.5, where different basis orderings and values of δ are compared in terms of the number of swaps, the Hadamard ratio, and the length of the shortest basis vector.
In particular, the computational experiments examine how changing δ affects the quality of the reduced basis and the computational effort required by the algorithm.

How LLL solves apprCVP?

In previous *Section 3.2 Babai's Algorithm*, we know that it is easy to solve SVP and CVP with an orthogonal basis for lattice L. We may not have an orthogonal basis directly, but we can get a *quasi-orthogonal* (reasonably orthogonal) basis by using LLL algorithm, and thus solve *apprCVP*.

Theorem 3.39 (LLL apprCVP Algorithm)**.** There is a constant C such that for any lattice L of dimension n given by a basis $v_1, \ldots, v_n$, the following algorithm solves apprCVP to within a factor of C^n:

1. Apply LLL to $v_1, \ldots, v_n$ to find an LLL-reduced basis.
2. Apply Babai's algorithm using this LLL-reduced basis.

Proof. Omitted. See [2], Exercise 7.52. □

Remark 3.40. There are two natural ways to combine LLL with Babai's algorithm, depending on which version of Babai's algorithm is used:

- Using the *closest vertex* method.
- Using the *closest plane* method.

In practice, pairing the closest plane method with an LLL-reduced basis tends to give better approximations than pairing it with the closest vertex method. See [2], Exercise 7.53, for further details.

3.7 Application of LLL to Cryptanalysis

The LLL algorithm has wide applications in cryptanalysis, such as attacks on lattice-based cryptosystems including Ajtai–Dwork, GGH, and NTRU.

Note that LLL has no trouble breaking the examples in this section, because the dimensions we use are so small. In practice, security of these cryptosystems require lattices of dimension around $500 - 1000$, which lead to impractical key lengths (except for NTRUEncrypt).

3.7.1 Application to GGH Cryptosystem

We now illustrate how Eve can attack the GGH cryptosystem using the LLL algorithm followed by Babai's algorithm.

Recall Example 3.15: Alice chooses the following good private basis:

$$v_1 = (-97, 19, 19), \qquad v_2 = (-36, 30, 86), \qquad v_3 = (-184, -64, 78).$$

These vectors form a relatively short and nearly orthogonal basis of the lattice L, with Hadamard ratio approximately 0.74. Also, Alice publishes a public basis:

$$w_1 = (-4179163, -1882253, 583183), w_2 = (-3184353, -1434201, 444361),$$
$$w_3 = (-5277320, -2376852, 736426),$$

which generates the same lattice:

$$L = \text{span}_{\mathbb{Z}}\{v_1, v_2, v_3\} = \text{span}_{\mathbb{Z}}\{w_1, w_2, w_3\}.$$

Bob's encrypted message is

$$e = (-79081427, -35617462, 11035473).$$

Eve's goal is to recover the lattice vector in L that is closest to e, i.e. solve a closest vector problem.

- Eve first applies the LLL algorithm to the public basis $\{w_1, w_2, w_3\}$. LLL produces the following quasi-orthogonal basis:

  $$b_1 = (36, -30, -86), \qquad b_2 = (61, 11, 67), \qquad b_3 = (-10, 102, -40).$$

  This basis generates exactly the same lattice L, but is much better suited to solving the CVP, since its Hadamard ratio is approximately 0.956083, which is closer to 1 than that of the private key.

- It is worth noting that
  $$b_1 = (36, -30, -86) = -v_2,$$
  so the LLL reduction has already produced one of Alice's private basis vectors, up to sign.

More generally, Eve does not need to recover Alice's exact private basis; she only needs a sufficiently good basis of L for Babai's algorithm to succeed.

- Eve next applies Babai's algorithm to the ciphertext e using the LLL-reduced basis. Babai's algorithm produces the lattice vector

$$v = (-79081423, -35617459, 11035471).$$

This vector is very close to the ciphertext since

$$e = v + (-4, -3, 2),$$

where $(-4, -3, 2)$ is the small error vector introduced during encryption in this example.

- Eve has therefore successfully recovered the lattice vector corresponding to Bob's plaintext. To recover the plaintext itself, she expresses v in terms of Alice's public basis:

$$v = 86w_1 - 35w_2 - 32w_3.$$

Hence the corresponding plaintext vector is

$$m = (86, -35, -32).$$

Therefore, Eve successfully breaks the GGH encryption by first using LLL to transform Alice's "bad" public basis into a much better basis, and then applying Babai's algorithm to find the lattice vector closest to the ciphertext. Finally, she expresses this lattice vector in terms of the original public basis to recover the plaintext:

$$\boxed{\text{Public Basis} \xrightarrow{\text{LLL}} \text{Quasi-orthogonal Basis} \xrightarrow{\text{Babai}} \text{Closest Lattice Vector} \xrightarrow{\text{Change of Basis}} \text{Plaintext.}}$$

We can easily verify these results by combining the code in Appendices A.1 and A.5.

This example demonstrates the vulnerability of the original GGH construction: although the public basis is intended to be difficult to work with, LLL can sometimes find a sufficiently good basis to make approximate CVP solving feasible.

3.7.2 Application to NTRUEncrypt

We now show how Eve attacks the NTRUEncrypt cryptosystem using the LLL algorithm. Recall Example 3.28: Here we let $(N, p, q, d) = (7, 3, 41, 2)$ and Alice chooses private keys:

$$f(x) = x^6 - x^4 + x^3 + x^2 - 1, \quad g(x) = x^6 + x^4 - x^2 - x,$$

while the chosen public key is

$$h(x) = 20x^6 + 40x^5 + 2x^4 + 38x^3 + 8x^2 + 26x + 30.$$

The message is $m(x) = -x^5 + x^3 + x^2 - x + 1$ and the small error is $r(x) = x^6 - x^5 + x - 1$. The ciphertext is then

$$e(x) \equiv p \cdot r(x) \star h(x) + m(x) \equiv 31x^6 + 19x^5 + 4x^4 + 2x^3 + 40x^2 + 3x + 25 \pmod{q}$$

Moreover, recalling Section 3.5.4 above, the associated NTRU lattice is generated by the rows of the matrix

$$M_h^{\mathrm{NTRU}} = \begin{pmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 30 & 26 & 8 & 38 & 2 & 40 & 20 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 20 & 30 & 26 & 8 & 38 & 2 & 40 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 40 & 20 & 30 & 26 & 8 & 38 & 2 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 2 & 40 & 20 & 30 & 26 & 8 & 38 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 38 & 2 & 40 & 20 & 30 & 26 & 8 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 8 & 38 & 2 & 40 & 20 & 30 & 26 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 26 & 8 & 38 & 2 & 40 & 20 & 30 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41
\end{pmatrix}.$$

Eve applies LLL reduction to M_h^{NTRU}. The algorithm performs 96 swap steps and returns the LLL-reduced matrix $M_{\mathrm{red}}^{\mathrm{NTRU}}$:

$$M_{\mathrm{red}}^{\mathrm{NTRU}} = \begin{pmatrix}
1 & 0 & -1 & 1 & 0 & -1 & -1 & -1 & 0 & -1 & 0 & 1 & 1 & 0 \\
0 & 1 & 1 & -1 & 0 & 1 & -1 & -1 & -1 & 0 & 1 & 0 & 1 & 0 \\
-1 & 1 & 0 & -1 & -1 & 1 & 0 & -1 & 0 & 1 & 1 & 0 & -1 & 0 \\
-1 & -1 & 1 & 0 & -1 & 1 & 0 & 1 & 0 & -1 & 0 & -1 & 0 & 1 \\
-1 & 1 & 0 & -1 & 1 & 0 & -1 & 0 & -1 & 0 & -1 & 0 & 1 & 1 \\
-1 & -1 & -1 & -1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 1 & -1 & 1 & 1 & -1 & -1 & 0 & 0 & 2 & 0 & 0 \\
-8 & -1 & 0 & 9 & 0 & -1 & 0 & -4 & 2 & 6 & 0 & -4 & 7 & -7 \\
8 & 1 & 0 & 0 & -8 & -1 & 2 & 0 & -5 & 8 & -7 & -3 & 7 & 6 \\
0 & -9 & -2 & -1 & -1 & 0 & -6 & -3 & 2 & 6 & 0 & -5 & 7 & 0 \\
0 & 8 & 0 & -9 & -1 & -8 & 8 & 2 & 7 & -11 & 3 & -5 & 2 & 2 \\
1 & 0 & 0 & 0 & -1 & -9 & 5 & -7 & 6 & 3 & -2 & -5 & -6 & 0 \\
-2 & 1 & 9 & -1 & 0 & -9 & 2 & 5 & 0 & -5 & 7 & -6 & -3 & 0 \\
3 & 2 & 3 & 3 & -6 & 2 & -6 & 11 & 6 & 8 & 0 & 9 & 5 & 2
\end{pmatrix}.$$

We can compare the relative quasi-orthogonality of the original and the reduced bases by computing their Hadamard ratios:

$$\mathcal{H}\left(M_h^{\mathrm{NTRU}}\right) = 0.1184, \qquad \mathcal{H}\left(M_{\mathrm{red}}^{\mathrm{NTRU}}\right) = 0.8574.$$

LLL does not certify that the shortest row of $M_{\mathrm{red}}^{\mathrm{NTRU}}$ is a copy of Alice's key: since h

comes from a cyclic convolution, every rotation $\pm x^i \star (f, g)$ has the same norm, so there is no unique minimal vector, and short vectors unrelated to the key can also appear by coincidence. For example, row 6, $(-1, -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, 0)$, has squared norm 7 (shortest row), but its second half is zero, so it cannot be a rotation of $g(x)$.

Eve instead tests each candidate row against the defining relation $g_i \equiv f_i \star h \pmod{q}$, discarding rows like row 6 that fail it.

Row 1, $(1, 0, -1, 1, 0, -1, -1, -1, 0, -1, 0, 1, 1, 0)$, passes this test. Splitting it into two pieces gives the polynomials $f'(x) = 1 - x^2 + x^3 - x^5 - x^6$ and $g'(x) = -1 - x^2 + x^4 + x^5$.

Note that $f'(x)$ and $g'(x)$ are not the same as Alice's original private key polynomials $f(x)$ and $g(x)$. However, they are simple rotations of Alice's key:

$$f'(x) = -x^3 \star f(x) \qquad \text{and} \qquad g'(x) = -x^3 \star g(x).$$

Since a rotation of a valid key pair is itself a valid key pair, Eve can use $f'(x)$ and $g'(x)$ to decrypt messages.

We may leave the Python check as an exercise :)

References

[1] Joseph A. Gallian. *Contemporary Abstract Algebra*. Brooks/Cole, Cengage Learning, 7th edition, 2010. Available online at: `https://www.ayman-badawi.com/2019%20%20Contemporary+Abstract+Algebra-7.pdf` (Chapter 9: Factor groups, Chapter 14: Factor rings).

[2] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. *An Introduction to Mathematical Cryptography*. Undergraduate Texts in Mathematics. Springer, 2nd edition, 2014. Available online at: `https://www.math.brown.edu/~jhs/MathCryptoHome.html`.

A Python Code

A.1 GGH Cryptosystem: Worked Example

```
# -*- coding: utf-8 -*-
"""GGH cryptosystem_ex .ipynb

Automatically generated by Colab.



"""

import numpy as np

#store the private key-good lattice
V = np.array([
    [-97, 19, 19],
    [-36, 30, 86],
    [-184, -64, 78]
])

print(V)

#calculate hadamard ratio of V
def hadamard_ratio(B):

    n = len(B) #dimension of L

    det = abs(np.linalg.det(B)) #determinant of L

    norms = np.linalg.norm(B,axis=1) #norms of each basis vector of L

    return (det / np.prod(norms))**(1/n)

print(hadamard_ratio(V))

#generate U and bad basis matrix
U = np.array([
    [4327, -15447, 23454],
    [3297, -11770, 17871],
    [5464, -19506, 29617]
], dtype=int)
print(round(np.linalg.det(U)))

W = U @ V
print(W)
print(hadamard_ratio(W))
```

```
45
46 m = np.array([86, -35, -32]) #plaintext
47 r = np.array([-4, -3, 2]) #small perturbation
48
49 #encryption
50 e = m @ W + r
51
52 print("Message:", m)
53 print("\nError:", r)
54 print("\nCiphertext:", e)
55
56 # babai's algorithm
57 def babai(B, target):
58
59     """
60     Babai Closest Vertex Algorithm
61
62     Input:
63         B      : lattice basis matrix
64         target : target vector w
65
66     Algorithm:
67         Solve:
68             target = t1*v1 + ... + tn*vn
69
70         Round:
71             ai = round(ti)
72
73         Return:
74             a1*v1 + ... + an*vn
75     """
76
77     # Find coefficients t
78     coeff_real = np.linalg.solve(B.T, target)
79
80     # Round coefficients
81     coeff = np.round(coeff_real).astype(int)
82
83     # Closest lattice vector
84     closest = coeff @ B
85
86     return closest, coeff
87
88 #Decryptionn
89 closest, coeff = babai(V,e)
90 print(closest== m @ W)
91 print("Babai coefficients:", coeff)
92 print("Closest lattice point:", closest)
93
```

```
m_recovered = closest @ np.linalg.inv(W)

m_recovered = np.round(m_recovered).astype(int)

print("Recovered message:", m_recovered)

print("Success:", np.all(m_recovered==m))

# Babai using private good basis V
closest_good, coeff_good =babai(V,e)


# Babai using public bad basis W

closest_bad, coeff_bad = babai(W,e)



distance_good = np.linalg.norm(
    e - closest_good
)


distance_bad = np.linalg.norm(
    e - closest_bad
)

print("\nUsing private good basis V:")

print("Closest lattice point:")
print(closest_good)

print("Distance to ciphertext:")
print(distance_good)


m_recovered_bad = closest_bad @ np.linalg.inv(W)

m_recovered_bad = np.round(m_recovered_bad).astype(int)

print("Recovered message by Eve:", m_recovered_bad)

print("Success (Eve) :", np.all(m_recovered_bad==m))

print("\nUsing public bad basis W:")

print("Closest lattice point:")
print(closest_bad)

```

```
print("Distance to ciphertext:")
print(distance_bad)


print("""
The private basis V is reduced and nearly orthogonal,
so Babai's algorithm can find the correct lattice point.

The public basis W is intentionally bad, therefore
Babai's algorithm is much less reliable.
""")
```

A.2 GGH Cryptosystem: Extension

```
# -*- coding: utf-8 -*-
"""GGH crypto .ipynb

Automatically generated by Colab.



"""

import numpy as np
import random

#calculate hadamard ratio
def hadamard_ratio(B):

    n = len(B) #dimension of L

    det = abs(np.linalg.det(B)) #determinant of L

    norms = np.linalg.norm(B,axis=1) #norms of each basis vector of L

    return (det / np.prod(norms))**(1/n)

#generate U randomly from GL(Z,n)
def generate_unimodular(n, operations=20, bound=5):

    U = np.eye(n, dtype=int)


    for _ in range(operations):

        operation = random.choice(
            ["swap", "add", "negate"]
        )

```

```
36
37          if operation == "swap":
38
39              i, j = random.sample(range(n),2)
40
41              U[[i,j]] = U[[j,i]]
42
43
44          elif operation == "add":
45
46              i, j = random.sample(range(n),2)
47
48              k = random.randint(-bound,bound)
49
50              U[i] += k*U[j]
51
52
53          elif operation == "negate":
54
55              i = random.randint(0,n-1)
56
57              U[i] *= -1
58
59
60      return U
61
62  #generate the public key of Alice
63  def generate_public_key(V, target_ratio=0.01):
64
65      while True:
66
67          U = generate_unimodular(len(V))
68
69          W = U @ V
70
71          if hadamard_ratio(W)<target_ratio:
72              return U,W
73
74  #baiba's algorithm
75  def babai(B, target):
76
77      """
78      Babai Closest Vertex Algorithm
79
80      Input:
81          B      : lattice basis matrix
82          target : target vector w
83
84      Algorithm:
```

```
85          Solve:
86              target = t1*v1 + ... + tn*vn
87
88          Round:
89              ai = round(ti)
90
91          Return:
92              a1*v1 + ... + an*vn
93      """
94
95      # Find coefficients t
96      coeff_real = np.linalg.solve(B.T, target)
97
98      # Round coefficients
99      coeff = np.round(coeff_real).astype(int)
100
101     # Closest lattice vector
102     closest = coeff @ B
103
104     return closest, coeff
105
106 # ====================================================
107 # Alice: Key Generation
108 # ====================================================
109
110 # Alice chooses a good private basis
111
112 V = np.array([
113     [-97, 19, 19],
114     [-36, 30, 86],
115     [-184, -64, 78]
116 ])
117
118
119 # Generate a bad public basis
120
121 U, W = generate_public_key(V)
122
123
124 print("==============================")
125 print("Alice's Key Generation")
126 print("==============================")
127
128 print("\nPrivate basis V:")
129 print(V)
130
131 print("\nHadamard ratio of V:")
132 print(round(hadamard_ratio(V),5))
133
```

```
134
135 print("\nUnimodular matrix U:")
136 print(U)
137
138 print("\ndet(U):")
139 print(round(np.linalg.det(U)))
140
141
142 print("\nPublic basis W:")
143 print(W)
144
145
146 print("\nHadamard ratio of W:")
147 print(round(hadamard_ratio(W),5))
148
149 # =======================================================
150 # Bob: Encryption
151 # =======================================================
152
153
154 def generate_message(n, bound=50):
155
156     return np.random.randint(
157         -bound,
158         bound+1,
159         size=n
160     )
161
162
163 def generate_error(n, bound=5):
164
165     return np.random.randint(
166         -bound,
167         bound+1,
168         size=n
169     )
170
171
172
173 m = generate_message(3)
174
175 r = generate_error(3)
176
177
178 # ciphertext
179
180 e = m @ W + r
181
182
```

```
183 print("\n==============================")
184 print("Bob's Encryption")
185 print("==============================")
186
187
188 print("\nMessage m:")
189 print(m)
190
191
192 print("\nError vector r:")
193 print(r)
194
195
196 print("\nCiphertext e:")
197 print(e)
198
199 # ====================================================
200 # Alice: Decryption
201 # ====================================================
202
203
204 closest, coeff = babai(V,e)
205
206
207 m_recovered = closest @ np.linalg.inv(W)
208
209 m_recovered = np.round(
210     m_recovered
211 ).astype(int)
212
213
214
215 print("\n==============================")
216 print("Alice's Decryption")
217 print("==============================")
218
219
220 print("\nBabai coefficients:")
221 print(coeff)
222
223
224 print("\nClosest lattice point:")
225 print(closest)
226
227
228 print("\nRecovered message:")
229 print(m_recovered)
230
231
```

```
232 print("\nSuccessful decryption:")
233 print(np.all(m_recovered==m))
234
235 # ====================================================
236 # Final Report
237 # ====================================================
238
239 print("\n\n=======================================")
240 print("          GGH Cryptosystem Report")
241 print("=======================================")
242
243 print("\n---------------------------------------")
244 print("1. Key Generation")
245 print("---------------------------------------")
246
247
248 print("\nPrivate good basis V:")
249 print(V)
250
251
252 print("\nHadamard ratio of private basis:")
253 print(round(hadamard_ratio(V),5))
254
255
256 print("\nUnimodular matrix U:")
257 print(U)
258
259
260 print("\ndet(U):")
261 print(round(np.linalg.det(U)))
262
263
264 print("\nPublic bad basis W = UV:")
265 print(W)
266
267
268 print("\nHadamard ratio of public basis:")
269 print(round(hadamard_ratio(W),5))
270
271
272 print("\nComparison:")
273 print(
274     "Private basis H(V) =",
275     round(hadamard_ratio(V),5),
276     " (good basis)"
277 )
278
279 print(
280     "Public basis H(W) =",
```

```
    round(hadamard_ratio(W),5),
    " (bad basis)"
)


print("\n--------------------------------------")
print("2. Encryption")
print("--------------------------------------")


print("\nMessage m:")
print(m)


print("\nError vector r:")
print(r)


print("\nCiphertext:")
print(e)


print("\nEncryption formula:")
print("e = mW + r")


print("\n--------------------------------------")
print("3. Decryption")
print("--------------------------------------")


print("\nBabai coefficients:")
print(coeff)


print("\nClosest lattice point:")
print(closest)


print("\nRecovered message:")
print(m_recovered)


print("\nDecryption successful:")
print(np.all(m_recovered==m))

# ====================================================
# Additional Experiments and Verification
# ====================================================
```

```
330
331
332 print("\n\n====================================")
333 print("     Additional Experiments")
334 print("====================================")
335
336
337 # ----------------------------------------------------
338 # 1. Verify that V and W generate the same lattice
339 # ----------------------------------------------------
340
341 print("\n--------------------------------------")
342 print("1. Same Lattice Verification")
343 print("--------------------------------------")
344
345
346 det_V = abs(round(np.linalg.det(V)))
347 det_W = abs(round(np.linalg.det(W)))
348
349
350 print("det(V) =", det_V)
351 print("det(W) =", det_W)
352
353
354 print(
355     "Same lattice determinant:",
356     abs(det_V) == abs(det_W)
357 )
358
359
360 # ----------------------------------------------------
361 # 2. Compare Babai using good and bad basis
362 # ----------------------------------------------------
363
364 print("\n--------------------------------------")
365 print("2. Good Basis vs Bad Basis")
366 print("--------------------------------------")
367
368
369 # Babai using private good basis V
370
371 closest_good, coeff_good = babai(V,e)
372
373
374 # Babai using public bad basis W
375
376 closest_bad, coeff_bad = babai(W,e)
377
378
```

```
379
380 distance_good = np.linalg.norm(
381     e - closest_good
382 )
383
384
385 distance_bad = np.linalg.norm(
386     e - closest_bad
387 )
388
389 print("\nUsing private good basis V:")
390
391 print("Closest lattice point:")
392 print(closest_good)
393
394 print("Distance to ciphertext:")
395 print(distance_good)
396
397
398
399 print("\nUsing public bad basis W:")
400
401 print("Closest lattice point:")
402 print(closest_bad)
403
404 print("Distance to ciphertext:")
405 print(distance_bad)
406
407
408 print("""
409 The private basis V is reduced and nearly orthogonal,
410 so Babai's algorithm can find the correct lattice point.
411
412 The public basis W is intentionally bad, therefore
413 Babai's algorithm is much less reliable.
414 """)
415
416
417 # ------------------------------------------------------
418 # 3. Effect of error size
419 # ------------------------------------------------------
420
421 print("\n--------------------------------------")
422 print("3. Effect of Error Size")
423 print("--------------------------------------")
424
425
426 error_sizes = [
427     1,
```

```
    5,
    10,
    50,
    100,
    500,
    1000
]


for size in error_sizes:


    r_test = np.array(
        [size,size,size]
    )


    # encryption

    e_test = m @ W + r_test


    # decryption

    closest_test, _ = babai(
        V,
        e_test
    )


    recovered_test = np.round(
        closest_test @ np.linalg.inv(W)
    ).astype(int)



    success = np.all(
        recovered_test == m
    )


    print(
        "Error size:",
        size,
        " | Success:",
        success
    )

#check for several trials
```

```
for i in range(30):

    U = generate_unimodular(3)

    W = U @ V

    m = generate_message(3)

    r = generate_error(3,bound=3)

    e = m @ W + r


    closest,coeff=babai(V,e)


    recovered=np.round(
        closest @ np.linalg.inv(W)
    ).astype(int)


    print(
        "Trial",
        i+1,
        ":",
        np.all(recovered==m)
    )

#strings to number
def string_to_numbers(message):

    return np.array(
        [ord(c) for c in message]
    )

#make the array fit into 3 dimensional: output a list of vectors
def split_message(numbers, dimension):

    blocks=[]

    for i in range(0,len(numbers),dimension):

        block=numbers[i:i+dimension]

        if len(block)<dimension:

            block=np.pad(
                block,
                (0,dimension-len(block))
```

```
            )

        blocks.append(block)

    return blocks

#encrytp messages
def encrypt(messages, W, error_bound=5):
    """
    GGH encryption

    Input:
        messages: list of message vectors
        W: public basis
        error_bound: size of error

    Output:
        ciphertext list
    """

    ciphertext=[]

    n=len(W)


    for m in messages:

        r=generate_error(3)


        e=m@W+r

        ciphertext.append(e)


    return ciphertext

def numbers_to_string(vectors):

    numbers = []

    for vector in vectors:

        for x in vector:

            x = int(x)

            # remove padding zeros
            if x != 0:
```

```
                numbers.append(x)


    return ''.join(
        chr(x)
        for x in numbers
    )


def decrypt(ciphertexts, V, W):

    recovered=[]


    for e in ciphertexts:


        closest, coeff = babai(
            V,
            e
        )


        m = closest @ np.linalg.inv(W)


        m=np.round(m).astype(int)


        recovered.append(m)


    return recovered

messages = split_message(
    string_to_numbers("Lattice-Based Cryptography"),
    3
)

print(messages)

#good basis
V = np.array([
    [-97, 19, 19],
    [-36, 30, 86],
    [-184, -64, 78]
])


```

```
# Generate a bad public basis

U, W = generate_public_key(V)

ciphertexts = encrypt(messages,W)


for c in ciphertexts:
    print(c)

numbers = decrypt(
    ciphertexts,
    V,
    W
)

print(numbers)


plaintext = numbers_to_string(numbers)


print(plaintext)

print("Original Message:")
print("Lattice-Based Cryptography")
print("-------------------------------------")

print("The array of message (input):")
print(messages)
print("-------------------------------------")

print("Encrypted ciphertext:")
print(ciphertexts)
print("-------------------------------------")

print("Decrypted result:")
print(numbers)
print("-------------------------------------")

print("Plaintext:")
print(plaintext)
```

A.3 NTRU Cryptosystem: Worked Example

```
# -*- coding: utf-8 -*-
"""ntru crypto.ipynb

Automatically generated by Colab.

```

```
6
7
8 """
9
10 import random
11 from sympy import symbols, Poly, invert
12
13 # =============================================================
14 # NTRUEncrypt
15 # =============================================================
16
17 x = symbols("x")
18
19
20 # -------------------------------------------------------------
21 # Polynomial operations in
22 #
23 #       R = Z[x] / <x^N - 1>
24 #
25 # -------------------------------------------------------------
26
27 def convolution(a, b):
28     """
29     Cyclic convolution:
30
31         a star b  in Z[x] / <x^N - 1>.
32
33     Polynomials are represented by coefficient lists:
34         [a_0, a_1, ..., a_{N-1}]
35     """
36
37     N = len(a)
38     result = [0] * N
39
40     for i in range(N):
41         for j in range(N):
42             result[(i + j) % N] += a[i] * b[j]
43
44     return result
45
46
47 def add(a, b):
48     """Add two polynomials."""
49     return [u + v for u, v in zip(a, b)]
50
51
52 def multiply_scalar(c, a):
53     """Multiply a polynomial by an integer c."""
```

```
54      return [c * u for u in a]
55
56
57  def mod_poly(a, modulus):
58      """Reduce every coefficient modulo modulus."""
59      return [u % modulus for u in a]
60
61
62  def centre_lift(a, modulus):
63      """
64      Centre-lift coefficients modulo modulus.
65
66      The representatives lie in
67
68          (-modulus/2, modulus/2].
69      """
70
71      result = []
72
73      for u in a:
74          u = u % modulus
75
76          if u > modulus / 2:
77              u -= modulus
78
79          result.append(u)
80
81      return result
82
83
84  # ------------------------------------------------------------
85  # Ternary polynomials
86  # ------------------------------------------------------------
87
88  def ternary(d1, d2, N):
89      """
90      Generate a random polynomial in T(d1, d2).
91
92      d1 coefficients are +1,
93      d2 coefficients are -1,
94      all remaining coefficients are 0.
95      """
96
97      if d1 + d2 > N:
98          raise ValueError("d1 + d2 cannot exceed N.")
99
100     coefficients = (
101         [1] * d1
102         + [-1] * d2
```

```
103         + [0] * (N - d1 - d2)
104     )
105
106     random.shuffle(coefficients)
107
108     return coefficients
109
110
111 # ------------------------------------------------------------
112 # Polynomial inverse
113 # ------------------------------------------------------------
114
115 def ntru_inverse(f, N, modulus):
116     """
117     Compute the inverse of f(x) in
118
119         (Z/modulus Z)[x] / <x^N - 1>.
120
121     That is, find F such that
122
123         f star F = 1 (mod modulus).
124
125     The polynomial f is given as a coefficient list.
126     """
127
128     # Convert coefficient list to SymPy polynomial
129     f_poly = sum(
130         coefficient * x**i
131         for i, coefficient in enumerate(f)
132     )
133
134     # Polynomial x^N - 1
135     ring_polynomial = x**N - 1
136
137     # Compute the inverse over Z/modulus Z
138     try:
139         inverse = invert(
140             Poly(f_poly, x, modulus=modulus),
141             Poly(ring_polynomial, x, modulus=modulus)
142         )
143     except Exception as error:
144         raise ValueError(
145             f"f is not invertible modulo {modulus}."
146         ) from error
147
148     # Convert back to coefficient list
149     inverse = inverse.as_expr()
150
151     return [
```

```
        int(inverse.coeff(x, i)) % modulus
        for i in range(N)
    ]


# ------------------------------------------------------------
# Polynomial display
# ------------------------------------------------------------

def polynomial_string(a):
    """Convert a coefficient list into a readable polynomial."""

    terms = []

    for i, coefficient in enumerate(a):

        if coefficient == 0:
            continue

        absolute = abs(coefficient)

        # Construct the term
        if i == 0:
            term = str(absolute)

        elif i == 1:
            if absolute == 1:
                term = "x"
            else:
                term = f"{absolute}x"

        else:
            if absolute == 1:
                term = f"x^{i}"
            else:
                term = f"{absolute}x^{i}"

        # Add sign
        if not terms:
            if coefficient < 0:
                terms.append("-" + term)
            else:
                terms.append(term)

        else:
            if coefficient > 0:
                terms.append(" + " + term)
            else:
                terms.append(" - " + term)
```

```
201
202     return "".join(terms) if terms else "0"
203
204
205 def print_polynomial(name, a):
206     """Print a polynomial."""
207     print(f"{name}(x) = {polynomial_string(a)}")
208
209 # ============================================================
210 # NTRU KEY GENERATION
211 # ============================================================
212
213 def generate_keys(N, p, q, d):
214     """
215     Generate an NTRU key pair.
216
217     Returns
218
219         f, g, Fp, Fq, h
220
221     where
222
223         f in T(d+1,d)
224         g in T(d,d)
225
226         Fp = f^(-1) mod p
227         Fq = f^(-1) mod q
228
229         h = Fq star g mod q
230     """
231
232     while True:
233
234         f = ternary(d + 1, d, N)
235         g = ternary(d, d, N)
236
237         try:
238             Fp = ntru_inverse(f, N, p)
239             Fq = ntru_inverse(f, N, q)
240             break
241
242         except ValueError:
243             # f was not invertible modulo p or q
244             continue
245
246     h = mod_poly(
247         convolution(Fq, g),
248         q
249     )
```

```
250
251     return f, g, Fp, Fq, h
252
253
254 # ============================================================
255 # NTRU ENCRYPTION
256 # ============================================================
257
258 def encrypt(m, r, h, p, q):
259     """
260     Encrypt
261
262         e = p h star r + m mod q.
263     """
264
265     hr = convolution(h, r)
266
267     e = add(
268         multiply_scalar(p, hr),
269         m
270     )
271
272     return mod_poly(e, q)
273
274
275 # ============================================================
276 # NTRU DECRYPTION
277 # ============================================================
278
279 def decrypt(e, f, Fp, p, q):
280     """
281     Decrypt an NTRU ciphertext.
282
283     Step 1:
284         a = f star e mod q
285
286     Step 2:
287         centre-lift a to a'
288
289     Step 3:
290         b = Fp star a' mod p
291
292     Step 4:
293         centre-lift b to recover m
294     """
295
296     # Step 1
297     a = mod_poly(
298         convolution(f, e),
```

```
        q
    )

    # Step 2
    a_prime = centre_lift(a, q)

    # Step 3
    b = mod_poly(
        convolution(Fp, a_prime),
        p
    )

    # Step 4
    m_recovered = centre_lift(b, p)

    return m_recovered, a, a_prime, b


# ============================================================
# VERIFICATION
# ============================================================

def verify_keys(f, g, Fp, Fq, h, p, q):
    """
    Verify the key identities:

        f star Fp = 1 mod p
        f star Fq = 1 mod q
        f star h  = g mod q
    """

    N = len(f)

    identity = [1] + [0] * (N - 1)

    check_Fp = mod_poly(
        convolution(f, Fp),
        p
    )

    check_Fq = mod_poly(
        convolution(f, Fq),
        q
    )

    check_h = mod_poly(
        convolution(f, h),
        q
    )
```

```
348
349     check_g = mod_poly(g, q)
350
351     return (
352         check_Fp == identity
353         and check_Fq == identity
354         and check_h == check_g
355     )
356
357 # ============================================================
358 # EXAMPLE
359 #
360 # Change ONLY this section when using another example.
361 # ============================================================
362
363 N = 7
364 p = 3
365 q = 41
366 d = 2
367
368
369 # Example 7.53:
370 #
371 # f(x) = x^6 - x^4 + x^3 + x^2 - 1
372 #
373 # coefficient order:
374 # [1, x, x^2, ..., x^6]
375
376 f = [
377     -1,
378      0,
379      1,
380      1,
381     -1,
382      0,
383      1
384 ]
385
386
387 # g(x) = x^6 + x^4 - x^2 - x
388
389 g = [
390      0,
391     -1,
392     -1,
393      0,
394      1,
395      0,
396      1
```

```
]


# ------------------------------------------------------------
# Compute inverses
# ------------------------------------------------------------

Fp = ntru_inverse(f, N, p)
Fq = ntru_inverse(f, N, q)


# ------------------------------------------------------------
# Public key
# ------------------------------------------------------------

h = mod_poly(
    convolution(Fq, g),
    q
)


# ------------------------------------------------------------
# Bob's message
#
# m(x) = -x^5 + x^3 + x^2 - x + 1
# ------------------------------------------------------------

m = [
     1,
    -1,
     1,
     1,
     0,
    -1,
     0
]


# ------------------------------------------------------------
# Bob's random polynomial
#
# r(x) = x^6 - x^5 + x - 1
# ------------------------------------------------------------

r = [
    -1,
     1,
     0,
     0,
```

```
     0,
    -1,
     1
]


# ============================================================
# RUN NTRU
# ============================================================

print("=" * 65)
print("NTRUEncrypt")
print("=" * 65)

print("\nParameters:")
print(f"N = {N}")
print(f"p = {p}")
print(f"q = {q}")
print(f"d = {d}")

print("\nCorrectness condition:")
print(f"q > (6d+1)p")
print(f"{q} > {(6*d + 1)*p}")


# ------------------------------------------------------------
# Keys
# ------------------------------------------------------------

print("\n" + "-" * 65)
print("KEY GENERATION")
print("-" * 65)

print_polynomial("f", f)
print_polynomial("g", g)
print_polynomial("Fp", Fp)
print_polynomial("Fq", Fq)
print_polynomial("h", h)


# Verify keys
print("\nKey verification:")
print(
    "f star Fp = 1 mod p, "
    "f star Fq = 1 mod q, "
    "f star h = g mod q"
)

print(
```

```
    "All identities verified:",
    verify_keys(f, g, Fp, Fq, h, p, q)
)


# ------------------------------------------------------------
# Encryption
# ------------------------------------------------------------

print("\n" + "-" * 65)
print("ENCRYPTION")
print("-" * 65)

print_polynomial("m", m)
print_polynomial("r", r)

e = encrypt(
    m,
    r,
    h,
    p,
    q
)

print("\nCiphertext:")
print_polynomial("e", e)


# ------------------------------------------------------------
# Decryption
# ------------------------------------------------------------

print("\n" + "-" * 65)
print("DECRYPTION")
print("-" * 65)

m_recovered, a, a_prime, b = decrypt(
    e,
    f,
    Fp,
    p,
    q
)

print_polynomial("a", a)
print_polynomial("a'", a_prime)
print_polynomial("b", b)
print_polynomial("m recovered", m_recovered)

```

```
544
545 # ------------------------------------------------------------
546 # Final verification
547 # ------------------------------------------------------------
548
549 print("\n" + "-" * 65)
550 print("FINAL RESULT")
551 print("-" * 65)
552
553 print_polynomial("original m", m)
554 print_polynomial("recovered m", m_recovered)
555
556 print("\nSuccessful decryption:", m_recovered == m)
```

A.4 Gaussian Lattice Reduction Algorithm

```
1  # -*- coding: utf-8 -*-
2  """Gaussian LRA dim2.ipynb
3
4  Automatically generated by Colab.
5
6
7
8  """
9
10 import numpy as np
11
12 #general guassian lattice reduction
13 def gauss_reduce(v1, v2):
14     v1 = np.array(v1, dtype=object)
15     v2 = np.array(v2, dtype=object)
16
17     while True:
18         # Ensure ||v1|| <= ||v2||
19         if np.dot(v2, v2) < np.dot(v1, v1):
20             v1, v2 = v2, v1
21
22         # Compute the nearest integer multiplier
23         m = round(np.dot(v1, v2) / np.dot(v1, v1))
24
25         # The basis is Gauss-reduced
26         if m == 0:
27             break
28
29         # Reduce v2
30         v2 = v2 - m * v1
31
32     return v1, v2
33
```

```

v1 = (66586820, 65354729)
v2 = (6513996, 6393464)

v1_reduced, v2_reduced = gauss_reduce(v1, v2)

print(v1_reduced)
print(v2_reduced)

#gaussian with each steps
def show_gauss_steps(v1, v2):
    v1 = np.array(v1, dtype=object)
    v2 = np.array(v2, dtype=object)

    step = 1

    while True:
        # Make v1 the shorter vector
        if np.dot(v2, v2) < np.dot(v1, v1):
            v1, v2 = v2, v1

        # Compute m
        m = round(np.dot(v1, v2) / np.dot(v1, v1))

        print(f"Step {step}:")
        print(f"  v1 = {tuple(v1)}")
        print(f"  v2 = {tuple(v2)}")
        print(f"  m  = {m}")
        print()

        # Stop when m = 0
        if m == 0:
            break

        # Reduce v2
        v2 = v2 - m * v1
        step += 1

v1 = (66586820, 65354729)
v2 = (6513996, 6393464)

show_gauss_steps(v1, v2)
```

A.5 LLL Lattice Reduction Algorithm

```
# -*- coding: utf-8 -*-
"""LLL lattice reduced.ipynb

Automatically generated by Colab.

```

```
6
7
8 """
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12
13 def gram_schmidt(B):
14     """
15     Compute the Gram--Schmidt vectors v_iˆ* and
16     coefficients mu_{i,j}.
17     """
18     n = len(B)
19
20     B_star = np.zeros_like(B, dtype=float)
21     mu = np.zeros((n, n), dtype=float)
22
23     for i in range(n):
24         B_star[i] = B[i].astype(float)
25
26         for j in range(i):
27             mu[i, j] = (
28                 np.dot(B[i], B_star[j])
29                 / np.dot(B_star[j], B_star[j])
30             )
31
32             B_star[i] -= mu[i, j] * B_star[j]
33
34     return B_star, mu
35
36 def lll(B, delta=3/4):
37     """
38     Implement the LLL algorithm.
39
40     The basis vectors are given by the rows of B.
41     """
42     B = np.array(B, dtype=int).copy()
43     n = len(B)
44
45     # Step [2]
46     k = 1       # Python indexing: k=1 corresponds to k=2
           mathematically
47
48     # Step [3]
49     B_star, mu = gram_schmidt(B)
50
51     swaps = 0
52     k_values = []
```

```
53
54      # Step [4]
55      while k < n:
56
57          # Record the mathematical value of k
58          k_values.append(k + 1)
59
60          # Steps [5]--[7]: Size reduction
61          for j in range(k - 1, -1, -1):
62
63              q = int(np.floor(mu[k, j] + 0.5))
64
65              B[k] = B[k] - q * B[j]
66
67              # Update Gram--Schmidt data
68              B_star, mu = gram_schmidt(B)
69
70          # Step [8]: Lovasz condition
71          lhs = np.dot(B_star[k], B_star[k])
72
73          rhs = ( delta - mu[k, k - 1]**2) * np.dot(B_star[k - 1],
                B_star[k - 1])
74
75          # Step [9]
76          if lhs >= rhs:
77              k += 1
78
79          # Steps [10]--[13]
80          else:
81              B[[k - 1, k]] = B[[k, k - 1]]
82
83              B_star, mu = gram_schmidt(B)
84
85              swaps += 1
86
87              # max(k-1,2) in mathematical indexing
88              k = max(k - 1, 1)
89
90      # Step [14]
91      return B, swaps, k_values
92
93  #calculate hadamard ratio
94  def hadamard_ratio(B):
95
96      n = len(B) #dimension of L
97
98      det = abs(np.linalg.det(B)) #determinant of L
99
100     norms = np.linalg.norm(B,axis=1) #norms of each basis vector of L
```

```
101
102     return (det / np.prod(norms))**(1/n)
103
104 #calculate shortest basis vector for a basis matrix
105 def shortest_basis_vector(B):
106     lengths = [np.linalg.norm(v) for v in B]
107     return min(lengths)
108
109 M = np.array([
110     [19,  2, 32, 46,  3, 33],
111     [15, 42, 11,  0,  3, 24],
112     [43, 15,  0, 24,  4, 16],
113     [20, 44, 44,  0, 18, 15],
114     [ 0, 48, 35, 16, 31, 31],
115     [48, 33, 32,  9,  1, 29]
116 ])
117
118 M_LLL, swaps, k_values = lll(M)
119
120 print("LLL-reduced basis:")
121 print(M_LLL)
122
123 print("\nNumber of swap steps:", swaps)
124
125 print("\nSequence of k values:")
126 print(k_values)
127
128 # compare hadamard ratio before and after LLL
129 print("\nHadamard ratio of M:")
130 print(round(hadamard_ratio(M),5))
131
132 print("\nHadamard ratio of M after LLL:")
133 print(round(hadamard_ratio(M_LLL),5))
134
135 # compare the shortest basis vector before and after LLL
136 original_min = shortest_basis_vector(M)
137 reduced_min = shortest_basis_vector(M_LLL)
138
139 print("\nShortest vector before LLL:")
140 print(original_min)
141 print("\nShortest vector after LLL: ")
142 print(reduced_min)
143
144 # Change the ordering of the basis vectors
145 M_reverse = M[::-1]
146
147 M_LLL_original, _, _ = lll(M, delta=3/4)
148 M_LLL_reverse, _, _ = lll(M_reverse, delta=3/4)
149
```

```
150 # Change the Lovasz parameter
151 M_LLL_099, _, _ = lll(M, delta=0.99)
152
153 print("LLL with original ordering:")
154 print(M_LLL_original)
155
156 print("\nLLL with reversed ordering:")
157 print(M_LLL_reverse)
158
159 print("\nLLL with delta = 0.99:")
160 print(M_LLL_099)
161
162 # ============================================================
163 # Comparison tests
164 # ============================================================
165
166 # ------------------------------------------------------------
167 # 1. Original ordering vs reversed ordering
168 # ------------------------------------------------------------
169
170 M_reverse = M[::-1]
171
172 M_LLL_original, swaps_original, k_original = lll(M, delta=3/4)
173 M_LLL_reverse, swaps_reverse, k_reverse = lll(M_reverse, delta=3/4)
174
175 print("=" * 60)
176 print("1. Effect of changing the ordering")
177 print("=" * 60)
178
179 print("\nOriginal ordering:")
180 print("Number of swaps:", swaps_original)
181 print("Hadamard ratio:", round(hadamard_ratio(M_LLL_original), 5))
182 print("Shortest basis vector:",
183       round(shortest_basis_vector(M_LLL_original), 5))
184
185 print("\nReversed ordering:")
186 print("Number of swaps:", swaps_reverse)
187 print("Hadamard ratio:", round(hadamard_ratio(M_LLL_reverse), 5))
188 print("Shortest basis vector:",
189       round(shortest_basis_vector(M_LLL_reverse), 5))
190
191
192 # ------------------------------------------------------------
193 # 2. Compare the k-sequences
194 # ------------------------------------------------------------
195
196 print("\n" + "=" * 60)
197 print("2. Sequence of k values")
198 print("=" * 60)
```

```
199
200 print("\nOriginal ordering:")
201 print(k_original)
202
203 print("\nReversed ordering:")
204 print(k_reverse)
205
206
207 # ------------------------------------------------------------
208 # 3. Effect of changing delta
209 # ------------------------------------------------------------
210
211 print("\n" + "=" * 60)
212 print("3. Effect of changing the Lovasz parameter delta")
213 print("=" * 60)
214
215 deltas = [0.75, 0.85, 0.90, 0.95, 0.99]
216
217 print(f"{'delta':<10}{'swaps':<10}{'H(B)':<15}{'shortest vector':<20}"
      )
218 print("-" * 55)
219
220 for delta in deltas:
221
222     B_LLL, swaps, k_values = lll(M, delta=delta)
223
224     H = hadamard_ratio(B_LLL)
225     shortest = shortest_basis_vector(B_LLL)
226
227     print(f"{delta:<10.2f}"
228           f"{swaps:<10}"
229           f"{H:<15.5f}"
230           f"{shortest:<20.5f}")
231
232
233 # ------------------------------------------------------------
234 # 4. Compare determinant before and after reduction
235 # ------------------------------------------------------------
236
237 print("\n" + "=" * 60)
238 print("4. Determinant preservation")
239 print("=" * 60)
240
241 det_original = abs(round(np.linalg.det(M)))
242 det_reduced = abs(round(np.linalg.det(M_LLL_original)))
243
244 print("Original determinant:", det_original)
245 print("Reduced determinant :", det_reduced)
246 print("Determinant preserved:", det_original == det_reduced)
```

```
247
248
249 # ------------------------------------------------------------
250 # 5. Overall comparison
251 # ------------------------------------------------------------
252
253 print("\n" + "=" * 60)
254 print("5. Overall comparison")
255 print("=" * 60)
256
257 print(f"{'Case':<25}"
258       f"{'delta':<10}"
259       f"{'swaps':<10}"
260       f"{'H(B)':<12}"
261       f"{'shortest':<15}")
262
263 print("-" * 72)
264
265 # Original ordering
266 print(f"{'Original ordering':<25}"
267       f"{'0.75':<10}"
268       f"{swaps_original:<10}"
269       f"{hadamard_ratio(M_LLL_original):<12.5f}"
270       f"{shortest_basis_vector(M_LLL_original):<15.5f}")
271
272 # Reversed ordering
273 print(f"{'Reversed ordering':<25}"
274       f"{'0.75':<10}"
275       f"{swaps_reverse:<10}"
276       f"{hadamard_ratio(M_LLL_reverse):<12.5f}"
277       f"{shortest_basis_vector(M_LLL_reverse):<15.5f}")
278
279 # Delta = 0.99
280 M_LLL_099, swaps_099, k_099 = lll(M, delta=0.99)
281
282 print(f"{'Original ordering':<25}"
283       f"{'0.99':<10}"
284       f"{swaps_099:<10}"
285       f"{hadamard_ratio(M_LLL_099):<12.5f}"
286       f"{shortest_basis_vector(M_LLL_099):<15.5f}")
287
288 # ============================================================
289 # Investigating the effect of delta
290 # ============================================================
291
292 # Use a range of delta values
293 deltas = np.linspace(0.51, 0.99, 49)
294
295 hadamard_values = []
```

```
shortest_values = []
swap_values = []

for delta in deltas:

    B_LLL, swaps, k_values = lll(M, delta=delta)

    hadamard_values.append(hadamard_ratio(B_LLL))
    shortest_values.append(shortest_basis_vector(B_LLL))
    swap_values.append(swaps)


# ------------------------------------------------------------
# 1. Delta vs Hadamard ratio
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

plt.plot(deltas, hadamard_values, marker='o', markersize=3)

plt.axvline(3/4, linestyle='--',
            label=r'$\delta=3/4$')

plt.xlabel(r'$\delta$')
plt.ylabel(r'$H(B)$')
plt.title(r'Effect of $\delta$ on the Hadamard ratio $H(B)$')
plt.legend()
plt.grid(True)

plt.show()


# ------------------------------------------------------------
# 2. Delta vs shortest basis vector
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

plt.plot(deltas, shortest_values, marker='o', markersize=3)

plt.axvline(3/4, linestyle='--',
            label=r'$\delta=3/4$')

plt.xlabel(r'$\delta$')
plt.ylabel('Shortest basis vector norm')
plt.title(r'Effect of $\delta$ on the shortest basis vector')
plt.legend()
plt.grid(True)

```

```
345 plt.show()
346
347
348 # ------------------------------------------------------------
349 # 3. Delta vs number of swaps
350 # ------------------------------------------------------------
351
352 plt.figure(figsize=(8, 5))
353
354 plt.plot(deltas, swap_values, marker='o', markersize=3)
355
356 plt.axvline(3/4, linestyle='--',
357             label=r'$\delta=3/4$')
358
359 plt.xlabel(r'$\delta$')
360 plt.ylabel('Number of swaps')
361 plt.title(r'Effect of $\delta$ on the number of LLL swaps')
362 plt.legend()
363 plt.grid(True)
364
365 plt.show()
366
367
368 # ============================================================
369 # Investigating the relationship between swaps and basis quality
370 # ============================================================
371
372 # ------------------------------------------------------------
373 # 4. Number of swaps vs Hadamard ratio
374 # ------------------------------------------------------------
375
376 plt.figure(figsize=(8, 5))
377
378 plt.scatter(swap_values, hadamard_values)
379
380 plt.xlabel('Number of swaps')
381 plt.ylabel(r'$H(B)$')
382 plt.title(r'Relationship between swaps and Hadamard ratio')
383 plt.grid(True)
384
385 plt.show()
386
387
388 # ------------------------------------------------------------
389 # 5. Number of swaps vs shortest basis vector
390 # ------------------------------------------------------------
391
392 plt.figure(figsize=(8, 5))
393
```

```
plt.scatter(swap_values, shortest_values)

plt.xlabel('Number of swaps')
plt.ylabel('Shortest basis vector norm')
plt.title(r'Relationship between swaps and shortest basis vector')
plt.grid(True)

plt.show()
```